2026-2 중간고사 대비

데이터베이스 · 중간 학습 노트

단원 6개 · 레슨 29개 · 트레이너 학습 탭과 같은 내용

U1 DB 시스템 개념 · 아키텍처

L1-1 · DB · DBMS · DB 시스템

L1-2 · DB 시스템의 4가지 특징

L1-3 · 데이터 모델 · 3단계 스키마 · 데이터 독립성

U2 ER 모델 (개념적 설계)

L2-1 · 설계 과정 · 개체와 속성

L2-2 · 관계 타입 · 차수 · 역할 · 제약조건

L2-3 · 관계 속성 · 약한 개체

L2-4 · COMPANY 설계 다듬기 · 실습 (MOVIE, CONF_REVIEW)

U3 관계 데이터 모델 · ER→관계 매핑

L3-1 · 관계 모델 기본 개념

L3-2 · 키 · 무결성 제약조건

L3-3 · 갱신 연산과 제약 위반

L3-4 · ER→관계 매핑 ① 개체 · 다중값 · 약한 개체

L3-5 · ER→관계 매핑 ② 관계 타입

U4 Basic SQL (DDL · DML · SELECT)

L4-1 · SQL 소개 · CREATE SCHEMA/TABLE · 제약조건

L4-2 · 참조 트리거 동작 · DROP · ALTER

L4-3 · INSERT · DELETE · UPDATE

L4-4 · SELECT-FROM-WHERE · 별칭

L4-5 · 다중집합 · 집합 연산 · ORDER BY · 연산자

U5 Advanced SQL (중첩 · 조인 · 집계 · 뷰)

L5-1 · 중첩 질의 · IN · ALL/ANY

L5-2 · EXISTS / NOT EXISTS · 나눗셈 질의

L5-3 · 조인 테이블 · NATURAL · INNER/OUTER

L5-4 · 집계 함수 · GROUP BY · HAVING

L5-5 · Assertion · Trigger

L5-6 · 뷰 (View)

U6 관계 대수 (Relational Algebra)

L6-1 · 관계 대수 소개 · 질의 트리

L6-2 · 단항 연산: SELECT · PROJECT · RENAME

L6-3 · 집합 연산 · 카티션 곱

L6-4 · JOIN · DIVISION · 완전 집합

L6-5 · 추가 연산: 집계 · 그룹 · 외부 조인

L6-6 · 관계 대수 작성 연습

U1 · 1_course_intro.pdf + 2_intro-ER.pdf p.3–10

DB 시스템 개념 · 아키텍처

DB·DBMS·DB 시스템의 차이, DB 시스템의 4가지 특징, 3단계 스키마와 데이터 독립성. 정의 한 줄짜리 서술형으로 나오기 좋은 파트라 영어 키워드를 같이 외워.

DB · DBMS · DB 시스템

교안 범위: 1_course_intro p.15–18, 24

§ DB, DBMS, DB 시스템

용어	정의
Database (DB)	논리적으로 일관된(logically coherent), 서로 관련된 데이터의 모음. DBMS가 관리한다
DBMS	데이터를 구조화된 방식으로 효율적으로 "관리"하는 소프트웨어. DB를 **정의(defining), 구축(constructing), 조작(manipulating), 공유(sharing)** 한다. 예) IBM DB2, Oracle, MS SQL Server, MySQL, SQLite
Database System	**DB + DBMS 소프트웨어**

> 시험 포인트: DBMS의 기능 4가지 동사(define, construct, manipulate, share)와 "DB 시스템 = DB + DBMS"를 짝으로.

§ 왜 DBMS를 쓰나

교안 예: 메시지 앱 데이터(친구 정보, 메시지)를 **텍스트 파일**에 한 줄씩, 쉼표로 속성을 나눠 저장하면 생기는 문제 세 묶음.

- **데이터 무결성 (Data Integrity)**: 메시지 길이 제한을 어떻게 지키나? 친구 정보를 실수로 중복 입력하면? 이름이 같은 친구는?
- **데이터 관리 (Data Management)**: 특정 메시지를 빨리 찾으려면? 여러 사용자가 동시에 읽고 쓰면?
- **데이터 지속성 (Data Durability)**: 앱이 쓰는 도중에 기계가 죽으면? 크래시 후 복구는?

→ 이런 기술 문제를 매번 직접 풀 수 없으니 DBMS를 쓴다. 응용 예: 전자상거래, 은행, SNS, 대학/도서관, 항공·철도 예약.

§ DBMS를 쓰지 않는 게 나을 때

DBMS의 오버헤드

- 하드웨어·소프트웨어·교육에 드는 **높은 초기 투자**
- 데이터를 정의·처리하는 **범용성(generality)** 자체의 비용
- 보안, 동시성 제어, 회복, 무결성 기능의 부담 → "너무 무겁다"

맞춤형(customized) DB가 나은 경우

- 단순하고 잘 정의되어 **절대 바뀌지 않을** 응용 (추가·삭제·갱신이 없음)
- DBMS로는 맞출 수 없는 **엄격한 실시간 요구**
- 저장 용량이 제한된 **임베디드 시스템**
- **다중 사용자 접근이 없음**

DB 시스템의 4가지 특징

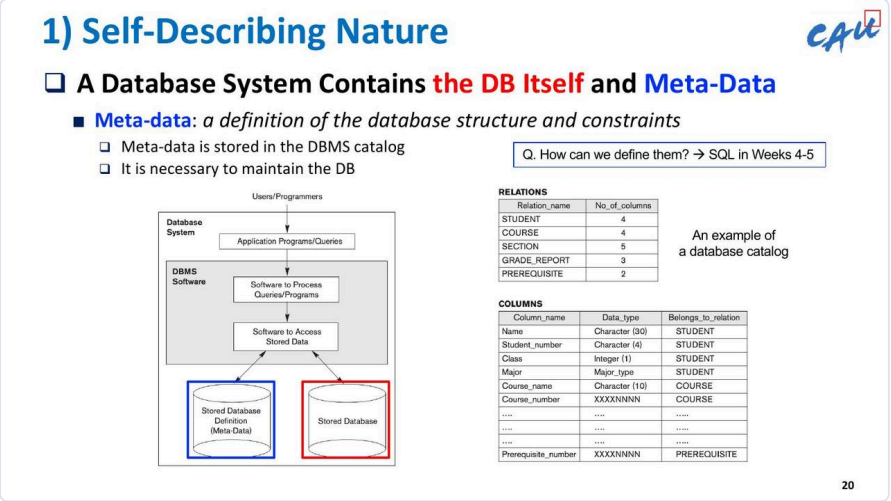
교안 범위: 1_course_intro p.19–25

§ 1) 자기 기술성 (Self-Describing Nature)

DB 시스템은 DB 자체뿐 아니라 **메타데이터**도 담고 있다.

- **메타데이터**: DB 구조와 제약조건의 정의
- 메타데이터는 **DBMS 카탈로그(catalog)** 에 저장된다. DB를 유지하는 데 꼭 필요하다
- 정의하는 방법 = SQL DDL (4–5주차)

함정: 메타데이터가 저장되는 곳은 "카탈로그". 파일 시스템은 데이터 구조 정의가 응용 프로그램 안에 박혀 있다.



01 p.20 · DB 시스템 = 저장된 DB + 메타데이터(카탈로그). 카탈로그 예: RELATIONS, COLUMNS 테이블

§ 2) 데이터 추상화 (Data Abstraction)

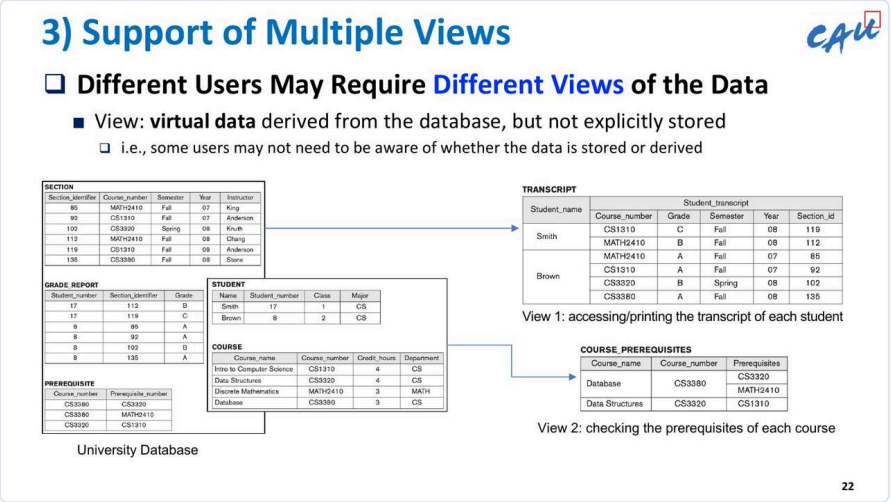
프로그램과 데이터 사이의 절연 (insulation between programs and data)

- DB 시스템은 데이터의 **개념적 표현(논리 수준)** 을 제공한다: 객체, 속성, 상호 관계
- 복잡한 **물리 수준** 구조(저장 방식, 연산 구현)는 드러내지 않는다
- 데이터 구조가 바뀌어도 프로그램은 영향을 받지 않는다 → **프로그램-데이터 독립성 (program-data independence)**

§ 3) 다중 뷰 지원 (Support of Multiple Views)

사용자마다 필요한 데이터 관점이 다르다.

- **뷰 (view)**: DB에서 파생됐지만 **명시적으로 저장되지 않은 가상 데이터**
- 사용자는 그 데이터가 저장된 것인지 파생된 것인지 몰라도 된다
- 예) 대학 DB에서 뷰 1 = 학생별 성적표 출력, 뷰 2 = 과목별 선수과목 확인

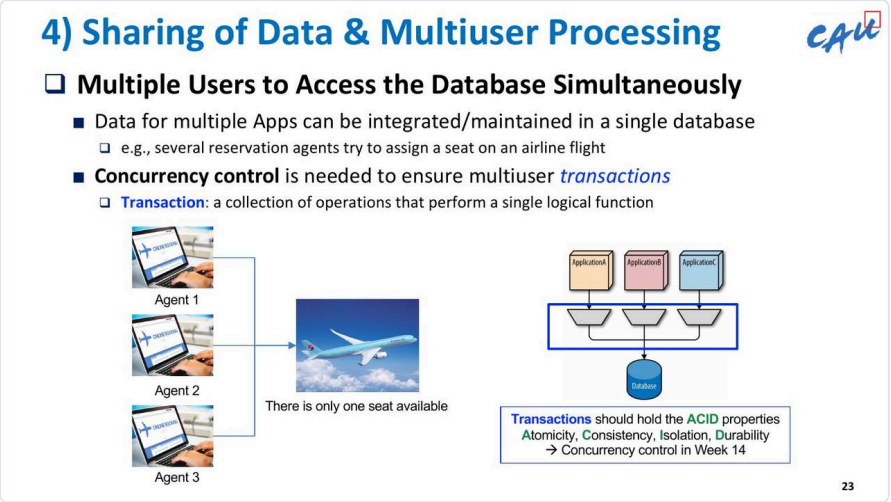


01 p.22 · 대학 DB의 두 뷰: 학생별 성적표(TRANSCRIPT), 과목별 선수과목(COURSE_PREREQUISITES)

§ 4) 데이터 공유와 다중 사용자 트랜잭션 처리

- 여러 사용자가 **동시에** DB에 접근. 여러 앱의 데이터를 하나의 DB에 통합해 관리
- 예) 좌석이 하나 남은 항공편에 예약 담당자 여러 명이 동시에 배정하려는 상황
- **동시성 제어 (concurrency control)** 가 필요
- **트랜잭션 (transaction)**: 하나의 논리적 기능을 수행하는 연산들의 모음
- 트랜잭션은 **ACID** 를 만족해야 한다: **A**tomicity(원자성), **C**onsistency(일관성), **I**solation(고립성), **D**urability(지속성) → 14주차

> 시험 포인트: 특징 4가지 이름을 영어로 정확히 + 각각 한 줄. "self-describing / data abstraction / multiple views / sharing & multiuser transaction processing".



01 p.23 · 좌석 하나에 예약 담당자 셋 → 동시성 제어, 트랜잭션의 ACID

데이터 모델 · 3단계 스키마 · 데이터 독립성

교안 범위: 2_intro-ER p.3–10

§ 데이터 모델 3가지 범주

데이터 모델: DB 구조를 기술하는 개념들의 모음. 데이터 추상화를 위한 "도구" (데이터 타입, 관계, 제약조건, 검색·갱신 연산).

- 추상화 = 데이터 이해에 필요한 본질은 강조하고, 데이터 조직·저장의 세부는 감추는 것

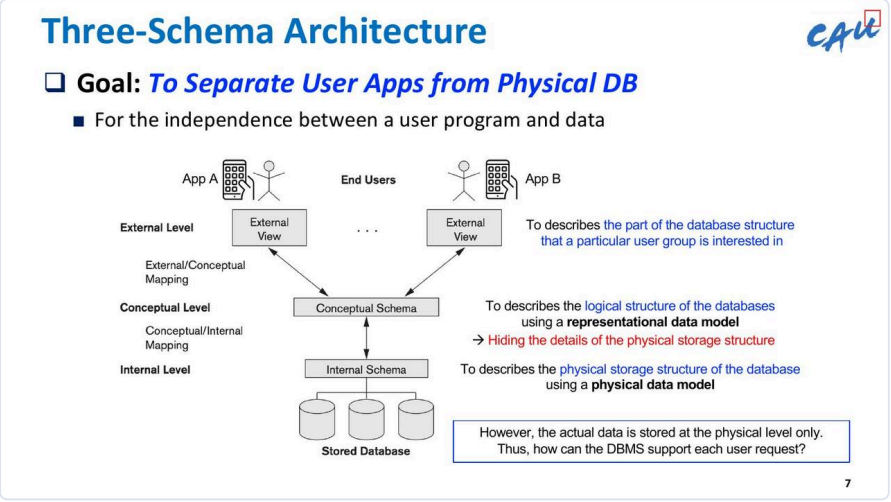
범주	수준	내용
개념적 (conceptual)	high-level	사용자가 데이터를 인식하는 방식에 가까운 개념. 예) ER 모델 (EMPLOYEE, DEPARTMENT, PROJECT)
표현적 (representational)	중간	DBMS가 데이터를 관리하는 방식. 물리 저장 세부는 숨기지만 컴퓨터에 바로 구현 가능. 예) 관계 데이터 모델
물리적 (physical)	low-level	데이터가 컴퓨터에 저장되는 방식: 레코드 형식, 순서, 접근 경로. 일반 사용자가 아니라 전문가용

§ 3단계 스키마 구조 (Three-Schema Architecture)

목표: 사용자 응용을 물리적 DB에서 분리 → 사용자 프로그램과 데이터의 독립.

수준	스키마	기술하는 것
외부 (external)	외부 스키마 / 사용자 뷰	특정 사용자 그룹이 관심 있는 DB 구조의 일부
개념 (conceptual)	개념 스키마	전체 DB의 논리 구조. **표현적 데이터 모델**로 기술. 엔티티, 데이터 타입, 관계, 연산, 제약조건. 물리 저장 구조는 숨김
내부 (internal)	내부 스키마	DB의 물리 저장 구조. **물리적 데이터 모델**로 기술. 저장·접근 경로의 세부

함정: 실제 데이터는 **물리(내부) 수준에만** 저장된다. 외부·개념 스키마는 "기술"일 뿐.



02 p.7 · 3단계 스키마 구조 (외부·개념·내부 수준과 사상)

§ 스키마 간 사상 (Mapping)

실제 데이터는 내부 수준에만 있으니, DBMS가 요청과 결과를 수준 사이에서 **변환**해야 한다.

- 요청: 외부 스키마 → 개념 스키마 → 내부 스키마로 바꿔서 저장된 DB에서 처리
- 결과: 저장된 DB에서 꺼낸 데이터를 사용자의 외부 뷰에 맞게 다시 형식화
- **사상 정보도 카탈로그에 저장된다**

---

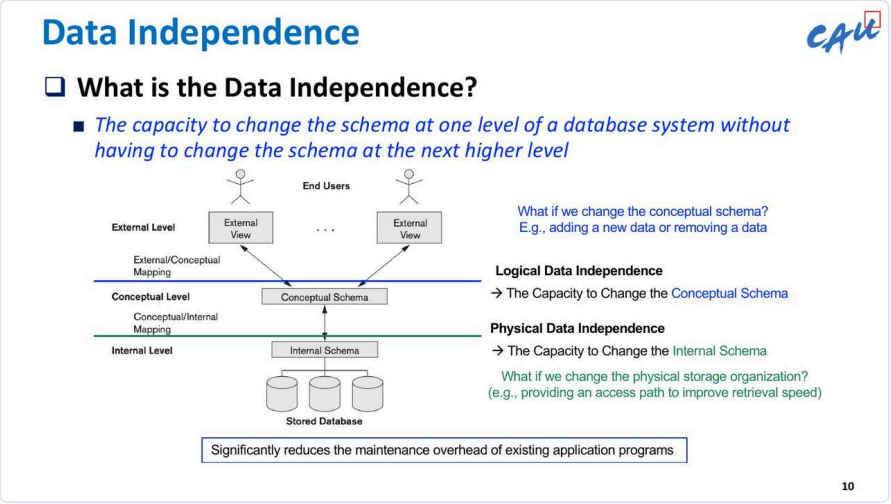
§ 데이터 독립성 (Data Independence)

한 수준의 스키마를 바꿔도 바로 위 수준의 스키마를 바꿀 필요가 없는 능력.

종류	바꾸는 것	예
논리적 데이터 독립성	**개념 스키마**	새 데이터 추가, 데이터 제거 → 외부 스키마·응용 프로그램은 그대로
물리적 데이터 독립성	**내부 스키마**	저장 구조 변경, 검색 속도를 위해 접근 경로(인덱스) 추가 → 개념 스키마 그대로

→ 기존 응용 프로그램의 **유지보수 부담을 크게 줄인다**.

> 시험 포인트: "논리 = 개념 스키마 변경, 물리 = 내부 스키마 변경"을 섞지 마. 물리적 독립성이 더 달성하기 쉽다 (인덱스 추가는 흔한 일).



02 p.10 · 논리적 데이터 독립성(개념 스키마 변경) vs 물리적 데이터 독립성(내부 스키마 변경)

U2 · 2_intro-ER.pdf p.11-47 + 2-2_ER_model-practices.pdf

ER 모델 (개념적 설계)

요구사항 → 개체·속성 → 관계와 제약조건 → 다듬기. 시험에서 요구사항 문장을 주고 ER 다이어그램이나 (min, max)를 쓰게 할 가능성이 크니, "요구사항 문장 → 어떤 표기"로 바꾸는 연습을 해.

설계 과정 · 개체와 속성

교안 범위: 2_intro-ER p.12–21

§ DB 설계 과정

1. **요구사항 수집·분석 (Requirements Collection and Analysis)**: 예상 사용자의 데이터 요구를 파악. 결과물 = 사용자의 상세 요구사항
2. **개념적 설계 (Conceptual Design)**: 데이터 모델을 고르고, 요구사항을 **개념 스키마**로 번역 (ER 모델)
3. **구현 (Implementation)**: 개념 스키마로 DB를 구현
   - **논리적 설계**: DB 스키마 (예: 관계 스키마, 3주차 ER→관계 매핑)
   - **물리적 설계**: DB의 물리적 배치

COMPANY 요구사항: 회사는 **부서**(고유한 이름·번호, 여러 위치, 관리자와 관리 시작일)로 조직되고, 부서는 여러 **프로젝트**(고유한 이름·번호, 위치 하나)를 통제한다. **사원**은 이름·SSN·주소·급여·성별·생일·일하는 프로젝트가 있고, 보험 때문에 사원의 **부양가족**(이름, 성별, 생일, 관계)도 관리한다.

§ 개체와 속성

ER (Entity-Relationship) 모델: 데이터를 **개체(entity)와 관계(relationship)의 모음**으로 기술. ER 다이어그램으로 그린다.

- **개체 (entity)**: 현실 세계에서 **독립적으로 존재**하고 다른 것과 구별되는 "것". 물리적 존재(사원)일 수도, 개념적 존재(회사)일 수도 있다
- **속성 (attribute)**: 개체를 기술하는 특정 성질. 예) 사원: 이름, 주소, 나이, 집 전화
- **관계 (relationship)**: 여러 개체 사이의 연관

§ 속성의 분류

분류	짝	예
① 복합 vs 단순	**복합 (composite)**: 더 작은 부분으로 나뉨 / **단순(원자, simple/atomic)**: 안 나뉨	Address = (Street_address, City, State, Zip) / City, Zip
② 단일값 vs 다중값	**단일값 (single-valued)**: 개체마다 값 하나 / **다중값 (multivalued)**: 값의 집합	Age, SSN / Phone numbers, College degrees, 부서의 Locations
③ 저장 vs 유도	**저장 (stored)**: DB에 실제 저장 / **유도 (derived)**: 다른 속성에서 계산	Birth_date / Age (생일에서 유도)

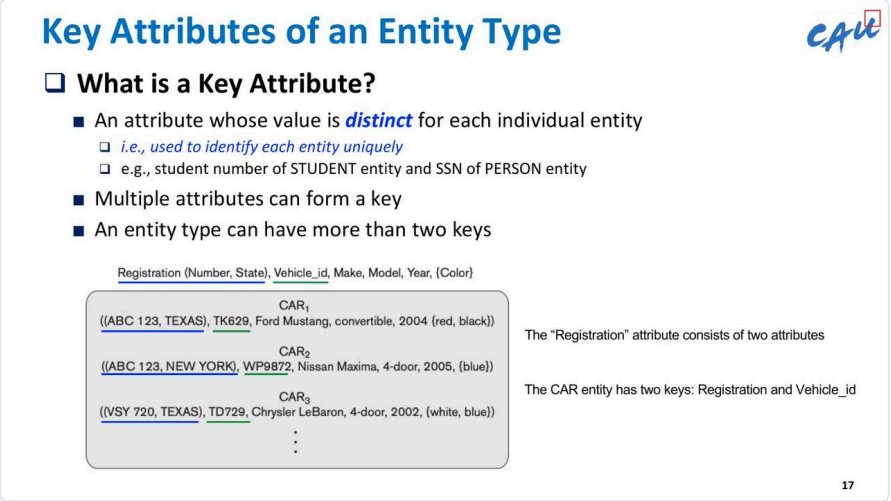
함정: "다중값"과 "복합"을 섞지 마. 복합 = 한 값이 여러 부분으로 쪼개짐, 다중값 = 값이 여러 개. 둘 다일 수도 있다 (예: {Address_phone(Phone, Address)}).

§ 키 속성

키 속성 (key attribute): 개체마다 값이 **서로 다른** 속성 → 각 개체를 유일하게 식별.

- 예) STUDENT의 학번, PERSON의 SSN
- 여러 속성이 모여서 하나의 키가 될 수 있다 (예: CAR의 Registration = (Number, State) 복합 키)
- 한 개체 타입이 키를 **둘 이상** 가질 수 있다 (예: CAR의 키 = Registration, Vehicle_id)

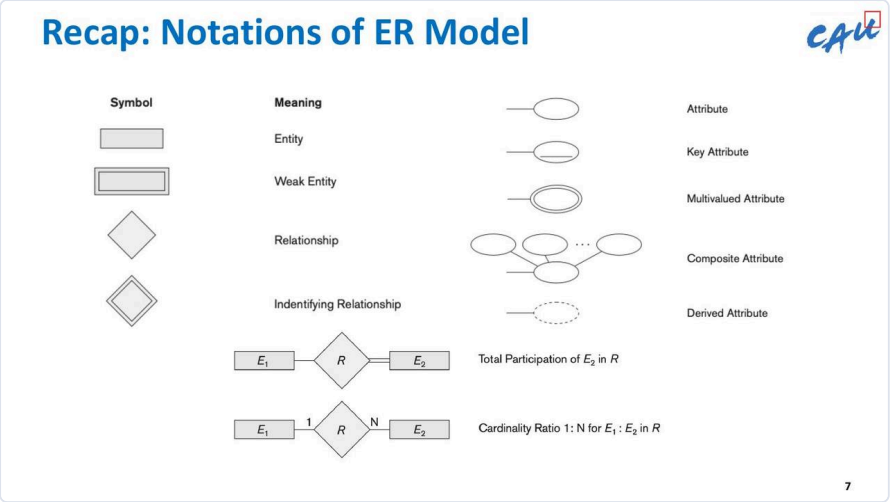
- 다이어그램에서 **밑줄**로 표시



02 p.17 · CAR의 키 두 개: 복합 키 Registration(Number, State)과 Vehicle_id

§ ER 다이어그램 표기법

기호	의미
직사각형	개체 타입
이중 직사각형	약한 개체 타입
마름모	관계 타입
이중 마름모	식별 관계 타입
타원	속성
밑줄 친 타원	키 속성
이중 타원	다중값 속성
타원에 타원이 달림	복합 속성
점선 타원	유도 속성
E_1 ─ R ═ E_2 (이중선)	E_2가 R에 **전체 참여**
E_1 —1— R —N— E_2	카디널리티 비 1:N
R —(min,max)— E	E가 R에 참여하는 구조적 제약



02-2 p.7 · ER 표기법 정리 (약한 개체, 식별 관계, 다중값·복합·유도 속성, 전체 참여, 1:N)

§ 초기 개념 설계: COMPANY

- **DEPARTMENT**: Name, Number, {Locations}, Manager, Manager_start_date. Name과 Number가 각각 키 (둘 다 고유). Locations는 다중값
- **PROJECT**: Name, Number, Location, Controlling_department. Name, Number가 키
- **EMPLOYEE**: Name(Fname, Minit, Lname 복합), SSN(키), Address, Salary, Birth_date, Sex, Department, Supervisor, {Works_on(Project, Hours)}
- **DEPENDENT**: Employee, Dependent_name, Sex, Birth_date, Relationship

문제: Manager, Department, Supervisor, Works_on, Employee 같은 속성은 사실 **다른 개체를 참조**한다. 관계가 속성 형태로 **암묵적으로** 들어가 있음 → 이걸 명시적인 **관계 타입**으로 바꿔야 한다 (L2-4 다듬기).

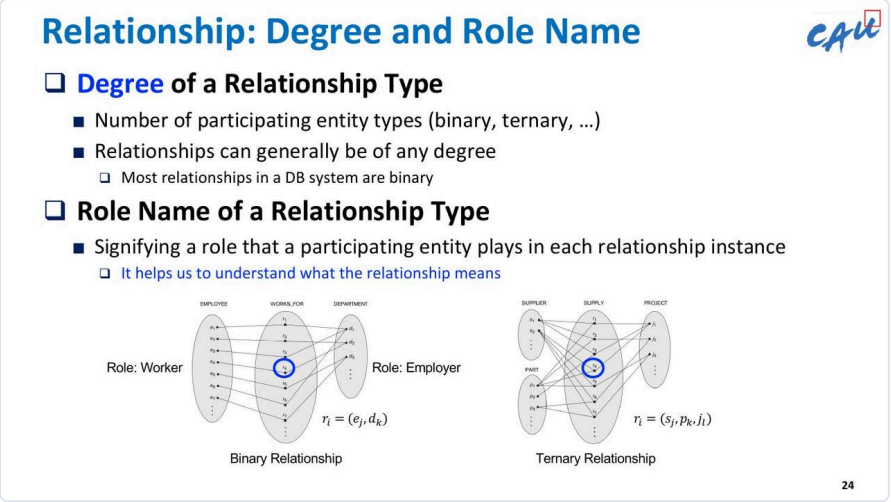
관계 타입 · 차수 · 역할 · 제약조건

교안 범위: 2_intro-ER p.23–30, 36–37

§ 관계 타입, 차수, 역할 이름

관계 타입: 이름 + 참여하는 개체 타입들로 기술. 참여 개체들 사이의 연관 집합을 정의한다.

- 예) WORKS_FOR: EMPLOYEE와 DEPARTMENT 사이. 각 사원을 그가 일하는 부서와 연관시킴
- **차수 (degree)**: 참여하는 개체 타입의 수. 2진(binary), 3진(ternary)... 대부분은 2진
  - 2진 예: $r_i = (e_j, d_k)$, 3진 예: SUPPLY의 $r_i = (s_j, p_k, j_1)$ (SUPPLIER, PART, PROJECT)
- **역할 이름 (role name)**: 참여 개체가 관계 인스턴스에서 맡는 역할. 관계의 의미를 이해하게 해줌. 예) WORKS_FOR에서 EMPLOYEE = worker, DEPARTMENT = employer

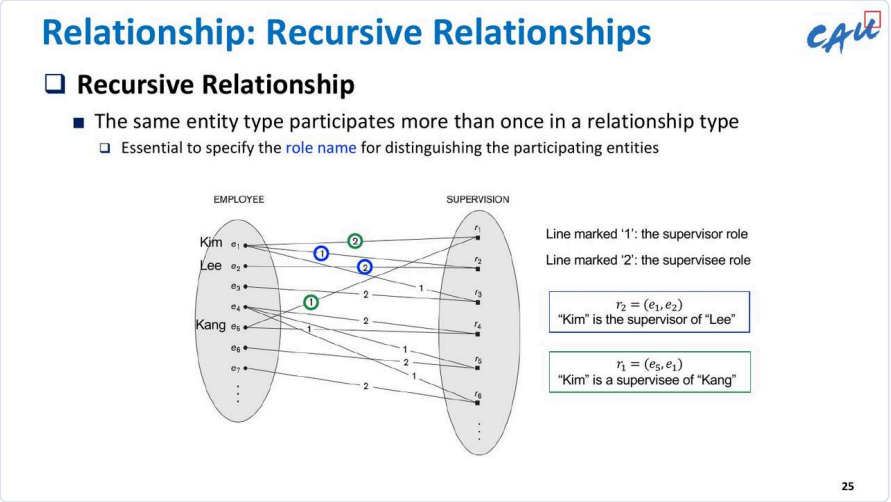


02 p.24 · 2진 관계 WORKS_FOR (역할 worker/employer)와 3진 관계 SUPPLY

§ 순환 관계 (Recursive Relationship)

같은 개체 타입이 한 관계 타입에 두 번 이상 참여하는 관계. 참여 개체를 구별하려면 **역할 이름이 필수**.

- 예) SUPERVISION: EMPLOYEE가 supervisor 역할(선 '1')과 supervisee 역할(선 '2')로 참여
- $r_2 = (e_1, e_2)$: Kim이 Lee의 상사 / $r_1 = (e_5, e_1)$: Kim은 Kang의 부하



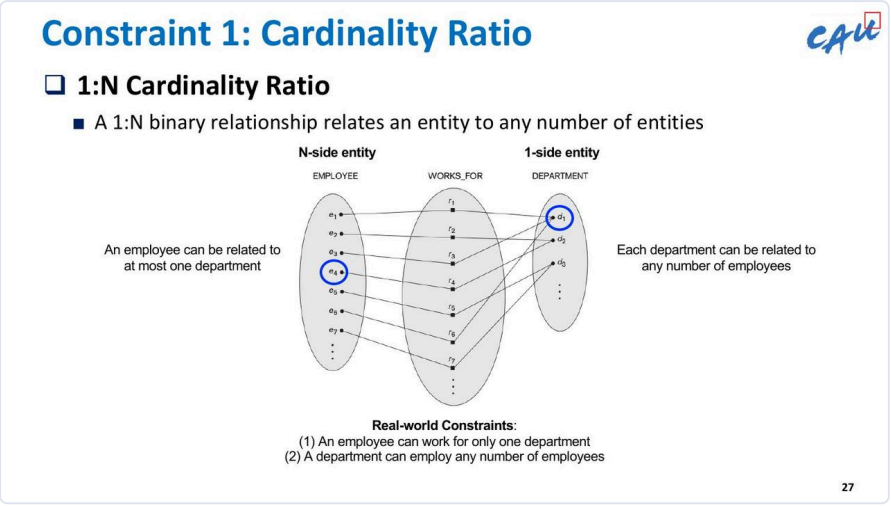
02 p.25 · 순환 관계 SUPERVISION: 선 1 = supervisor, 선 2 = supervisee

§ 제약조건 ① 카디널리티 비

한 개체가 참여할 수 있는 관계 인스턴스의 최대 개수. 2진 관계에서 1:1, 1:N, M:N.

비	예	현실 제약
1:N	WORKS_FOR (DEPARTMENT 1 : EMPLOYEE N)	사원은 한 부서에서만 일하고, 부서는 사원을 몇 명이든 둘 수 있다
1:1	MANAGES (EMPLOYEE 1 : DEPARTMENT 1)	사원은 최대 한 부서를 관리, 부서는 관리자가 최대 한 명
M:N	WORKS_ON (EMPLOYEE M : PROJECT N)	사원은 프로젝트 여러 개, 프로젝트도 사원 여러 명

함정: 1:N에서 **N쪽** 개체 하나는 1쪽 개체 **최대 하나**와 연결된다. "사원 하나 → 부서 최대 하나"니까 사원이 N쪽.

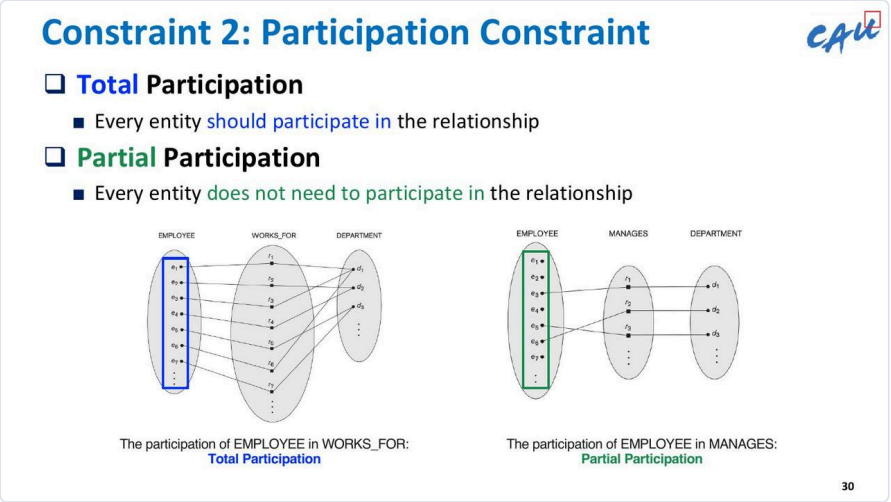


02 p.27 · 1:N 관계 WORKS_FOR: N쪽 EMPLOYEE, 1쪽 DEPARTMENT

§ 제약조건 ② 참여 제약

개체의 존재가 다른 개체와의 관계에 의존하는지 지정. 최소 참여 수와 관련.

- **전체 참여 (total)**: **모든** 개체가 관계에 참여해야 함. 이중선. 예) 모든 사원은 어떤 부서에서 일해야 하므로 EMPLOYEE는 WORKS_FOR에 전체 참여 (= existence dependency)
- **부분 참여 (partial)**: 모든 개체가 참여할 필요는 없음. 단일선. 예) 모든 사원이 부서를 관리하진 않으므로 EMPLOYEE는 MANAGES에 부분 참여



02 p.30 · EMPLOYEE는 WORKS_FOR에 전체 참여, MANAGES에 부분 참여

§ (min, max) 표기

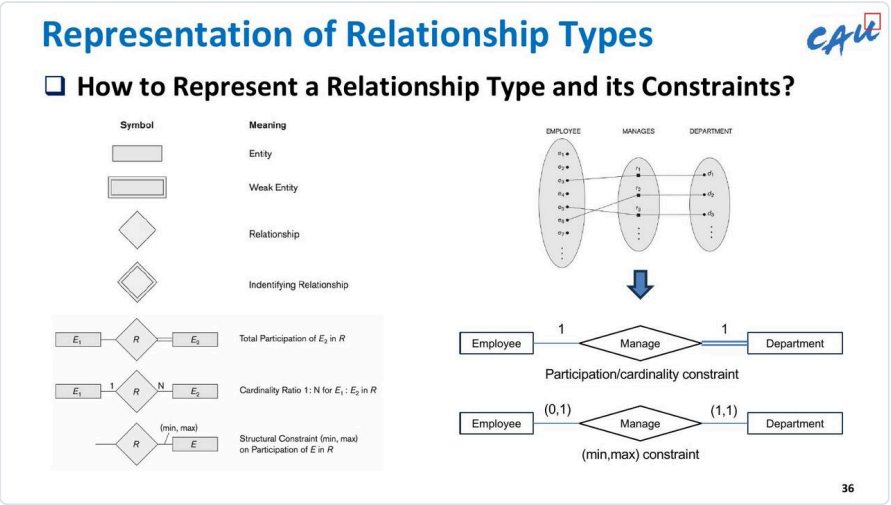
각 참여에 **(min, max)** 를 붙여서 카디널리티와 참여를 함께 표시. 0 ≤ min ≤ max, max ≥ 1.

- **min = 0 → 부분 참여, min ≥ 1 → 전체 참여**
- max = 그 개체가 참여할 수 있는 관계 인스턴스 수의 최대
- 예) MANAGES: EMPLOYEE (0,1), DEPARTMENT (1,1)

함정: (min,max)는 **그 개체 쪽에 붙고, 그 개체 하나가** 관계에 몇 번 참여하는지를 뜻한다. 1:N 표기의 "1", "N"은 **반대편**에 붙는다 (DEPARTMENT —1— WORKS_FOR —N— EMPLOYEE). 그래서 WORKS_FOR는 (min,max)로 EMPLOYEE (1,1), DEPARTMENT (4,N)처럼 위치가 뒤집혀 보인다.

교안 연습 1 (INSTRUCTOR–TEACHES–COURSE–USES–TEXT):

- 강의는 교재를 쓸 수도 안 쓸 수도, 다섯 권 초과는 안 됨 → COURSE의 USES 참여 **(0, 5)**
- 강의는 최소 1명, 최대 4명이 가르침 → COURSE의 TEACHES 참여 **(1, 4)**
- 교재는 어떤 강의들에서 쓰임 → TEXT의 USES 참여 **(1, N)**
- 강사는 2~4개 강의를 가르침 → INSTRUCTOR의 TEACHES 참여 **(2, 4)**

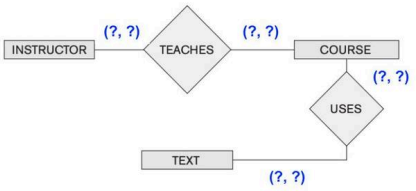


02 p.36 · 관계 표기와 (min,max): MANAGES에서 EMPLOYEE (0,1), DEPARTMENT (1,1)

Exercise 1: Constraints on ER Diagram

❑ **Suggest the (min, max) constraints on the following diagram**

- A course may or may not use a textbook
- A course may not use more than five books
- A course can be taught by at least one instructor and by a maximum of 4 instructors
- A textbook is used in some courses
- An instructor teaches from two to four courses



37

02 p.37 · 연습 1: INSTRUCTOR–TEACHES–COURSE–USES–TEXT의 (min,max) 채우기

관계 속성 · 약한 개체

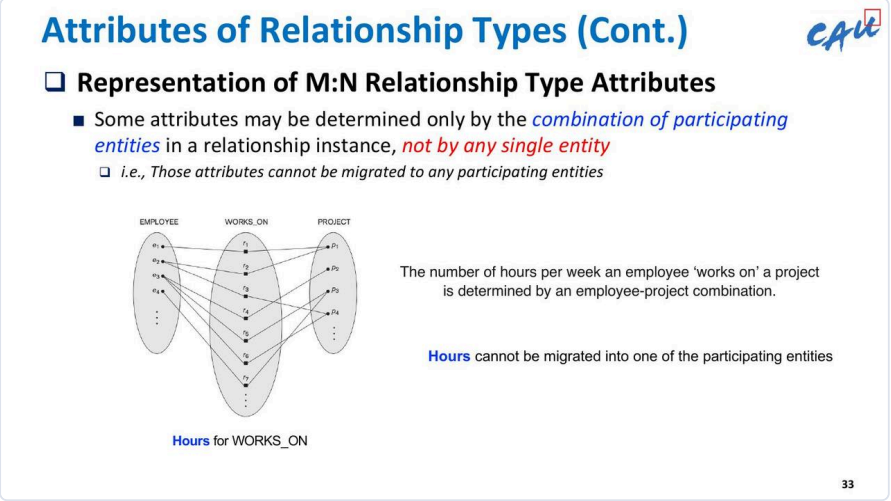
교안 범위: 2_intro-ER p.31–35

§ 관계 타입의 속성

관계 타입도 속성을 가질 수 있다. 예) MANAGES의 StartDate, WORKS_ON의 Hours, WORKS_FOR의 StartDate.

관계	속성을 옮길(migrate) 수 있는 곳
1:1	**어느 쪽** 참여 개체로든 옮길 수 있음 (MANAGES.StartDate → DEPARTMENT 또는 EMPLOYEE)
1:N	**N쪽 개체로만** 옮길 수 있음 (WORKS_FOR.StartDate → EMPLOYEE)
M:N	**어느 개체로도 못 옮김**. 개체 조합에 의해서만 결정되기 때문 (WORKS_ON.Hours는 사원–프로젝트 조합으로 결정)

> 시험 포인트: "1:N 관계의 속성은 왜 N쪽으로만?" → N쪽 개체 하나는 관계 인스턴스에 최대 1번 참여하니까 값이 하나로 정해진다. 1쪽 개체는 여러 인스턴스에 참여하므로 값이 여러 개가 됨.



02 p.33 · M:N 관계 WORKS_ON의 Hours: 사원–프로젝트 조합으로만 정해짐

§ 약한 개체 타입

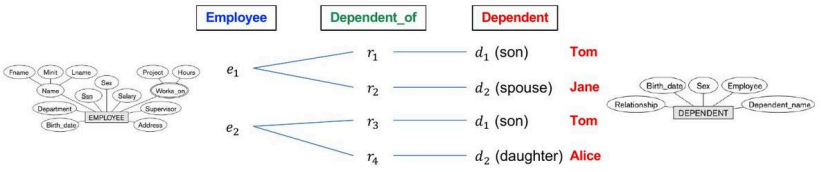
약한 개체 타입 (weak entity type): 자기 자신의 **키 속성이 없는** 개체 타입. **식별(소유) 개체 타입**에 의존한다.

- 약한 개체는 반드시 식별 개체와 연관되어야 함
- **식별 관계 (identifying relationship)**: 약한 개체 타입과 소유자(owner) 개체 타입을 잇는 관계. 이중 마름모
- 약한 개체 타입은 식별 관계에 **항상 전체 참여**
- **부분 키 (partial key)**: 같은 소유자에 딸린 약한 개체들끼리 구별하는 속성. 점선 밑줄
  - 예) DEPENDENT의 Dependent_name (같은 사원의 부양가족끼리는 이름이 안 겹친다고 가정)
  - 부분 키만으로는 식별 못 함 (다른 사원의 부양가족 Tom이 또 있을 수 있음) → **소유자 키 + 부분 키**로 식별

Weak Entity Types (Cont.)

❑ Partial Key

- To uniquely identify weak entities that are related to the same owner entity
- Example 1: EMPLOYEE-DEPENDENT
  - ❑ The **Dependent_Name** of DEPENDENT is the partial key since no two dependents have the same name



However, a partial key alone cannot identify its corresponding weak entity
Thus, it can be used as *a key together with the identifying relationship*

35

02 p.35 · 약한 개체 DEPENDENT의 부분 키 Dependent_name (Tom이 두 사원에게 있음)

COMPANY 설계 다듬기 · 실습 (MOVIE, CONF_REVIEW)

교안 범위: 2_intro-ER p.38–47, 2-2 p.14–26

§ COMPANY 설계 다듬기

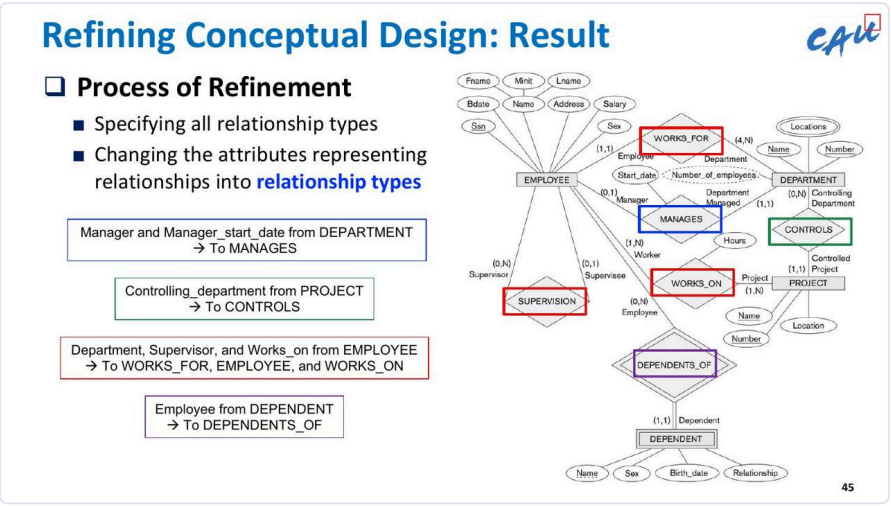
관계를 표현하던 속성을 **관계 타입으로** 바꾼다.

관계	참여	카디널리티	참여 제약	속성
MANAGES	EMPLOYEE, DEPARTMENT	1:1	EMPLOYEE 부분, DEPARTMENT 전체	Start_date (Manager_start_date)
WORKS_FOR	EMPLOYEE, DEPARTMENT	1:N (DEPARTMENT 1, EMPLOYEE N)	둘 다 전체	없음
CONTROLS	DEPARTMENT, PROJECT	1:N (DEPARTMENT 1, PROJECT N)	PROJECT 전체, DEPARTMENT 부분	없음
SUPERVISION	EMPLOYEE (supervisor, supervisee)	1:N (supervisor 1, supervisee N)	둘 다 부분	없음
WORKS_ON	EMPLOYEE, PROJECT	M:N	둘 다 전체	Hours
DEPENDENTS_OF	EMPLOYEE(식별), DEPENDENT(약한)	1:N (EMPLOYEE 1)	EMPLOYEE 부분, DEPENDENT 전체	없음 (식별 관계)

옮겨진 것: DEPARTMENT의 Manager·Manager_start_date → MANAGES, PROJECT의 Controlling_department → CONTROLS, EMPLOYEE의 Department·Supervisor·Works_on → WORKS_FOR·SUPERVISION·WORKS_ON, DEPENDENT의 Employee → DEPENDENTS_OF.

주의: 교안 p.42 SUPERVISION 설명은 1쪽/N쪽 역할이 뒤바뀌어 적혀 있어. 상사(supervisor) 한 명이 여러 부하(supervisee)를 두므로 **supervisor가 1쪽**이 맞고, 결과 다이어그램(p.45)의 (min,max)도 supervisor (0,N), supervisee (0,1)이야.

ER 스키마 설계 3단계 요약: (1) 요구사항에서 개체 타입과 속성 기술 → (2) 관계 타입과 제약조건 지정 → (3) 관계를 표현하던 속성을 관계 타입으로 바꿈. 인스턴스가 아니라 **스키마**에 집중.



02 p.45 · 다듬은 COMPANY ER 다이어그램 (관계 6개와 (min,max))

§ 실습: MOVIE / CONF_REVIEW

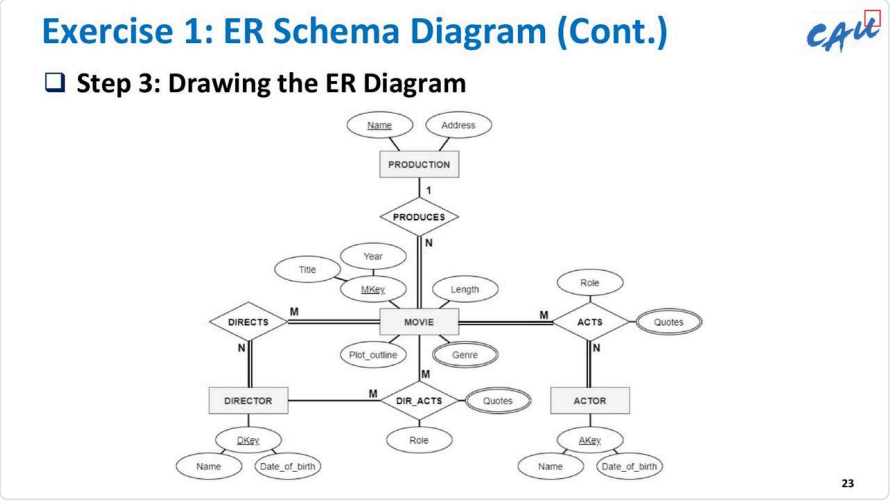
MOVIE DB (2-2 p.14–23)

- MOVIE: 키 MKey(Title, Year) 복합, Length, Plot_outline, {Genre} 다중값
- ACTOR, DIRECTOR: 키 (Name, Date_of_birth). PRODUCTION: 키 Name, Address
- **PRODUCES** (PRODUCTION 1 : MOVIE N): 영화는 제작사 하나가 만든다. MOVIE 전체 참여
- **ACTS** (ACTOR M : MOVIE N): 관계 속성 Role, {Quotes}(다중값). 양쪽 전체 참여
- **DIRECTS** (DIRECTOR M : MOVIE N): MOVIE 전체 참여 (감독이 최소 1명)
- **DIR_ACTS**: 감독도 영화에 출연할 수 있음 (Role, {Quotes})

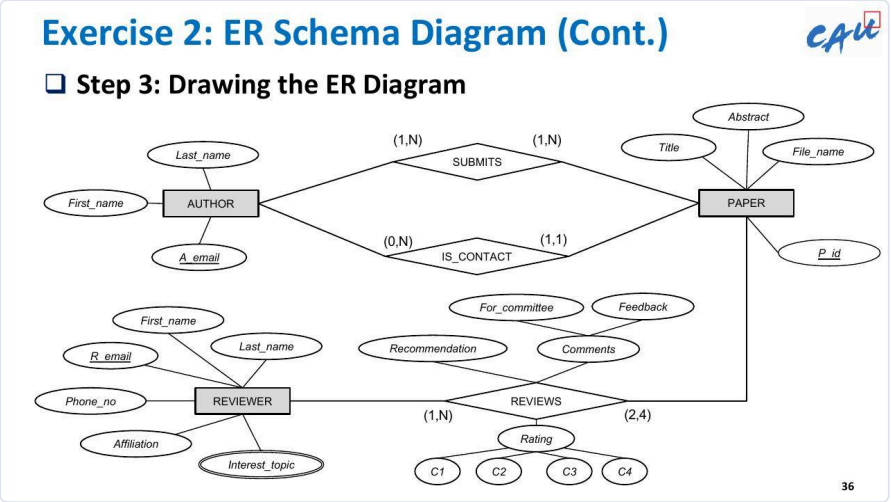
CONF_REVIEW DB (2-2 p.24–26, 풀이는 db-exercises)

- AUTHOR(A_email 키, First_name, Last_name), PAPER(P_id 키, Title, Abstract, File_name), REVIEWER(R_email 키, 이름, Phone_no, Affiliation, {Interest_topic})
- **SUBMITS** AUTHOR (1,N) – PAPER (1,M): 저자는 논문을 내야 기록되고, 논문은 여러 저자 가능 (M:N)
- **IS_CONTACT** AUTHOR (0,N) – PAPER (1,1): 논문마다 연락 저자가 정확히 한 명. SUBMITS만으로는 이 요구를 못 나타내서 따로 둠
- **REVIEWS** REVIEWER (1,N) – PAPER (2,4): 논문당 리뷰어 2~4명. 관계 속성 Recommendation, Comments(For_committee, Feedback) 복합, Rating(C1 Technical_merit, C2 Readability, C3 Originality, C4 Relevance) 복합

> 시험 포인트: "~마다 정확히 하나" = (1,1), "할 수도 안 할 수도" = min 0, "최소 a 최대 b" = (a,b), "여러 개" = max N. 요구사항 문장에서 이 단어들을 찾아 표기로 옮기는 게 핵심.



02-2 p.23 · MOVIE DB ER 다이어그램



db-exercises p.36 · CONF_REVIEW DB ER 다이어그램 (SUBMITS, IS_CONTACT, REVIEWS)

U3 · 3-1_RDB.pdf

관계 데이터 모델 · ER→관계 매핑

용어(릴레이션·투플·속성·도메인·차수), 키 계열(슈퍼키 ⊃ 키 = 후보 키 ∋ 기본 키), 무결성 제약 3가지, 삽입·삭제·갱신이 어떤 제약을 깨는지, 그리고 ER→관계 매핑. 매핑은 "FK를 어디에 넣나"만 정확하면 된다.

관계 모델 기본 개념

교안 범위: 3-1 p.4–12

§ 관계 데이터 모델

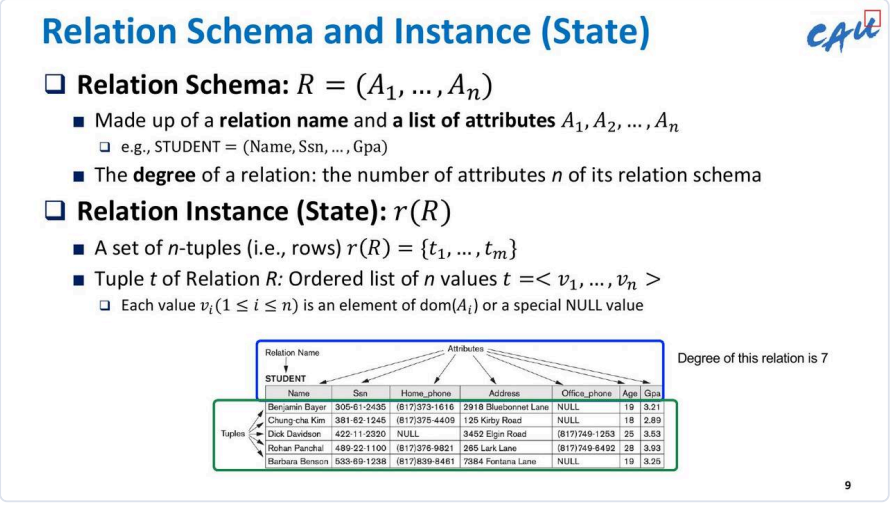
- 1970년 IBM의 **Ted Codd**가 제안
- 데이터를 **릴레이션(relation, 테이블)** 으로 표현. 각 릴레이션은 **투플(tuple, 행)** 과 **속성(attribute, 열 헤더)** 으로 구성
- 각 행 = 특정 개체에 대한 사실(관련된 데이터 값들의 모음), 속성 = 각 행의 값을 어떻게 해석할지 지정

장점 4가지: **단순성** (평평한 테이블 구조), **데이터 무결성** (유효·일관·정확한 데이터 보장), **쉬운 데이터 조작** (SQL 같은 강력한 고급 언어로 복잡한 조건도 유연하게), **보안** (인증, 권한, 암호화)

§ 도메인 · 릴레이션 스키마 · 인스턴스

- **도메인 D**: 각 속성에 허용되는 **원자 값(atomic value)들의 집합**. 도메인 = 데이터 타입 + 제약조건
  - 예) Usa_phone_numbers: (ddd)ddd-dddd 형식, Employee_ages: 15~80 사이 정수
- **릴레이션 스키마 R(A_1, ..., A_n)**: 릴레이션 이름 + 속성 리스트. 예) STUDENT(Name, Ssn, ..., Gpa)
- **차수 (degree)**: 릴레이션 스키마의 **속성 개수 n**. 예) STUDENT(Name, Ssn, Home_phone, Address, Office_phone, Age, Gpa)는 차수 7
- **릴레이션 인스턴스(상태) r(R)**: n-투플들의 **집합** {t_1, ..., t_m}
- **투플 t**: n개 값의 **순서 있는** 리스트 <v_1, ..., v_n>. 각 v_i는 dom(A_i)의 원소이거나 특수값 **NULL**

함정: 차수(degree) = 속성(열) 수. 투플(행) 수는 카디널리티(cardinality)라고 부른다. 섞지 마.



03 p.9 · 릴레이션 이름, 속성, 투플, 차수(7)

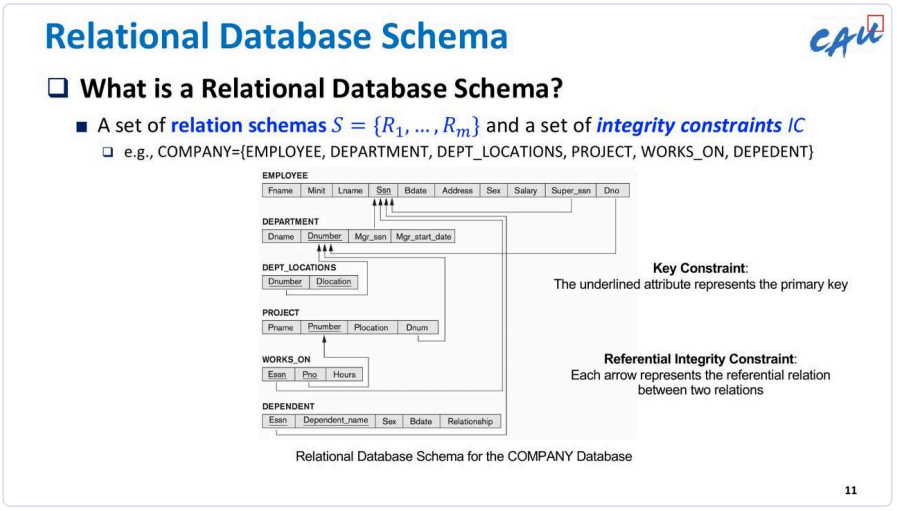
§ 릴레이션의 특성

- **투플의 값은 원자 값**이어야 함 (더 쪼개지지 않음)
  - **복합 속성·다중값 속성은 허용 안 됨**
  - 다중값 속성 → **별도 릴레이션**으로 표현
  - 복합 속성 → **단순 구성 속성들**로만 표현

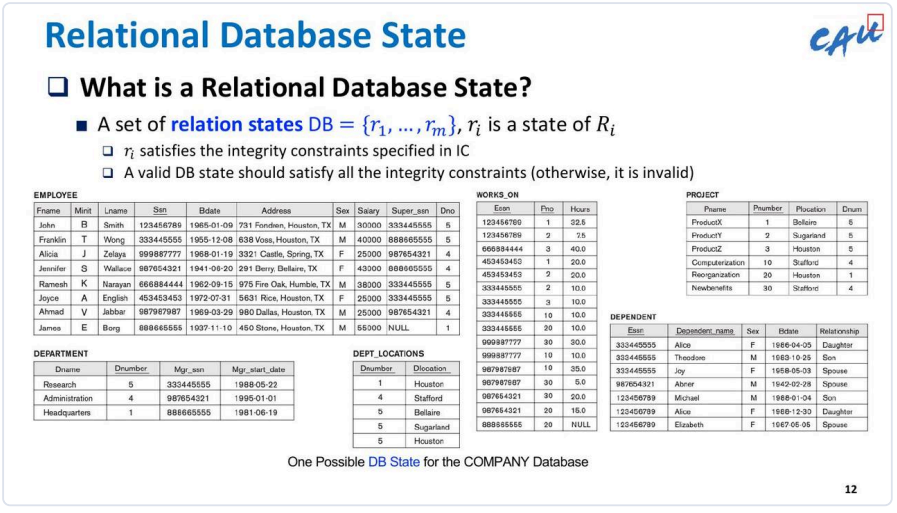
- **NULL**: 값을 모르거나(unknown), 얻을 수 없거나(unavailable), 해당 투플에 적용되지 않는(undefined/not applicable) 경우
- 릴레이션은 투플의 **집합**이므로 정의상 **투플끼리는 모두 달라야** 한다

§ 관계 DB 스키마와 상태

- **관계 DB 스키마 S** = 릴레이션 스키마들의 집합 {R₁, …, Rₘ} + **무결성 제약조건 집합 IC**
  - 예) COMPANY = {EMPLOYEE, DEPARTMENT, DEPT_LOCATIONS, PROJECT, WORKS_ON, DEPENDENT}
  - 스키마 그림에서 **밑줄 = 기본 키**, **화살표 = 참조 무결성** (FK → 참조되는 PK)
- **관계 DB 상태 DB** = 릴레이션 상태들의 집합 {r₁, …, rₘ}, rᵢ는 Rᵢ의 상태
  - 모든 무결성 제약을 만족하면 **유효(valid) 상태**, 하나라도 어기면 **무효(invalid) 상태**



03 p.11 · COMPANY 관계 DB 스키마 (밑줄 = PK, 화살표 = 참조 무결성)



03 p.12 · COMPANY DB 상태 (문제 풀이에 계속 쓰는 데이터)

키 · 무결성 제약조건

교안 범위: 3-1 p.13–21

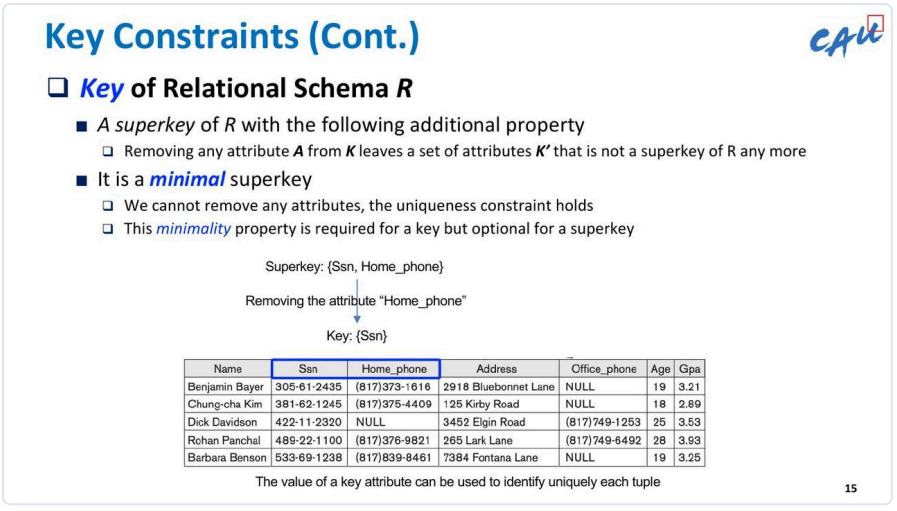
§ 슈퍼키와 키

DBMS는 제약조건을 정의하고 강제하는 기능을 제공해야 한다. 핵심 제약 3가지 = **키 제약, 개체 무결성, 참조 무결성**.

키 제약: "서로 다른 두 투플은 SK 속성들에 대해 같은 값을 가질 수 없다" → 모든 상태 r에서 $t_1[SK] \neq t_2[SK]$.

- **슈퍼키 (SK)**: 각 투플을 **유일하게 식별하는 속성 부분집합**. 모든 릴레이션은 최소 하나의 기본 슈퍼키(속성 전체)를 가진다
- **키 (K)**: 슈퍼키이면서, 어떤 속성 A를 빼도 더 이상 슈퍼키가 아닌 것 = **최소(minimal) 슈퍼키**
  - 최소성은 키에는 필수, 슈퍼키에는 선택
  - 예) {Ssn, Home_phone}은 슈퍼키. Home_phone을 빼도 {Ssn}이 유일 → {Ssn, Home_phone}은 키가 아님, {Ssn}이 키

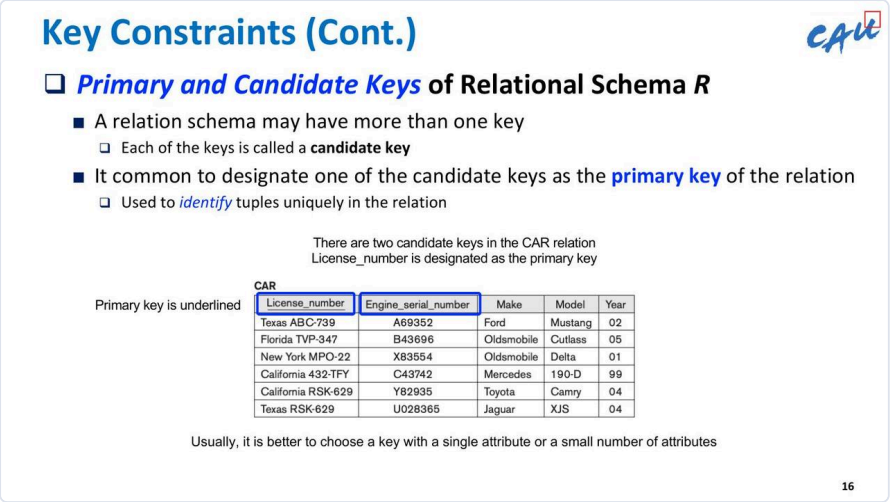
함정: 키 ⊂ 슈퍼키. "모든 키는 슈퍼키지만, 모든 슈퍼키가 키는 아니다".



03 p.15 · 슈퍼키 {Ssn, Home_phone}에서 Home_phone을 빼면 키 {Ssn}

§ 후보 키와 기본 키

- 릴레이션 스키마에 키가 여러 개면, 각각을 **후보 키 (candidate key)** 라고 부른다
- 후보 키 중 하나를 **기본 키 (primary key)** 로 지정해 투플을 식별하는 데 쓴다. 스키마에서 **밑줄**
- 예) CAR의 후보 키 = License_number, Engine_serial_number → License_number를 PK로 지정
- 보통은 **속성 하나 또는 적은 수**로 된 키를 PK로 고르는 게 좋다



03 p.16 · CAR의 후보 키 두 개 중 License_number를 PK로

---

§ 개체 무결성 제약

"기본 키 값은 NULL일 수 없다" → 모든 투플 t에 대해 t[PK] ≠ NULL.

- 이유: PK가 NULL인 투플이 둘 이상이면, 다른 릴레이션에서 참조할 때 **서로 구별할 수 없다**
- 예) Office_phone처럼 NULL이 있는 속성은 키가 될 수 없음
- 복합 PK면 **PK의 어느 부분도** NULL이면 안 됨

---

§ 참조 무결성 제약

"투플 t_1의 FK 값은 어떤 투플 t_2의 PK 값과 같거나, NULL이어야 한다"

- **외래 키 (FK)**: 릴레이션 R_1의 속성 집합으로, R_2를 **참조**한다. $t_1[FK] = t_2[PK]$이면 t_1이 t_2를 참조
- R_1의 FK는 R_2의 PK와 **같은 도메인**을 가진다
- R_1 = **참조하는(referencing) 릴레이션**, R_2 = **참조되는(referenced) 릴레이션**
- 예) EMPLOYEE.Dno → DEPARTMENT.Dnumber. 'John Smith'의 Dno=5 → 'Research' 부서 투플을 참조 = Smith는 Research에서 일함
- 필요한 이유: 두 릴레이션 투플 사이의 **일관성** 유지. Dno=3인데 3번 부서가 없으면 그 값의 의미를 해석할 수 없다
- FK는 **자기 자신의 릴레이션을 참조**할 수도 있다 (EMPLOYEE.Super_ssn → EMPLOYEE.Ssn)

교안 연습 (FK 찾기):

- SALESPERSON(Ssn, Name, Start_year, Dept_no), TRIP(Ssn, From_city, To_city, Departure_date, Return_date, Trip_id), EXPENSE(Trip_id, Account#, Amount)
  → TRIP.Ssn → SALESPERSON, EXPENSE.Trip_id → TRIP
- STUDENT(Ssn, ...), COURSE(Course#, ...), ENROLL(Ssn, Course#, Quarter, Grade), BOOK_ADOPTION(Course#, Quarter, Book_isbn), TEXT(Book_isbn, ...)
  → ENROLL.Ssn → STUDENT, ENROLL.Course# → COURSE, BOOK_ADOPTION.Course# → COURSE, BOOK_ADOPTION.Book_isbn → TEXT

Referential Integrity Constraint

❑ What is a *Referential Integrity Constraint*?

- ***"A value of FK in a tuple t_1 either has a value of PK for some tuple t_2 or is NULL"***
  - *Foreign Key (FK):* a set of attributes in relation schema R_1 **references** R_2
  - i.e., $t_1(FK) = t_2(PK)$ (the tuple t_1 **references** the tuple t_2)
- The FK of R_1 has the same domain(s) as the PK of R_2



18

03 p.18 · EMPLOYEE.Dno(FK) → DEPARTMENT.Dnumber(PK)

갱신 연산과 제약 위반

교안 범위: 3-1 p.23–30

§ 삽입 (Insert)

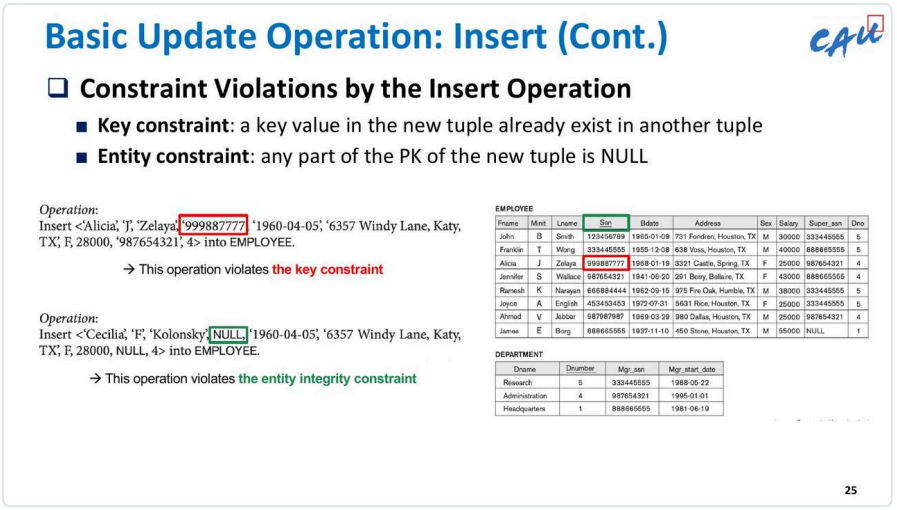
연산 종류: **검색(retrieval)** 과 **갱신(update)**. 기본 갱신 연산 = Insert, Delete, Update(Modify). 적용할 때마다 제약이 깨질 수 있다.

Insert: 릴레이션에 투플 하나 추가. 릴레이션 이름 + 값 리스트(속성 순서대로).

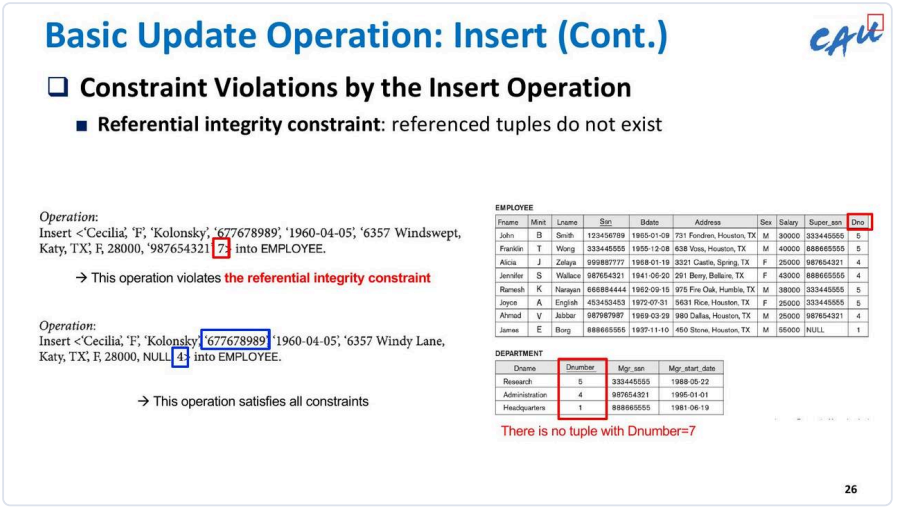
Insert는 세 가지 제약을 다 어길 수 있다.

위반	원인	교안 예 (COMPANY 상태)
키 제약	새 투플의 키 값이 이미 다른 투플에 있음	<'Alicia', ..., **'999887777'**, ..., 4> → Ssn 999887777은 이미 Zelaya
개체 무결성	새 투플의 PK(의 일부)가 NULL	<'Cecilia', ..., **NULL**, ..., 4>
참조 무결성	FK가 참조하는 투플이 없음	<'Cecilia', ..., '677678989', ..., **7**> → Dnumber=7인 부서 없음

<'Cecilia', 'F', 'Kolonsky', '677678989', ..., NULL, 4> 는 **모든 제약을 만족** (Super_ssn이 NULL인 건 FK니까 허용).



03 p.25 · Insert의 키 제약 위반, 개체 무결성 위반



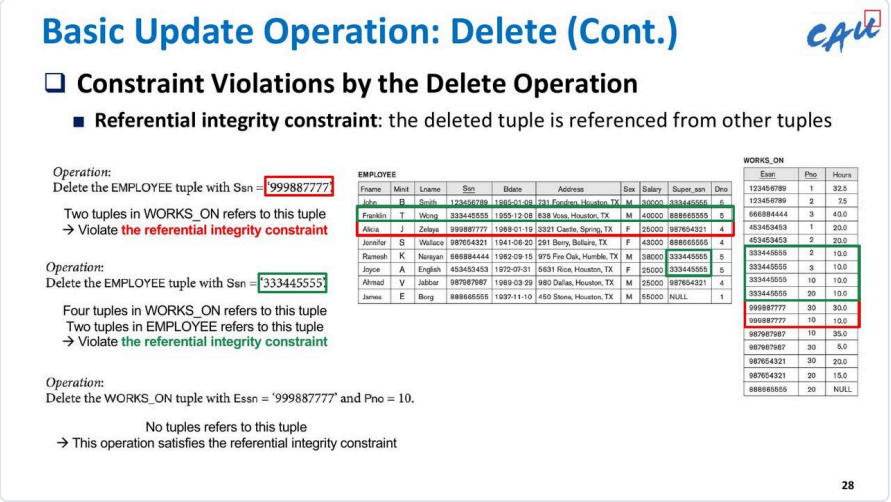
03 p.26 · Insert의 참조 무결성 위반 (Dno=7)과 모든 제약을 만족하는 경우

§ 삭제 (Delete)

Delete: 릴레이션 이름 + 지울 투플의 PK 값 지정.

Delete는 참조 무결성만 어길 수 있다 (지운 투플을 다른 투플이 참조하고 있을 때).

- EMPLOYEE에서 Ssn='999887777' 삭제 → WORKS_ON 투플 2개가 참조 → **위반**
- EMPLOYEE에서 Ssn='333445555' 삭제 → WORKS_ON 4개 + EMPLOYEE 2개(Super_ssn)가 참조 → **위반** (실제로는 DEPARTMENT.Mgr_ssn, DEPENDENT.Essn도 참조함)
- WORKS_ON에서 Essn='999887777' AND Pno=10 삭제 → 아무도 참조 안 함 → 만족



03 p.28 · Delete의 참조 무결성 위반

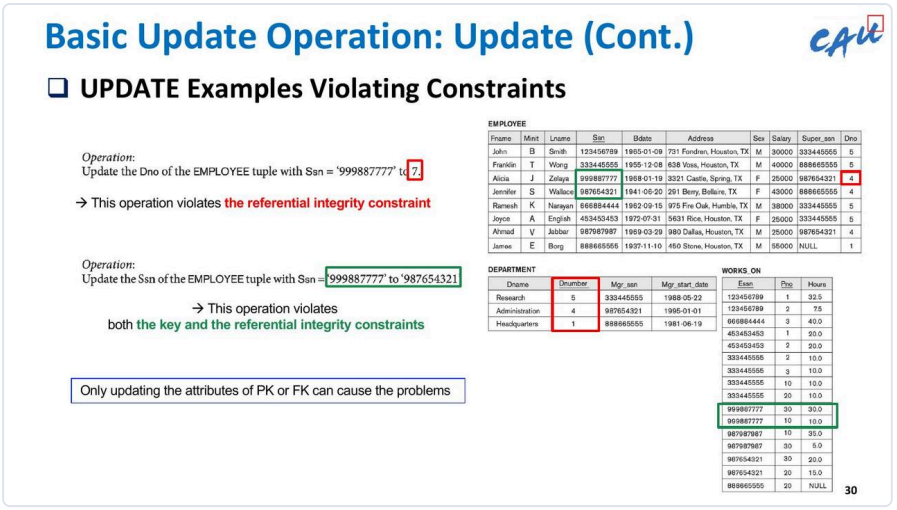
§ 수정 (Update)

Update: 바꿀 속성과 새 값 지정.

Update는 키 제약과 참조 무결성을 어길 수 있다. PK나 FK 속성을 바꿀 때만 문제가 된다.

- Ssn='999887777'인 사원의 Dno를 **7**로 → 7번 부서 없음 → **참조 무결성** 위반
- Ssn='999887777'을 **'987654321'** 로 → 이미 Wallace의 Ssn(키 제약) + WORKS_ON 등에서 999887777을 참조하던 투플이 고아가 됨(참조 무결성) → **둘 다** 위반
- PK를 NULL로 바꾸면 개체 무결성도 깨진다 (교안 요약엔 키·참조만 나옴)

> 시험 포인트: 요약표 그대로 — Insert = 키·개체·참조 **전부 가능**, Delete = **참조만**, Update = **키·참조** (PK/FK를 바꿀 때만).



03 p.30 · Update의 참조 무결성 위반, 키 + 참조 무결성 동시 위반

ER→관계 매핑 ① 개체 · 다중값 · 약한 개체

교안 범위: 3-1 p.32–36

§ 정규 개체 타입 매핑

매핑 순서: 개체 타입(정규·약한) → 다중값 속성 → 관계 타입(1:1, 1:N, M:N, n진).

정규(regular) 개체 타입 E

1. E의 **단순 속성을 모두** 포함하는 릴레이션 R 생성. 복합 속성은 **단순 구성 속성만** 넣음 (Name → Fname, Minit, Lname)
2. E의 키 속성 중 하나를 R의 **PK**로 지정 (나머지 키는 UNIQUE 후보 키)
3. 다중값 속성은 여기서 넣지 않음 → 다음 단계

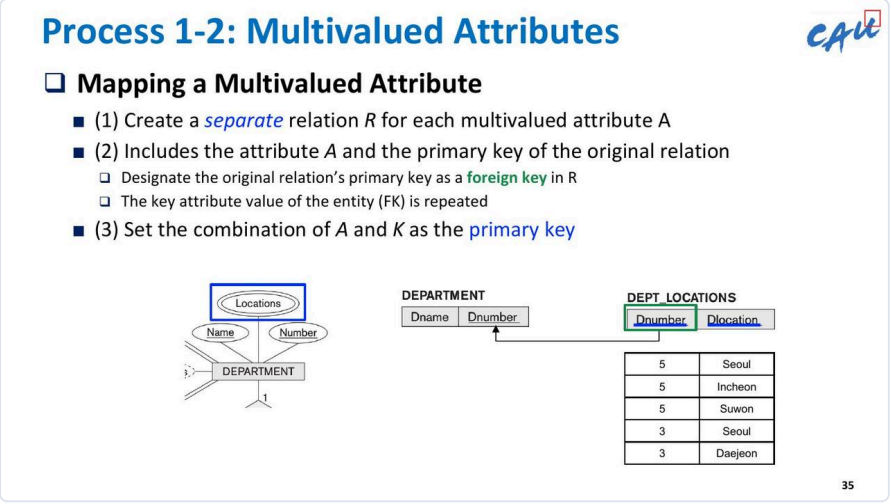
예) DEPARTMENT(Dname, **Dnumber**) — Locations는 아직 빠짐.

§ 다중값 속성 매핑

다중값 속성 A (릴레이션에서는 허용 안 됨)

1. A마다 **별도 릴레이션 R** 생성
2. R에 A와, 원래 릴레이션의 PK K를 넣는다. K는 R에서 **FK** (원래 릴레이션 참조). 키 값이 반복됨
3. **(A, K) 조합을 PK**로

예) DEPT_LOCATIONS(**Dnumber**, **Dlocation**), Dnumber → DEPARTMENT. (5, Seoul), (5, Incheon), (5, Suwon), (3, Seoul), (3, Daejeon)



03 p.35 · 다중값 속성 Locations → DEPT_LOCATIONS(Dnumber, Dlocation)

§ 약한 개체 타입 매핑

약한 개체 타입 W

1. W의 단순 속성을 모두 포함하는 릴레이션 R 생성 (복합은 구성 속성만)
2. **소유 개체 타입의 PK를 FK로** 포함 → 이게 **식별 관계 타입을 매핑**한 것
3. **소유자 PK + 부분 키 조합을 PK**로

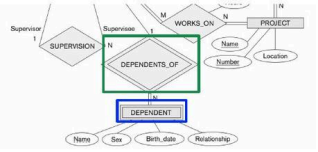
예) DEPENDENT(**Essn**, **Dependent_name**, Sex, Bdate, Relationship), Essn → EMPLOYEE.Ssn

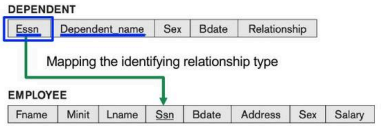
함정: 식별 관계(DEPENDENTS_OF)는 별도 릴레이션을 만들지 않는다. 소유자 PK를 FK로 넣는 것 자체가 식별 관계의 매핑.

Process 2: Weak Entity Types

❑ Mapping a Weak Entity Type

- (1) Create a relation *R* including all the simple attributes of the weak entity type *W*
  - For a composite attribute, include only the simple component attributes of the composite attribute
- (2) Include the primary key of the owner entity type as a ***foreign key***
  - This takes care of mapping the identifying relationship type of *W*
- (3) Set the combination of the owner's PK and the partial key as the primary key





36

03 p.36 · 약한 개체 DEPENDENT → PK (Essn, Dependent_name)

ER→관계 매핑 ② 관계 타입

교안 범위: 3-1 p.37–49

§ 1:1 관계 매핑

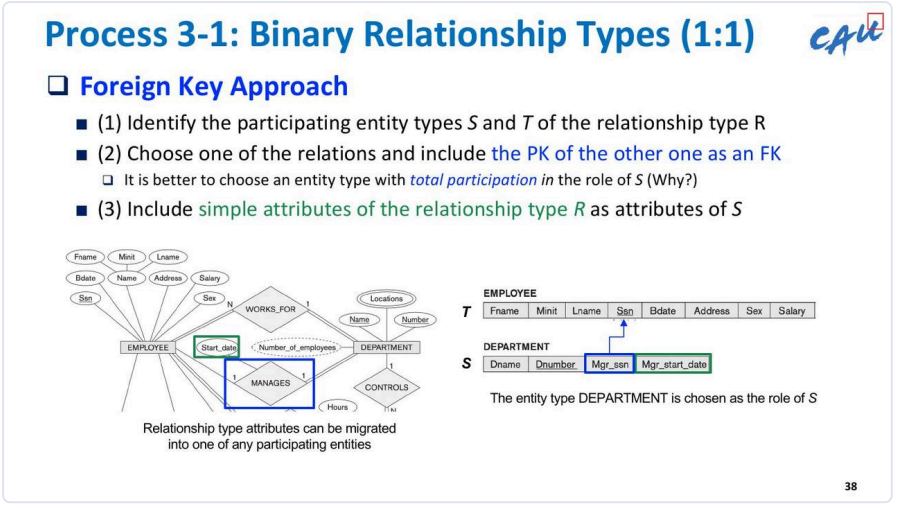
방법 3가지: **외래 키 방식** (가장 많이 쓰고 유용), **합병 릴레이션 방식** (두 개체를 한 릴레이션으로), **교차 참조(관계 릴레이션) 방식**.

외래 키 방식

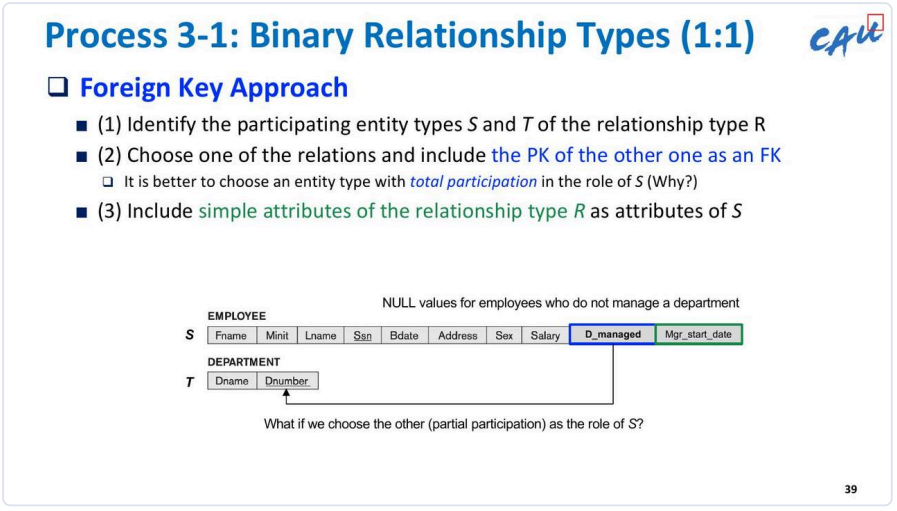
1. 참여 개체 타입 S, T 확인
2. 한쪽(S)을 골라 **상대(T)의 PK를 FK로** 넣는다. **전체 참여 쪽을 S로** 고르는 게 낫다
3. 관계의 단순 속성도 S에 넣는다

예) MANAGES: DEPARTMENT(전체 참여)를 S로 → DEPARTMENT(Dname, **Dnumber**, Mgr_ssn, Mgr_start_date), Mgr_ssn → EMPLOYEE.

왜 전체 참여 쪽? 반대로 EMPLOYEE(부분 참여)를 S로 하면 EMPLOYEE에 D_managed, Mgr_start_date가 들어가는데, 부서를 관리하지 않는 대부분의 사원은 이 값이 **NULL**이 된다. 전체 참여 쪽에 넣으면 NULL이 생기지 않는다.



03 p.38 · 1:1 MANAGES: 전체 참여 쪽 DEPARTMENT에 Mgr_ssn, Mgr_start_date



03 p.39 · 부분 참여 쪽 EMPLOYEE를 고르면 NULL이 많아짐

§ 1:N 관계 매핑

방법 2가지: **외래 키 방식** (릴레이션 수가 줄어서 선호), 교차 참조 방식.

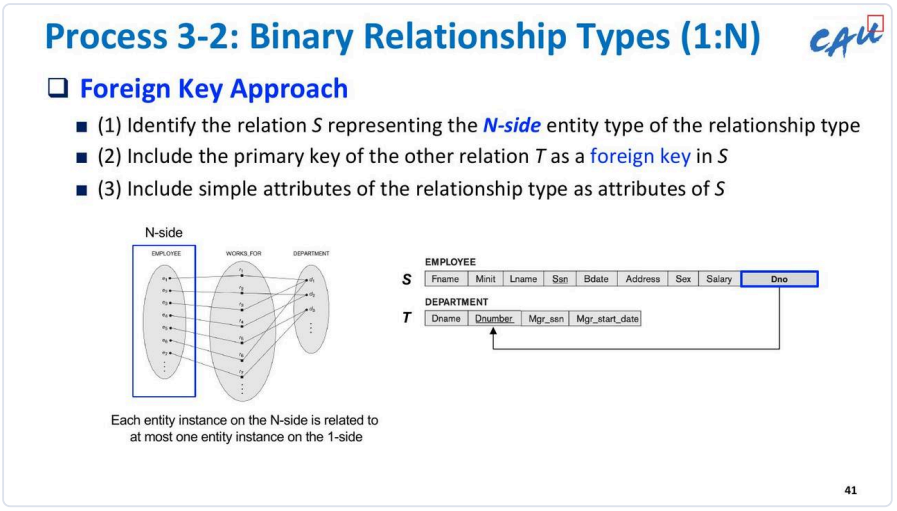
외래 키 방식

1. **N쪽** 개체 타입을 나타내는 릴레이션 S 확인
2. S에 상대(1쪽) 릴레이션 T의 **PK를 FK로** 넣는다
3. 관계의 단순 속성도 S에 넣는다

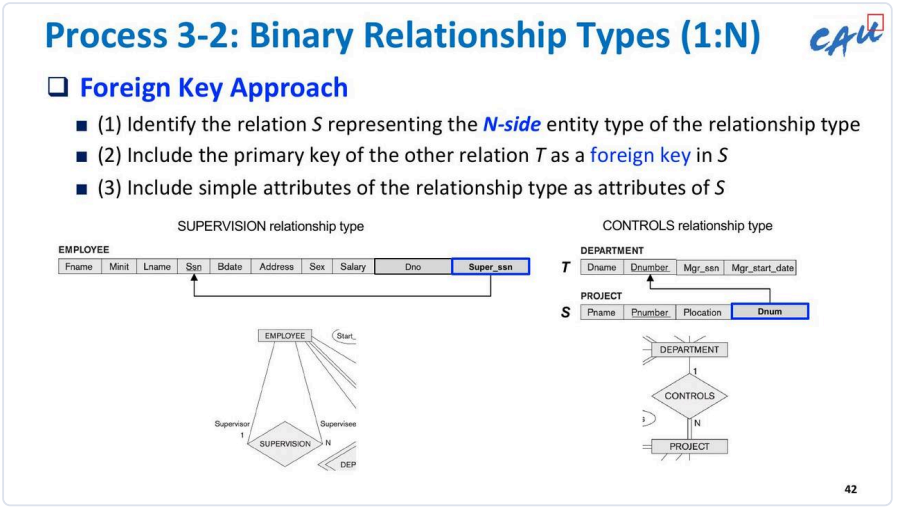
이유: N쪽 개체 인스턴스 하나는 1쪽 개체 **최대 하나**와 연결 → FK 값 하나로 표현 가능.

- WORKS_FOR → EMPLOYEE에 **Dno** (→ DEPARTMENT)
- SUPERVISION → EMPLOYEE에 **Super_ssn** (→ EMPLOYEE, 자기 참조)
- CONTROLS → PROJECT에 **Dnum** (→ DEPARTMENT)

함정: 1쪽에 FK를 넣으면 다중값이 돼서 원자성을 어긴다. FK는 **항상 N쪽**.



03 p.41 · 1:N WORKS_FOR: N쪽 EMPLOYEE에 Dno



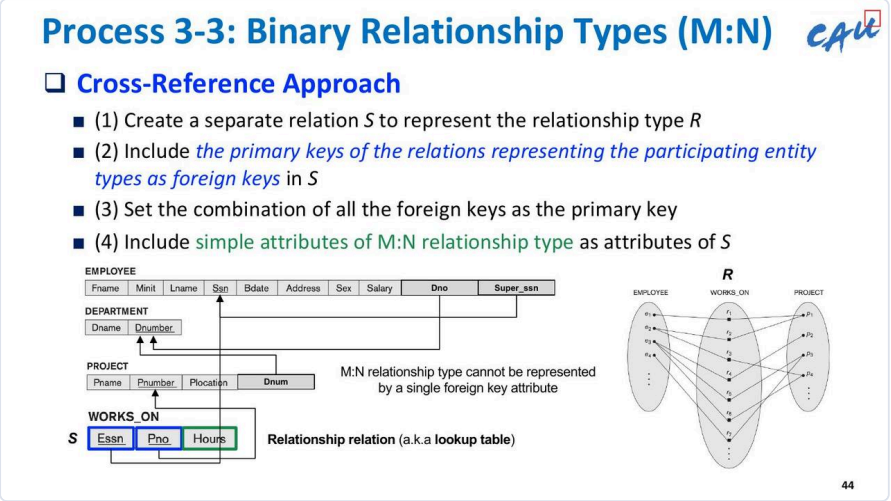
03 p.42 · SUPERVISION → Super_ssn, CONTROLS → Dnum

§ M:N 관계 매핑

방법 **하나뿐**: **교차 참조 (관계 릴레이션) 방식**. M:N은 FK 하나로 표현할 수 없다.

1. 관계 타입 R을 나타내는 **별도 릴레이션 S** 생성
2. 참여 개체 릴레이션들의 **PK를 FK로** S에 넣는다
3. **FK들의 조합을 PK**로
4. M:N 관계의 단순 속성도 S에 넣는다

예) WORKS_ON(**Essn**, **Pno**, Hours), Essn → EMPLOYEE, Pno → PROJECT. 관계 릴레이션 = lookup table.



03 p.44 · M:N WORKS_ON → 관계 릴레이션 WORKS_ON(Essn, Pno, Hours)

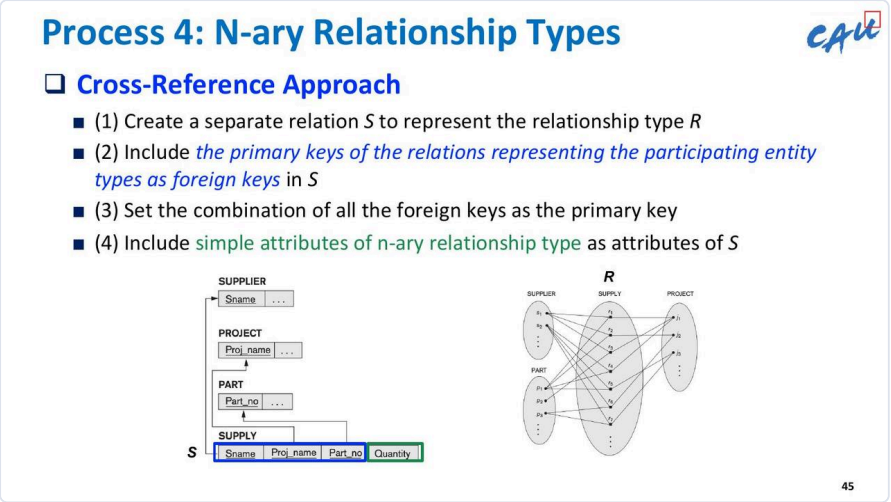
---

§ n진 관계 매핑

교차 참조 방식 (M:N과 같은 절차)

1. 별도 릴레이션 S 생성
2. 참여하는 n개 개체 릴레이션의 PK를 FK로 넣음
3. FK 전부의 조합을 PK로 (보통. 어느 참여에 max가 1이면 그 FK는 PK에서 빠질 수 있다)
4. 관계의 단순 속성 포함

예) SUPPLY(**Sname**, **Proj_name**, **Part_no**, Quantity) — SUPPLIER, PROJECT, PART 참조.



03 p.45 · 3진 SUPPLY → SUPPLY(Sname, Proj_name, Part_no, Quantity)

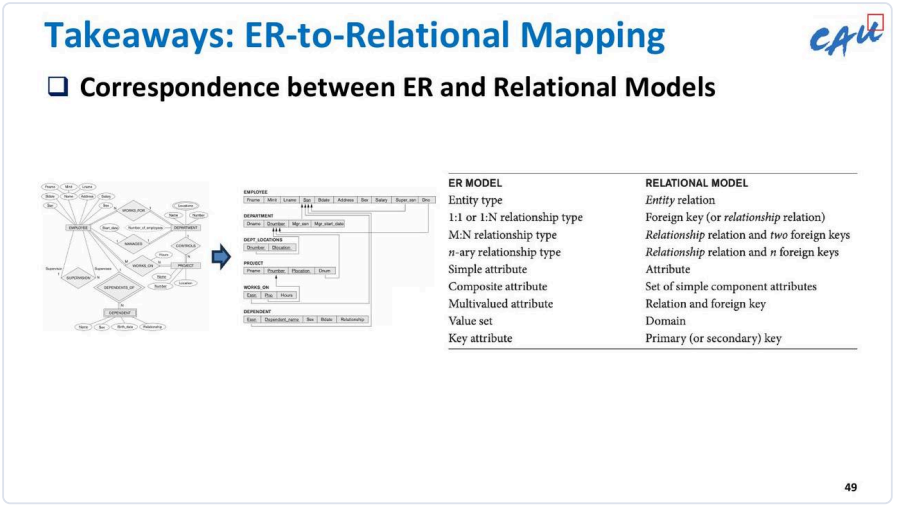
---

§ ER ↔ 관계 모델 대응

ER 모델	관계 모델
개체 타입	개체 릴레이션
1:1 또는 1:N 관계 타입	외래 키 (또는 관계 릴레이션)
M:N 관계 타입	관계 릴레이션 + FK **2개**
n진 관계 타입	관계 릴레이션 + FK **n개**
단순 속성	속성
복합 속성	단순 구성 속성들의 집합
다중값 속성	릴레이션 + 외래 키
값 집합 (value set)	도메인
키 속성	기본 키 (또는 보조 키)

교안 연습 3 (UNIVERSITY: COLLEGE, INSTRUCTOR, DEPT, STUDENT, COURSE, SECTION) 풀이 순서:

1. 정규 개체 → COLLEGE(CName, COffice, CPhone), INSTRUCTOR(Id, Rank, IName, IOffice, IPhone), DEPT(DName, DCode, DOffice, DPhone), COURSE(CCode, Credits, CoName, Level, CDesc), SECTION(SecId, SecNo, Sem, Year, Bldg, RoomNo, DaysTime), STUDENT(Sid, DOB, FName, MName, LName, Addr, Phone, Major)
2. 1:1 → DEAN: COLLEGE에 DeanId (COLLEGE가 (1,1) 전체 참여), CHAIR: DEPT에 ChairId, CStartDate
3. 1:N → INSTRUCTOR.Employed(EMPLOYS), DEPT.Admined(ADMINS), COURSE.Offered(OFFERS), SECTION.Taught(TEACHES), SECTION.Belongs(SECS), STUDENT.Dept(HAS)
4. M:N → TAKES(**Sid**, **SecId**, Grade)

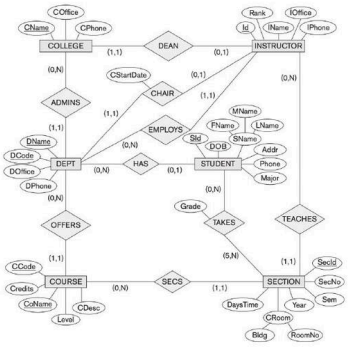


03 p.49 · ER 모델과 관계 모델의 대응표

Exercises: ER-to-Relational Mapping

❑ **Exercise 3:**

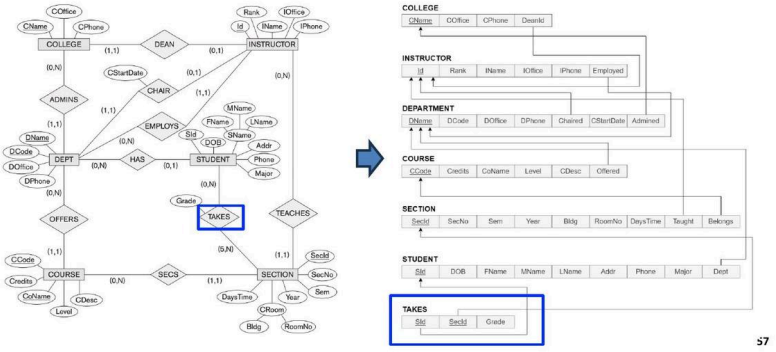
- Map the ER schema for UNIVERSITY database into a relational database schema



03 p.46 · 연습 3: UNIVERSITY ER 스키마

Exercises: ER-to-Relational Mapping (Cont.)

❑ **Step 4: Mapping of Binary Relationship Types (M:N)**



db-exercises p.57 · 연습 3 매핑 결과 (M:N TAKES까지)

U4 · 4-1_SQL.pdf + 3-2_setup-for-sql.pdf + SQL-week6-lab.pdf

Basic SQL (DDL · DML · SELECT)

CREATE TABLE의 제약조건 문법, 참조 트리거 동작(ON DELETE/UPDATE),
INSERT·DELETE·UPDATE가 무엇을 어기는지, 그리고 SELECT-FROM-WHERE. SQL은 직접 써봐야 늘어. 풀이 문제는 교재 COMPANY 상태(3-1 p.12)에서 실제로 돌려본 결과를 정답에 붙여놨어.

SQL 소개 · CREATE SCHEMA/TABLE · 제약조건

교안 범위: 4-1 p.3–10

§ SQL과 NL2SQL

- **SQL (Structured Query Language)**: 특수 목적 **선언적(declarative)** 언어. IBM의 첫 DBMS용으로 설계, 원래 이름 **SEQUEL** (Structured English Query Language). 'S-Q-L' 또는 'SEQUEL'로 읽음
- **DDL (Data Definition Language)**: DB와 **테이블**에 대한 명령. CREATE / ALTER / DROP (schema, table)
- **DML (Data Manipulation Language)**: 테이블 안 **투플**에 대한 명령. 검색 SELECT, 수정 INSERT / DELETE / UPDATE
- **NL2SQL**: 자연어 질의를 SQL로 바꾸는 기술. 어려움 = 자연어의 모호성, 스키마 이해·연결(schema linking), 처음 보는 DB로의 일반화, 복잡한 질의 생성. LLM 기반 방법도 아직 완벽과 거리가 멀다 → **SQL을 제대로 이해하는 게 중요**

> **수업 메모** 03주-2 (조교)
> 실습 환경: VS Code + Python(3.11 정도), pip install pandas sqlglot. pandas는 SQL 실행 결과를 CSV로 저장, sqlglot은 SQL 문법 검사. test.py에서 'change me'를 학번으로 바꾸는 것 말고는 코드를 고치지 말 것. NL_queries.txt를 보고 mysql.py에 SQL을 쓰고 test.py를 돌리면 학번 이름의 폴더가 생기고, 그걸 압축해 e-class에 제출. 압축 파일 이름은 반드시 학번.

§ CREATE SCHEMA / CREATE TABLE

SQL 스키마: 테이블, 타입, 제약조건, 뷰, 도메인 등 스키마를 기술하는 요소들. 스키마 이름으로 식별.

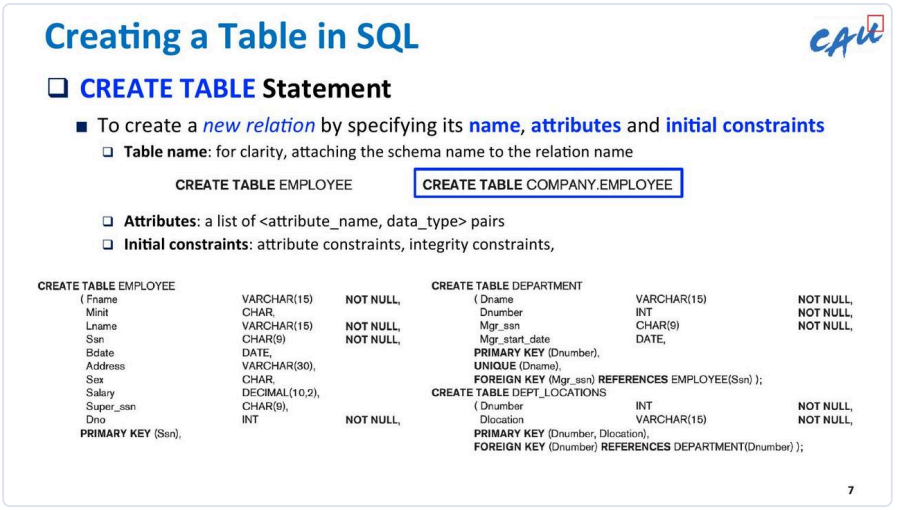
```
CREATE SCHEMA COMPANY AUTHORIZATION 'Jsmith';
```

- **권한 식별자 (authorization identifier)**: 스키마를 소유한 사용자

CREATE TABLE: 이름, 속성, 초기 제약조건을 지정해 새 릴레이션 생성.

- 테이블 이름에 스키마 이름을 붙일 수 있음: `CREATE TABLE COMPANY.EMPLOYEE`
- 속성 = <속성 이름, 데이터 타입> 쌍의 리스트
- 초기 제약조건 = 속성 제약, 무결성 제약

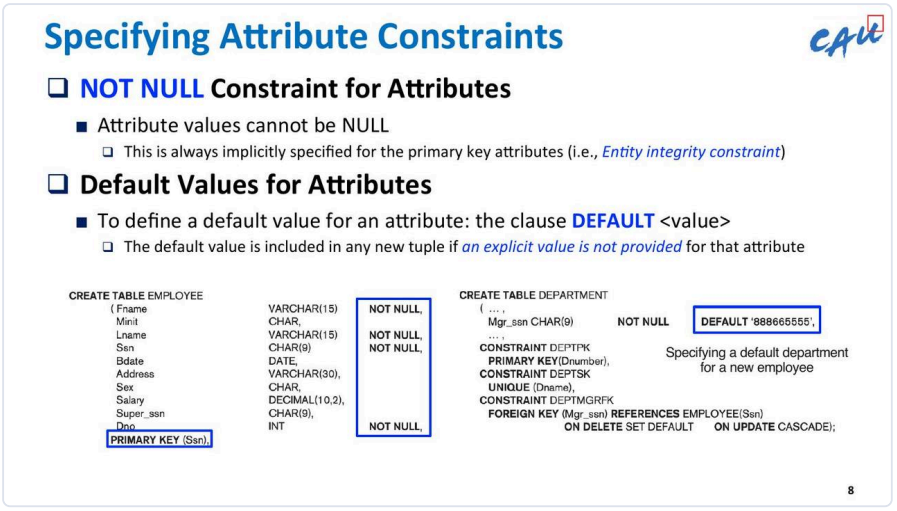
```
CREATE TABLE EMPLOYEE
( Fname      VARCHAR(15)    NOT NULL,
  Minit      CHAR,
  Lname      VARCHAR(15)    NOT NULL,
  Ssn        CHAR(9)        NOT NULL,
  Bdate      DATE,
  Address    VARCHAR(30),
  Sex        CHAR,
  Salary     DECIMAL(10,2),
  Super_ssn  CHAR(9),
  Dno        INT            NOT NULL,
  PRIMARY KEY (Ssn) );
```



04 p.7 · CREATE TABLE EMPLOYEE, DEPARTMENT, DEPT_LOCATIONS

§ 속성 제약: NOT NULL, DEFAULT

- **NOT NULL**: 그 속성 값은 NULL이 될 수 없다. **PK 속성에는 항상 암묵적으로** 적용 (= 개체 무결성)
- **DEFAULT <값>**: 새 투플에 그 속성 값을 명시하지 않으면 기본값이 들어감
  - 예) 새 사원의 기본 부서: `Dno INT NOT NULL DEFAULT 1`, DEPARTMENT의 `Mgr_ssn CHAR(9) NOT NULL DEFAULT '888665555'`



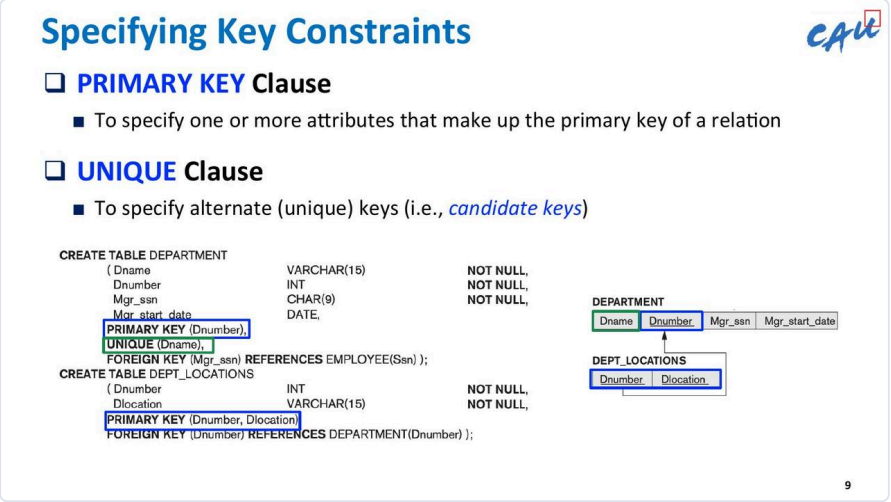
04 p.8 · NOT NULL과 DEFAULT

§ 키 제약: PRIMARY KEY, UNIQUE, FOREIGN KEY

```
CREATE TABLE DEPARTMENT
( Dname           VARCHAR(15)  NOT NULL,
  Dnumber         INT          NOT NULL,
  Mgr_ssn         CHAR(9)      NOT NULL,
  Mgr_start_date  DATE,
  PRIMARY KEY (Dnumber),
  UNIQUE (Dname),
  FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) );
CREATE TABLE DEPT_LOCATIONS
( Dnumber    INT          NOT NULL,
  Dlocation  VARCHAR(15)  NOT NULL,
  PRIMARY KEY (Dnumber, Dlocation),
  FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber) );
```

- **PRIMARY KEY**: PK를 이루는 속성(들). 복합이면 괄호 안에 여러 개
- **UNIQUE**: 대체(alternate) 키 = **후보 키**
- **FOREIGN KEY ... REFERENCES 테이블(속성)**: 다른 릴레이션의 PK를 참조하는 FK
- 제약에 이름 붙이기: `CONSTRAINT DEPTPK PRIMARY KEY (Dnumber)` → 나중에 ALTER로 지울 때 이름이 필요

> 시험 포인트: 참조되는 PK가 갱신·삭제되면 참조 무결성이 깨질 수 있는데, **SQL의 기본 동작은 그 연산을 거부(reject)** 하는 것. 다른 방법 = 참조 트리거 동작 (L4-2).



04 p.9 · PRIMARY KEY, UNIQUE, FOREIGN KEY

참조 트리거 동작 · DROP · ALTER

교안 범위: 4-1 p.11–20

§ 참조 트리거 동작 (Referential Triggered Actions)

"다른 투플이 참조하는 투플이 삭제/갱신되면?" 에 대한 동작을 FK에 붙인다.

- **트리거**: `ON DELETE`, `ON UPDATE`
- **동작**: `SET NULL`, `CASCADE`, `SET DEFAULT` (지정 안 하면 기본 = 거부, 교안 연습에서는 REJECT라고 부름)

동작	참조되던 투플이 삭제될 때	갱신될 때
SET NULL	참조하던 FK를 NULL로	FK를 NULL로
SET DEFAULT	FK를 기본값으로	FK를 기본값으로
CASCADE	**참조하던 투플도 삭제**	참조하던 FK를 **새 값으로** 바꿈

```
CREATE TABLE EMPLOYEE
( ...,
  Dno INT NOT NULL DEFAULT 1,
  CONSTRAINT EMPPK PRIMARY KEY (Ssn),
  CONSTRAINT EMPSUPERFK FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)
      ON DELETE SET NULL      ON UPDATE CASCADE,
  CONSTRAINT EMPDEPTFK  FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber)
      ON DELETE SET DEFAULT   ON UPDATE CASCADE );
```

교안 토론 (p.13): EMPSUPERFK의 ON DELETE를 **CASCADE**로 바꾸고 `DELETE FROM EMPLOYEE WHERE Lname = 'Borg';` 를 실행하면?

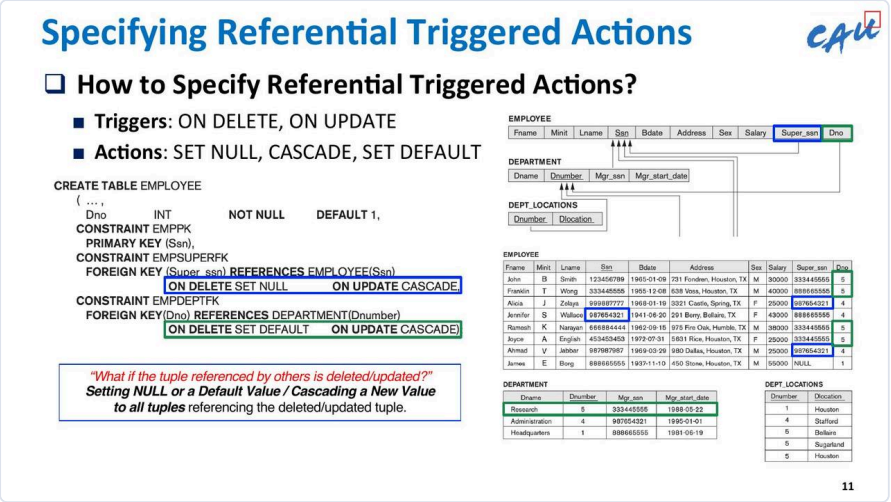
- Borg(888665555)를 상사로 둔 Wong, Wallace가 같이 삭제 → 그들을 상사로 둔 Smith, Narayan, English, Zelaya, Jabbar도 연쇄 삭제 → **EMPLOYEE 투플 8개가 전부 사라진다**. CASCADE는 재귀적으로 전파된다는 점이 핵심
- (EMPSUPERFK만 봤을 때 얘기. 실제로는 WORKS_ON.Essn, DEPENDENT.Essn 같은 다른 FK가 기본 동작(거부)이면, 지워질 사원을 그 투플들이 참조하고 있어서 DELETE 전체가 거부된다)

교안 연습 1 (LIBRARY) 정답 예:

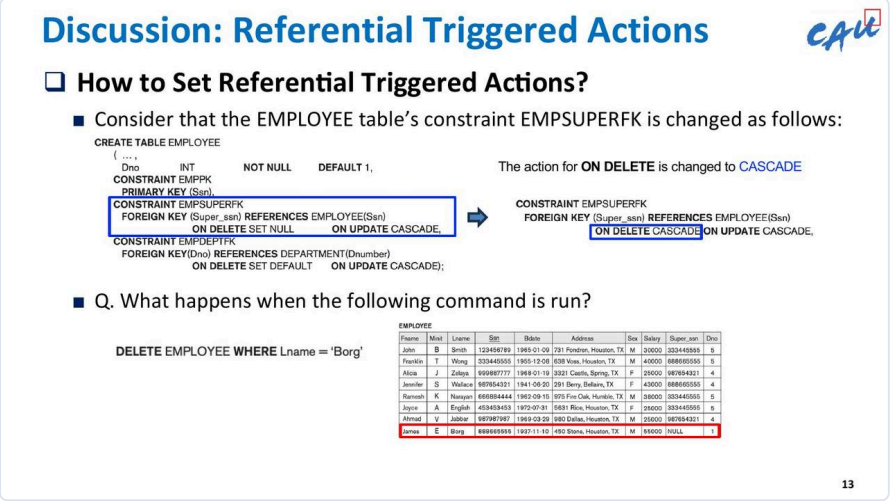
FK	ON DELETE	ON UPDATE	이유
BOOK_AUTHORS.Book_id → BOOK	CASCADE	CASCADE	책이 지워지면 저자 정보도 필요 없음
BOOK.Publisher_name → PUBLISHER	REJECT	CASCADE	그 출판사 책이 있으면 출판사 삭제 금지
BOOK_COPIES.Book_id → BOOK	CASCADE	CASCADE	책 목록에서 빼면 사본도 관리 안 함
BOOK_COPIES.Branch_id → LIBRARY_BRANCH	CASCADE	CASCADE	지점 삭제 = 폐점
BOOK_LOANS.Book_id → BOOK	REJECT (또는 CASCADE)	CASCADE	목록에서 빼도 대출된 책은 돌려받아야 함
BOOK_LOANS.Branch_id → LIBRARY_BRANCH	CASCADE (또는 REJECT)	CASCADE	폐점하면 대출을 탕감

FK	ON DELETE	ON UPDATE	이유
BOOK_LOANS.Card_no → BORROWER	REJECT	CASCADE	책을 다 반납해야 회원 탈퇴 가능

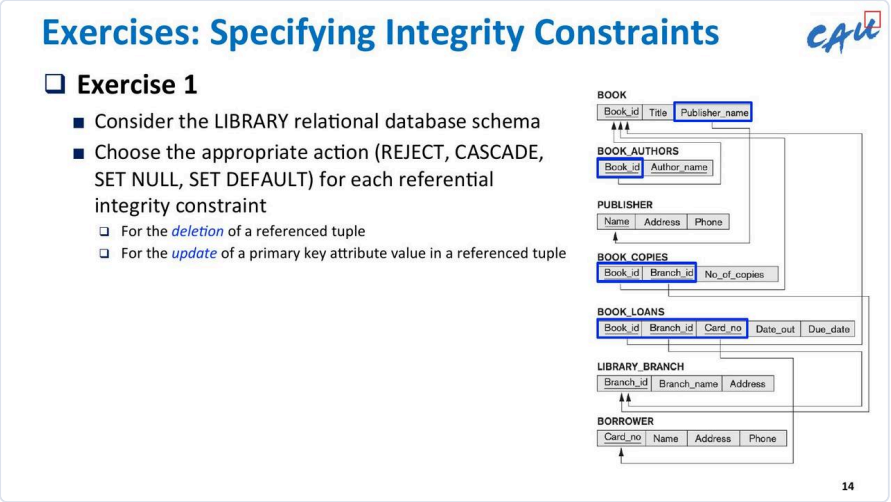
> 시험 포인트: 정답이 하나가 아니라 **이유가 맞으면** 인정되는 유형. ON UPDATE는 거의 다 CASCADE (키가 바뀌면 참조도 따라가야 추적 가능).



04 p.11 · ON DELETE SET NULL / SET DEFAULT, ON UPDATE CASCADE



04 p.13 · 토론: EMPSUPERFK를 ON DELETE CASCADE로 바꾸고 Borg 삭제



04 p.14 · 연습 1: LIBRARY 스키마

§ DROP SCHEMA / DROP TABLE

스키마 수정 = 투플이 아니라 **스키마**를 바꾸는 것 (테이블·속성·제약 추가/삭제).

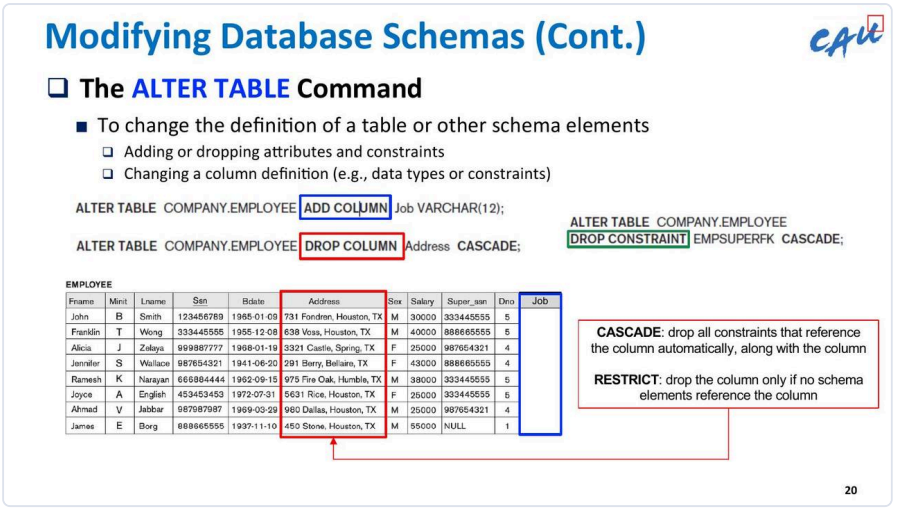
- **DROP SCHEMA**: 스키마 전체 삭제
  - `CASCADE` : 스키마의 모든 요소(테이블, 도메인 등)까지 삭제
  - `RESTRICT` : 스키마에 **요소가 하나도 없을 때만** 삭제 (사용자가 요소를 하나씩 지워야 함)
- **DROP TABLE**: 특정 테이블 삭제. 모든 투플 + **카탈로그의 테이블 정의까지** 제거
  - 투플만 지우고 정의는 남기려면 **DELETE** 명령
  - `CASCADE` : 그 테이블을 참조하는 제약과 요소도 자동으로 같이 삭제
  - `RESTRICT` : 어떤 제약이나 요소도 그 테이블을 참조하지 않을 때만 삭제

§ ALTER TABLE

테이블 정의 변경: 속성·제약 추가/삭제, 열 정의(타입, 제약) 변경.

```
ALTER TABLE COMPANY.EMPLOYEE ADD COLUMN Job VARCHAR(12);
ALTER TABLE COMPANY.EMPLOYEE DROP COLUMN Address CASCADE;
ALTER TABLE COMPANY.EMPLOYEE DROP CONSTRAINT EMPSUPERFK CASCADE;
```

- DROP COLUMN `CASCADE` : 그 열을 참조하는 제약도 자동으로 같이 삭제 / `RESTRICT` : 그 열을 참조하는 요소가 없을 때만
- ADD COLUMN으로 추가된 열은 기존 투플에서 NULL (또는 DEFAULT 값). 그래서 새 열에 NOT NULL을 걸려면 DEFAULT가 필요



04 p.20 · ALTER TABLE ADD COLUMN / DROP COLUMN / DROP CONSTRAINT

INSERT · DELETE · UPDATE

교안 범위: 4-1 p.22–27

§ INSERT

사용법 1: 모든 속성 값 — 테이블을 만들 때의 속성 **순서대로** 값을 나열

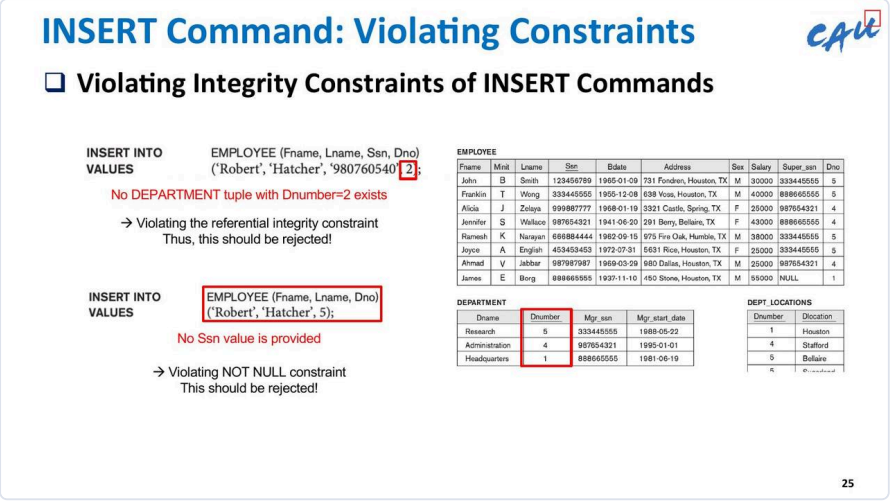
```
INSERT INTO EMPLOYEE
VALUES ('Richard', 'K', 'Marini', '653298653', '1962-12-30',
        '98 Oak Forest, Katy, TX', 'M', 37000, '653298653', 4);
```

사용법 2: 속성 이름 명시 — **NOT NULL이면서 DEFAULT가 없는 속성은 반드시** 포함

```
INSERT INTO EMPLOYEE (Fname, Lname, Dno, Ssn)
VALUES ('Richard', 'Marini', 4, '653298653');
```

제약 위반 예 (거부됨)

- `INSERT INTO EMPLOYEE (Fname, Lname, Ssn, Dno) VALUES ('Robert', 'Hatcher', '980760540', 2);` → Dnumber=2인 부서 없음 → **참조 무결성**
- `INSERT INTO EMPLOYEE (Fname, Lname, Dno) VALUES ('Robert', 'Hatcher', 5);` → Ssn 없음 → **NOT NULL** (PK라 개체 무결성)



04 p.25 · INSERT의 참조 무결성 위반과 NOT NULL 위반

§ DELETE와 UPDATE

DELETE: WHERE로 **한 릴레이션**에서 지울 투플을 고름

- 한 번에 한 테이블에서만 명시적으로 삭제. 참조 트리거 동작(ON DELETE CASCADE)이 있으면 다른 릴레이션으로 전파될 수 있다
- 지워지는 투플 수는 경우마다 다름 (COMPANY 상태 기준):

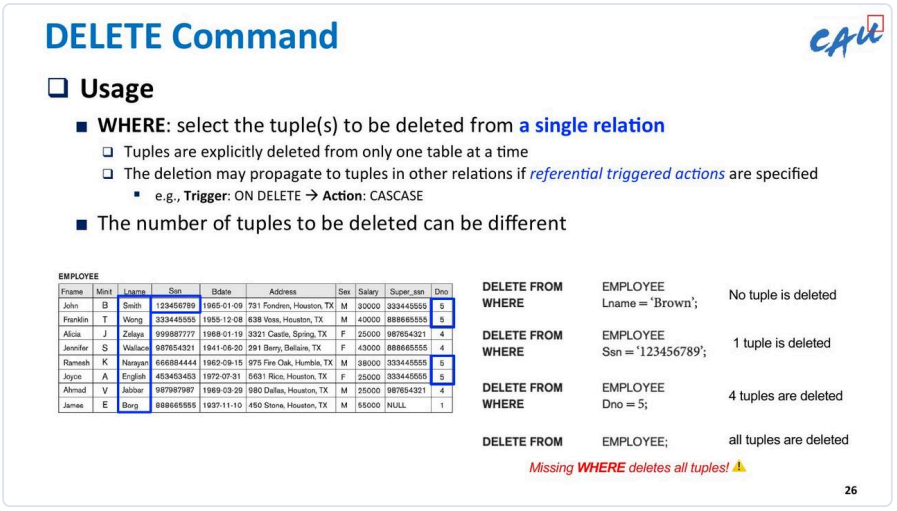
```
DELETE FROM EMPLOYEE WHERE Lname = 'Brown';        -- 0개
DELETE FROM EMPLOYEE WHERE Ssn = '123456789';      -- 1개
DELETE FROM EMPLOYEE WHERE Dno = 5;                -- 4개
DELETE FROM EMPLOYEE;                              -- 전부!
```

주의: **WHERE를 빼면 모든 투플이 삭제된다.** 테이블 정의는 남음 (DROP TABLE과 다름).

UPDATE: WHERE로 **한 릴레이션**에서 고친 투플을 고르고, **SET**으로 바꿀 속성과 새 값 지정

```
UPDATE PROJECT  SET Plocation = 'Bellaire', Dnum = 5  WHERE Pnumber = 10;
UPDATE EMPLOYEE SET Salary = Salary * 1.1             WHERE Dno = 5;
```

- UPDATE 하나로 여러 투플을 고칠 수 있다 (Dno=5인 4명이 한꺼번에 10% 인상)
- ON UPDATE CASCADE가 있으면 다른 릴레이션으로 전파



04 p.26 · DELETE로 지워지는 투플 수 (0, 1, 4, 전부)

SELECT-FROM-WHERE · 별칭

교안 범위: 4-1 p.28–34

§ SELECT-FROM-WHERE 블록

```
SELECT  <attribute list>      -- 투영 속성 (projection attributes)
FROM    <table list>          -- 질의에 필요한 릴레이션
WHERE   <condition>;          -- 선택 조건 + 조인 조건
```

질의 1: 'John B. Smith'의 생일과 주소

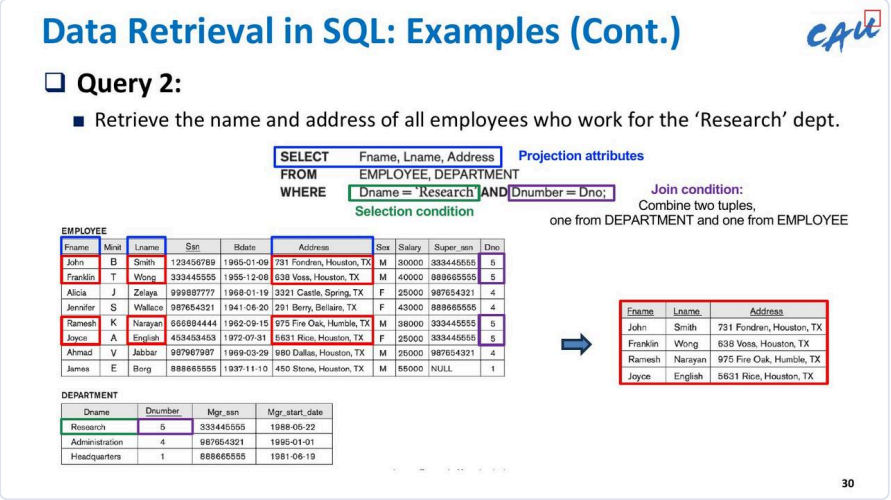
```
SELECT Bdate, Address
FROM   EMPLOYEE
WHERE  Fname = 'John' AND Minit = 'B' AND Lname = 'Smith';
```

→ 1965-01-09, 731 Fondren, Houston, TX

질의 2: 'Research' 부서에서 일하는 모든 사원의 이름과 주소

```
SELECT Fname, Lname, Address
FROM   EMPLOYEE, DEPARTMENT
WHERE  Dname = 'Research' AND Dnumber = Dno;
```

- `Dname = 'Research'` = **선택 조건** (한 테이블에서 관심 있는 투플 고르기)
- `Dnumber = Dno` = **조인 조건** (DEPARTMENT 투플과 EMPLOYEE 투플을 결합)
- 결과: Smith, Wong, Narayan, English (4명)



04 p.30 · 질의 2: 선택 조건 Dname='Research' + 조인 조건 Dnumber=Dno

§ 모호한 속성 이름과 별칭

자격 부여 (qualify): 다른 릴레이션에 같은 이름의 속성이 있으면 `릴레이션이름.속성`으로 구분.

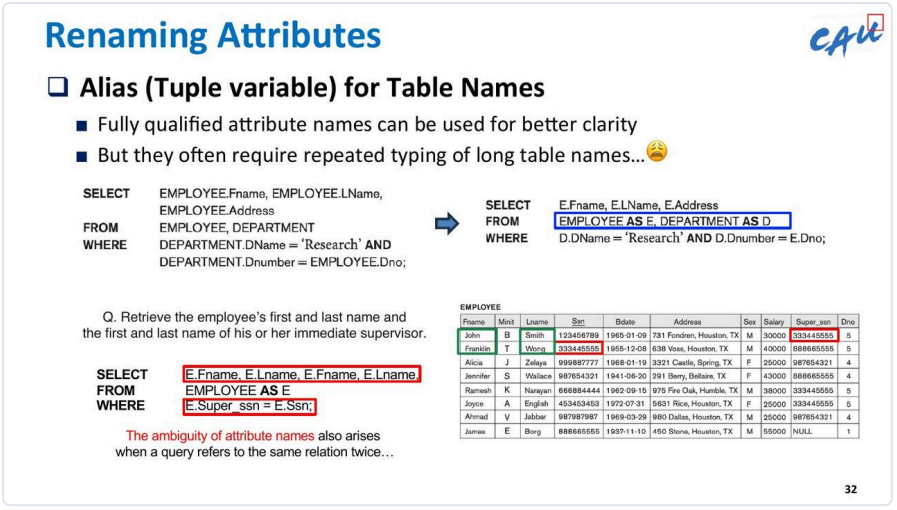
```
SELECT Fname, EMPLOYEE.Name, Address
FROM   EMPLOYEE, DEPARTMENT
WHERE  DEPARTMENT.Name = 'Research' AND DEPARTMENT.Dnumber = EMPLOYEE.Dnumber;
```

별칭 (alias, 투플 변수): 긴 이름 반복을 줄이고, **같은 릴레이션을 두 번** 참조할 때 필수.

```
SELECT E.Fname, E.Lname, S.Fname, S.Lname
FROM   EMPLOYEE AS E, EMPLOYEE AS S
WHERE  E.Super_ssn = S.Ssn;
```

→ 사원과 그 직속 상사의 이름. E = 사원 역할, S = 상사 역할. 상사가 없는 Borg는 결과에 없다 (7행).

함정: 교안 p.32의 `FROM EMPLOYEE AS E WHERE E.Super_ssn = E.Ssn` 은 별칭이 하나라서 "자기 자신이 자기 상사인 사원"을 찾는 질의가 된다 (결과 0행). 같은 테이블을 두 역할로 쓰려면 별칭이 **두 개** 필요하다는 걸 보여주는 예.



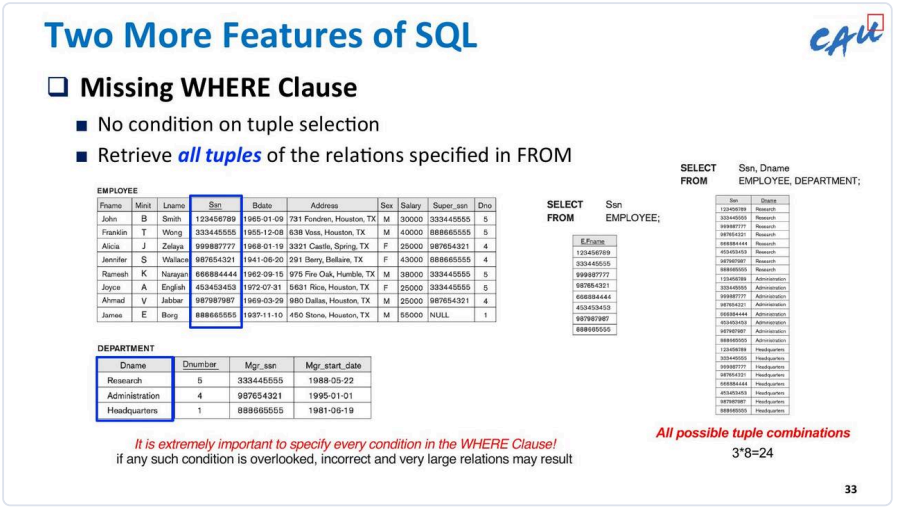
04 p.32 · 별칭 (같은 테이블을 두 번 쓰는 사원–상사 질의)

§ WHERE 생략과 *

WHERE 생략: 투플 선택 조건이 없음 → FROM의 릴레이션 투플을 전부 검색.

- 릴레이션이 둘 이상이면 **가능한 모든 투플 조합 (카티션 곱)** 이 나온다
- `SELECT Ssn, Dname FROM EMPLOYEE, DEPARTMENT;` → 8 × 3 = **24행**
- 조건 하나라도 빠뜨리면 틀리고 아주 큰 결과가 나오니 **WHERE에 모든 조건을 다 써야** 한다

별표 (*): 선택된 투플의 **모든 속성** 값. `SELECT * FROM EMPLOYEE WHERE Dno = 5;` → 4행, 10열.



04 p.33 · WHERE 생략 → 8 × 3 = 24개 조합

다중집합 · 집합 연산 · ORDER BY · 연산자

교안 범위: 4-1 p.35–41

§ 다중집합과 DISTINCT

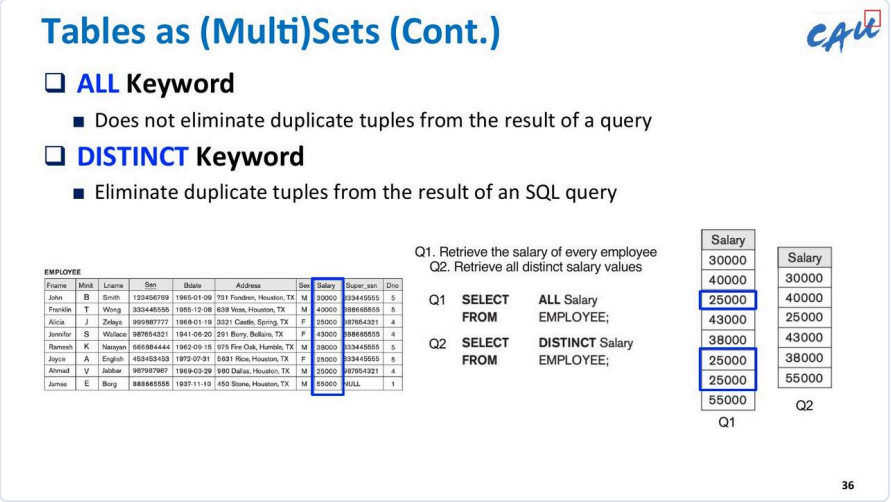
SQL은 테이블을 집합이 아니라 다중집합(multiset)으로 다룬다 → 테이블이나 질의 결과에 **중복 투플**이 있을 수 있고, SQL은 **중복을 자동으로 제거하지 않는다**.

왜?

- 중복 제거는 **비싼** 연산 (투플을 정렬한 다음 중복을 지워야 함)
- 사용자가 중복을 보고 싶을 때도 있음
- 집계 함수를 쓸 때 중복이 필요 (예: 급여 평균) → 5주차

키워드

- **ALL** (기본): 중복 유지. `SELECT ALL Salary FROM EMPLOYEE;` → 8행 (25000이 3번)
- **DISTINCT**: 중복 제거. `SELECT DISTINCT Salary FROM EMPLOYEE;` → 6행



04 p.36 · ALL Salary (8행) vs DISTINCT Salary (6행)

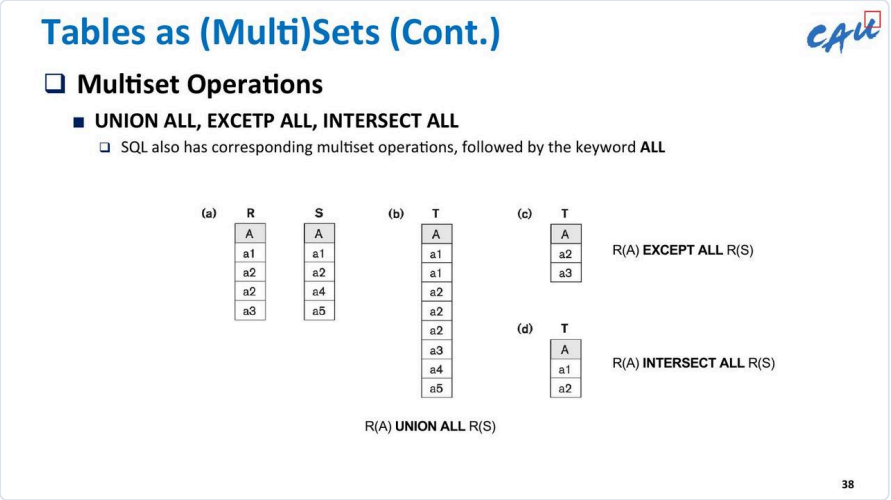
§ 집합 연산 UNION / EXCEPT / INTERSECT

- **UNION**(합), **EXCEPT**(차), **INTERSECT**(교). 결과는 투플의 **집합** → **중복 제거됨**
- **타입 호환(type-compatible)** 되는 릴레이션에만 적용 (속성 수가 같고 대응 속성의 도메인이 같음)
- 다중집합 버전: **UNION ALL, EXCEPT ALL, INTERSECT ALL** (중복 유지)
  - R(A) = {a1, a2, a2, a3}, S(A) = {a1, a2, a4, a5}
  - R UNION ALL S = {a1, a1, a2, a2, a2, a3, a4, a5}
  - R EXCEPT ALL S = {a2, a3}
  - R INTERSECT ALL S = {a1, a2}

질의 4: 성이 'Smith'인 사원이 **일꾼**으로든 그 프로젝트를 통제하는 부서의 **관리자**로든 관련된 프로젝트 번호

```
( SELECT DISTINCT Pnumber
  FROM   PROJECT, DEPARTMENT, EMPLOYEE
  WHERE  Dnum = Dnumber AND Mgr_ssn = Ssn AND Lname = 'Smith' )
UNION
( SELECT DISTINCT Pnumber
  FROM   PROJECT, WORKS_ON, EMPLOYEE
  WHERE  Pnumber = Pno AND Essn = Ssn AND Lname = 'Smith' );
```

→ Smith는 관리자가 아니라 첫 블록은 비고, 일꾼으로 1, 2 → 결과 {1, 2}



04 p.38 · UNION ALL, EXCEPT ALL, INTERSECT ALL

---

§ ORDER BY · 산술 연산 · BETWEEN

ORDER BY: 결과 투플을 속성 값 기준으로 정렬. **기본은 오름차순(ASC)**, 내림차순은 DESC.

```
SELECT   D.Dname, E.Lname, E.Fname, P.Pname
FROM     DEPARTMENT AS D, EMPLOYEE AS E, WORKS_ON AS W, PROJECT AS P
WHERE    D.Dnumber = E.Dno AND E.Ssn = W.Essn AND W.Pno = P.Pnumber
ORDER BY D.Dname DESC, E.Lname ASC, E.Fname ASC;
```

산술 연산자 + − * /: 숫자 속성에 적용. **릴레이션 상태는 바뀌지 않는다** (결과에만 반영)

```
SELECT E.Fname, E.Lname, 1.1 * E.Salary AS Increased_sal
FROM   EMPLOYEE AS E, WORKS_ON AS W, PROJECT AS P
WHERE  E.Ssn = W.Essn AND W.Pno = P.Pnumber AND P.Pname = 'ProductX';
```

→ Smith 33000, English 27500

BETWEEN: 범위 조건. `(Salary BETWEEN 30000 AND 40000)` = `((Salary ⩾ 30000) AND (Salary ⩽ 40000))` → **양 끝 포함**

```
SELECT * FROM EMPLOYEE WHERE (Salary BETWEEN 30000 AND 40000) AND Dno = 5;
```

→ Smith(30000), Wong(40000), Narayan(38000)

실습 메모 (3-2, week6 lab): 실습은 SQLite + Python(pandas, sqlglot). NL_queries.txt를 보고 mysql.py에 SQL을 쓰고 test.py로 결과 CSV를 만든다. 제출 압축 파일 이름은 반드시 학번. **SQLite는 ALL, ANY를 지원하지 않는다.**

U5 · 5-1_Advanced-SQL.pdf

Advanced SQL (중첩 · 조인 · 집계 · 뷰)

중첩 질의(IN, ALL/ANY, EXISTS), 조인 테이블(NATURAL, OUTER), 집계와 GROUP BY/HAVING, Assertion·Trigger, 뷰. SQL 작성 문제의 본체라서 문제를 많이 풀어봐. 특히 "모든 ~를 하는" 질의(나눗셈)와 GROUP BY + HAVING은 꼭.

중첩 질의 · IN · ALL/ANY

교안 범위: 5-1 p.3–6

§ 중첩 질의와 IN

중첩 질의 (nested query): 다른 질의의 **WHERE 절 안에** 들어간 완전한 SELECT-FROM-WHERE 블록.

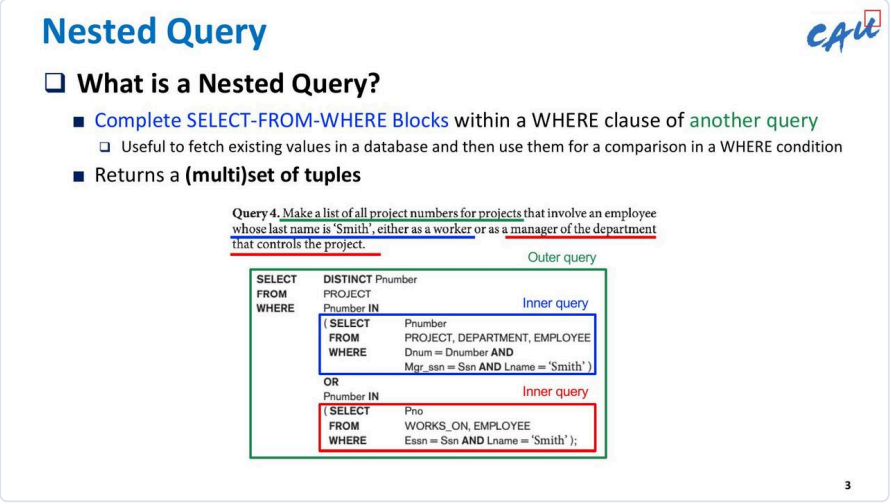
- DB에 있는 값을 가져와서 WHERE 조건의 비교에 쓸 때 유용
- 결과는 투플의 (다중)집합 → 바깥 질의 = outer query, 안쪽 = inner query

IN: 값 v를 값의 (다중)집합 V와 비교. **v가 V의 원소 중 하나면 TRUE**.

```
SELECT DISTINCT Pnumber
FROM   PROJECT
WHERE  Pnumber IN ( SELECT Pnumber
                    FROM   PROJECT, DEPARTMENT, EMPLOYEE
                    WHERE  Dnum = Dnumber AND Mgr_ssn = Ssn AND Lname = 'Smith' )
   OR  Pnumber IN ( SELECT Pno
                    FROM   WORKS_ON, EMPLOYEE
                    WHERE  Essn = Ssn AND Lname = 'Smith' );
```

→ 질의 4(UNION 버전)와 같은 결과 {1, 2}. 안쪽 질의 V1, V2가 각각 프로젝트 번호의 집합.

> **수업 메모** 05주-1
> 안쪽 질의의 결과가 무슨 형태인지 먼저 생각하라고 함: 'Pnumber들의 집합'이 나오고, IN은 지금 투플의 Pnumber가 그 집합에 들어 있는지 확인한다. 두 조건을 OR로 이어서 둘 중 하나만 만족해도 결과에 들어감.



05 p.3 · 중첩 질의: 바깥 질의 안에 안쪽 질의 두 개

§ ALL / ANY (SOME)

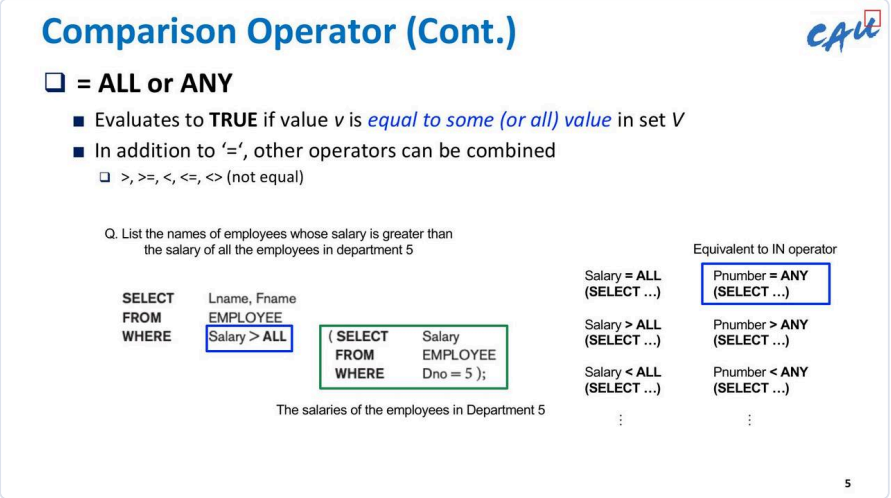
= ANY (= SOME): v가 V의 **어떤** 값과 같으면 TRUE → **IN과 동치**. **= ALL**: **모든** 값과 같으면 TRUE.

- = 말고 >, >=, <, <=, <> 와도 결합 가능

```
SELECT Lname, Fname
FROM   EMPLOYEE
WHERE  Salary > ALL ( SELECT Salary FROM EMPLOYEE WHERE Dno = 5 );
```

→ 5번 부서 **모든** 사원보다 급여가 많은 사원. 5번 부서 급여 = {30000, 40000, 38000, 25000} → 40000 초과 → Wallace(43000), Borg(55000)

함정: `> ANY`는 "어느 하나보다라도 크면" = 최솟값보다 크면. `> ALL`은 최댓값보다 커야 함. 그리고 실습 환경 **SQLite는 ALL/ANY를 지원 안 하니** `> (SELECT MAX(Salary) ...)`로 바꿔 써.



05 p.5 · Salary > ALL (5번 부서 급여), = ANY는 IN과 동치

§ 중첩 질의의 모호성과 상관 질의

여러 단계 중첩에서 바깥 FROM과 안쪽 FROM에 **같은 이름의 속성**이 있으면 모호해진다 → **별칭**으로 해결.

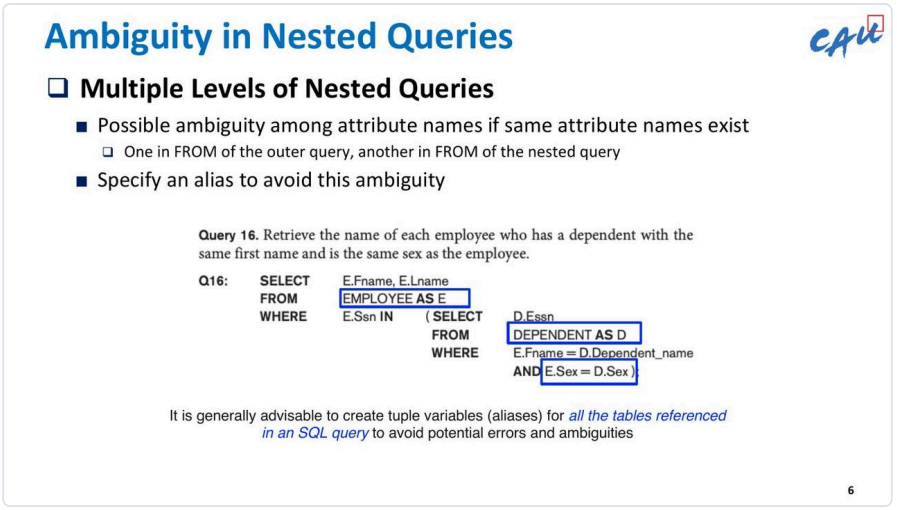
질의 16: 부양가족 중에 **이름(first name)과 성별이 사원 자신과 같은** 사람이 있는 사원

```
SELECT E.Fname, E.Lname
FROM   EMPLOYEE AS E
WHERE  E.Ssn IN ( SELECT D.Essn
                  FROM   DEPENDENT AS D
                  WHERE  E.Fname = D.Dependent_name AND E.Sex = D.Sex );
```

- 안쪽 질의가 바깥의 E를 참조 → **상관 중첩 질의 (correlated nested query)**: 바깥 투플마다 안쪽 질의를 다시 평가하는 것처럼 동작
- 안쪽에서 별칭 없이 Sex라고 쓰면 가장 가까운(안쪽) 테이블의 속성으로 해석된다
- COMPANY 상태에서는 결과 0행 (그런 부양가족이 없음)

시험 포인트: "질의에서 참조하는 **모든 테이블에 별칭을** 붙이는 게 일반적으로 권장된다" (교안 문장 그대로).

수업 메모 `05주-1`
질의가 복잡해질수록 조건이 헷갈리니까, 모든 테이블에 E, D, S 같은 별칭(투플 변수)을 붙이는 관례를 '습관으로 만들라'고 강조.



05 p.6 · 질의 16: 바깥 E를 참조하는 안쪽 질의 (별칭으로 모호성 제거)

EXISTS / NOT EXISTS · 나눗셈 질의

교안 범위: 5-1 p.7–9

§ EXISTS와 NOT EXISTS

중첩 질의 결과가 비었는지 검사하는 불리언 함수. 보통 WHERE에서 씀.

- **EXISTS (Q)**: Q의 결과에 투플이 **하나라도 있으면** TRUE
- **NOT EXISTS (Q)**: Q의 결과가 **비어 있으면** TRUE

질의 6: 부양가족이 **없는** 사원

```
SELECT Fname, Lname
FROM   EMPLOYEE
WHERE  NOT EXISTS ( SELECT * FROM DEPENDENT WHERE Ssn = Essn );
```

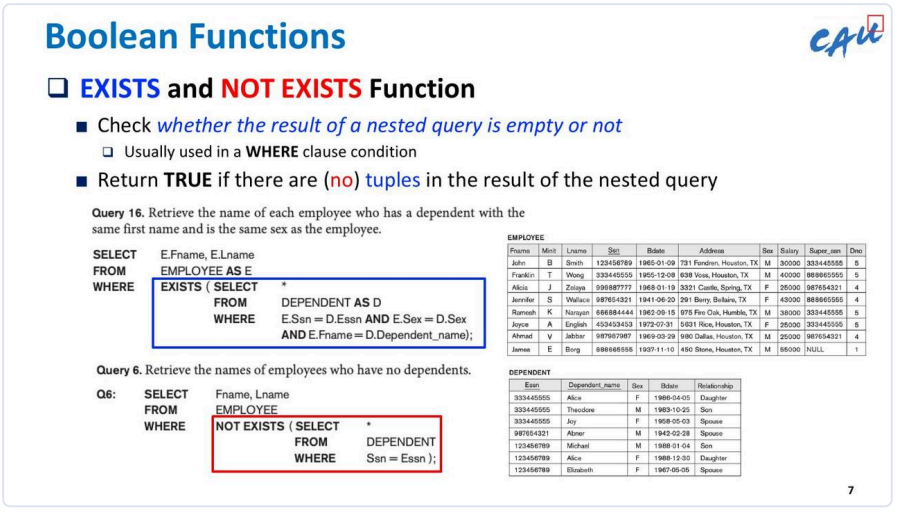
→ Zelaya, Narayan, English, Jabbar, Borg (5명)

부양가족이 하나 이상 있는 관리자: 두 조건을 AND

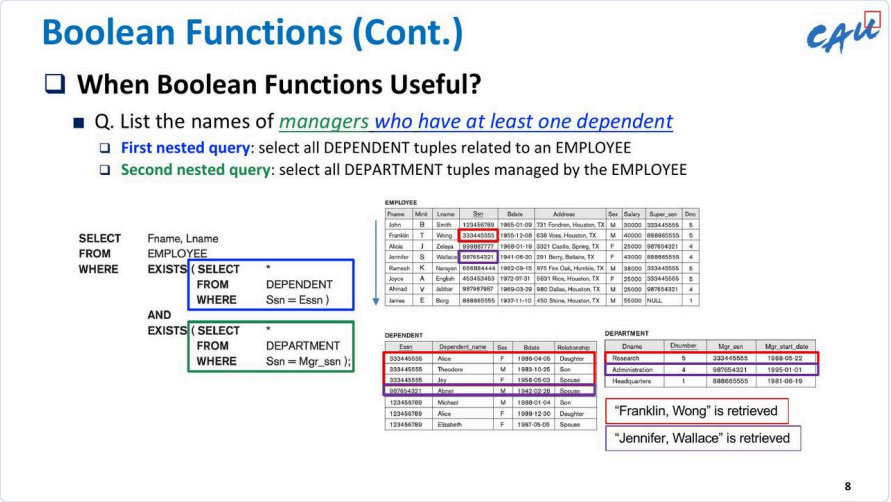
```
SELECT Fname, Lname
FROM   EMPLOYEE
WHERE  EXISTS ( SELECT * FROM DEPENDENT  WHERE Ssn = Essn )
  AND  EXISTS ( SELECT * FROM DEPARTMENT WHERE Ssn = Mgr_ssn );
```

→ Franklin Wong, Jennifer Wallace

> **수업 메모** 05주-1
> NOT EXISTS 읽는 법: 안쪽 질의 결과가 비어 있지 않으면 그 사원은 부양가족이 있다는 뜻. 결과가 비었을 때만 NOT EXISTS가 참이 되어 그 사원이 나온다. '부양가족 있는 관리자'는 조건을 '관리자인 사원'과 '부양가족이 하나 이상인 사원' 두 부분으로 쪼개서 EXISTS 두 개를 AND로.



05 p.7 · EXISTS (질의 16)와 NOT EXISTS (질의 6, 부양가족 없는 사원)



05 p.8 · EXISTS 두 개: 부양가족 있는 관리자 → Wong, Wallace

§ "모든 ~에 대해" (나눗셈) 질의

연습 1: 5번 부서가 통제하는 **모든** 프로젝트에서 일하는 사원의 이름

```
SELECT Fname, Lname
FROM   EMPLOYEE
WHERE  NOT EXISTS ( ( SELECT Pnumber FROM PROJECT WHERE Dnum = 5 )
                    EXCEPT
                    ( SELECT Pno FROM WORKS_ON WHERE Ssn = Essn ) );
```

읽는 법: (5번 부서 프로젝트 전체) − (이 사원이 일하는 프로젝트) = **빠진 프로젝트**. 빠진 게 **없으면**(NOT EXISTS) 전부 하는 것.

- 예: 5번 부서 프로젝트 = {1, 2, 3}. A는 {1, 2, 4} → 차집합 {3} ≠ ∅ → 탈락. B는 {1, 2, 3, 4} → 차집합 ∅ → 선택
- COMPANY 상태에서 5번 부서 프로젝트 = {1, 2, 3}인데 세 개 다 하는 사원이 없어서 결과는 **0행**

EXCEPT 없이 NOT EXISTS 두 번으로도 쓸 수 있다 ("이 사원이 안 하는 5번 부서 프로젝트가 존재하지 않는다"):

```
SELECT E.Fname, E.Lname
FROM   EMPLOYEE AS E
WHERE  NOT EXISTS ( SELECT * FROM PROJECT AS P
                    WHERE  P.Dnum = 5
                      AND  NOT EXISTS ( SELECT * FROM WORKS_ON AS W
                                        WHERE W.Essn = E.Ssn AND W.Pno = P.Pnumber ) );
```

함정: "어떤" 프로젝트에서 일하는 사원(EXISTS, IN)과 "모든" 프로젝트에서 일하는 사원(NOT EXISTS + EXCEPT)을 구별해. 문장에 **all**이 나오면 나눗셈 패턴.

수업 메모 `05주-1`

문제를 천천히 읽으라고 강조: 5번 부서가 프로젝트를 여러 개(예: 5~7개) 통제하는데, 그중 '일부'가 아니라 '전부'에서 일하는 사원만 찾아야 한다. 칠판 예: 5번 부서 프로젝트 = {1, 2, 3}, 사원 A = {1, 2, 4}, 사원 B = {1, 2, 3, 4} → {1,2,3} − A = {3} (비어 있지 않음, 탈락), {1,2,3} − B = ∅ (선택). 'NOT EXISTS + EXCEPT'가 한 가지 풀이.

조인 테이블 · NATURAL · INNER/OUTER

교안 범위: 5-1 p.10–14

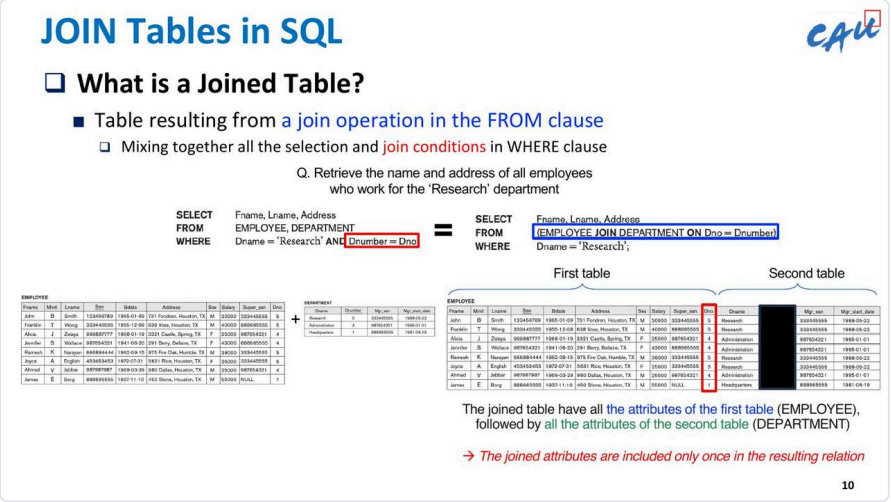
§ 조인 테이블

조인 테이블 (joined table): **FROM 절에서** 조인 연산으로 만들어지는 테이블. WHERE에 선택 조건과 조인 조건이 섞이는 걸 피할 수 있다.

```
SELECT Fname, Lname, Address
FROM   (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber)
WHERE  Dname = 'Research';
```

- 조인된 테이블 = 첫 테이블(EMPLOYEE)의 모든 속성 + 둘째 테이블(DEPARTMENT)의 모든 속성
- WHERE에 `Dnumber = Dno`를 쓴 질의 2와 결과가 같다

> **수업 메모** 05주-1
> Dname = 'Research'는 선택 조건, Dnumber = Dno는 조인 조건. 조인 조건을 FROM 절의 JOIN ... ON으로 명시적으로 쓸 수 있고, 결과는 WHERE에 쓴 것과 같다.



05 p.10 · FROM 절의 EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber

§ NATURAL JOIN

같은 이름을 가진 속성 쌍마다 암묵적인 동등 조건을 만든다 → 조인 조건을 쓰지 않음.

- 이름이 다르면 먼저 **이름을 바꿔서** 맞춘다

```
SELECT Fname, Lname, Address
FROM   (EMPLOYEE NATURAL JOIN
        (DEPARTMENT AS DEPT (Dname, Dno, Mssn, Msdate)))
WHERE  Dname = 'Research';
```

- DEPARTMENT의 Dnumber를 Dno로 바꿔서 EMPLOYEE.Dno와 이름을 맞춤
- 조인 속성은 결과에 **한 번만** 나온다

수업 메모 05주-1

NATURAL JOIN은 이름이 같은 속성 쌍을 조인 속성으로 가정한다. DEPARTMENT를 DEPT(Dname, Dno, Mssn, Msdate)로 별칭을 주면서 Dnumber를 Dno로 바꿔야 EMPLOYEE.Dno와 이름이 맞아서 자연 조인이 된다.

§ INNER JOIN과 OUTER JOIN

내부 조인 (inner join): 기본 조인. **다른 릴레이션에 짝이 있는 투플만** 결과에 들어감.

```
SELECT E.Lname AS Employee_name, S.Lname AS Supervisor_name
FROM   EMPLOYEE AS E, EMPLOYEE AS S
WHERE  E.Super_ssn = S.Ssn;
```

→ 7행. Super_ssn이 **NULL인 Borg는 빠진다**.

외부 조인 (outer join): 짝이 없는 투플도 결과에 넣고, 상대 속성은 **NULL로 채움(padding)**.

```
SELECT E.Lname AS Employee_name, S.Lname AS Supervisor_name
FROM   (EMPLOYEE AS E LEFT OUTER JOIN EMPLOYEE AS S ON E.Super_ssn = S.Ssn);
```

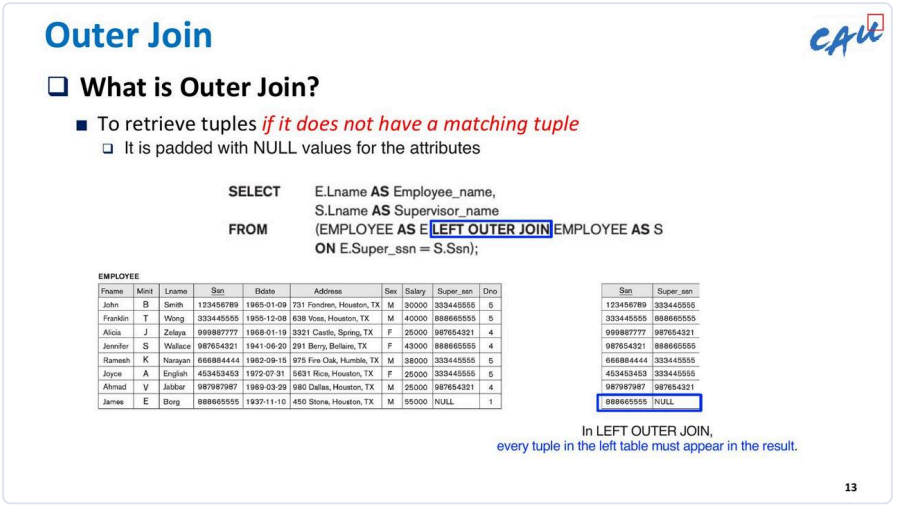
→ 8행. Borg의 Supervisor_name = NULL.

조인	결과에 들어가는 투플
INNER JOIN	조인 조건을 만족하는 쌍만
LEFT OUTER JOIN	**왼쪽 테이블의 모든 투플** (짝 없으면 오른쪽 속성 NULL)
RIGHT OUTER JOIN	**오른쪽 테이블의 모든 투플**
FULL OUTER JOIN	양쪽 모든 투플

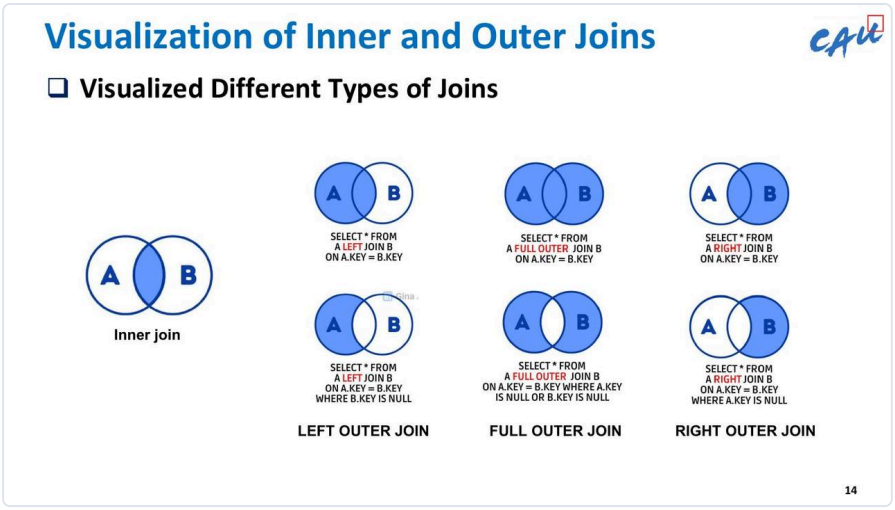
함정: OUTER JOIN에서 "짝 없는 것만" 고르려면 `WHERE B.KEY IS NULL`을 덧붙인다 (교안 p.14 벤다이어그램 아래줄). NULL 비교는 `= NULL`이 아니라 **IS NULL**.

수업 메모 05주-1

Super_ssn = Ssn 자기 조인에서 James Borg는 Super_ssn이 없어서(NULL) 내부 조인 결과에 안 나온다. 상사가 없는 사원까지 보고 싶으면 LEFT OUTER JOIN. 어떤 조인을 쓸지는 응용 상황에 따라 고른다.



05 p.13 · LEFT OUTER JOIN: Borg도 결과에 (상사 NULL)



05 p.14 · 내부·외부 조인 벤다이어그램

집계 함수 · GROUP BY · HAVING

교안 범위: 5-1 p.15–22, 48

§ 집계 함수

집계 함수 (aggregate function): 여러 투플의 정보를 **투플 하나짜리 요약**으로. 주로 SELECT나 HAVING에서 씀.

- **COUNT**: 투플 수 또는 값의 수
- **SUM, MAX, MIN, AVG**: 숫자 값의 (다중)집합에 적용

질의 19: 전체 사원의 급여 합·최대·최소·평균

```
SELECT SUM(Salary), MAX(Salary), MIN(Salary), AVG(Salary)
FROM   EMPLOYEE;
```

→ 281000, 55000, 25000, 35125

질의 20: Research 부서만

```
SELECT SUM(Salary), MAX(Salary), MIN(Salary), AVG(Salary)
FROM   (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber)
WHERE  Dname = 'Research';
```

→ 133000, 40000, 25000, 33250

> **수업 메모** 05주-1
> 집계 함수가 없다면 급여를 전부 읽어 와서 Python이나 C로 for/while 루프를 돌려 합·최대·평균을 직접 계산해야 한다. 집계 함수로 한 줄에 해결. 결과: 최대 55,000, 최소 25,000, 평균 = 합 ÷ 8.

§ COUNT와 NULL · 중첩 질의에서의 집계

- **COUNT(*)**: 결과의 **행(투플) 수**. 질의 21 `SELECT COUNT(*) FROM EMPLOYEE;` → 8, 질의 22 Research만 → 4
- **COUNT(속성)**: 그 속성의 **값** 수. **NULL은 세지 않는다**. 예) `COUNT(Super_ssn)` → 7
- **COUNT(DISTINCT Salary)**: 서로 다른 급여 값의 수 → 6 (질의 23)
- 집계 결과는 값 하나라서 **조건의 일부**로 쓸 수 있다:

```
SELECT Lname, Fname
FROM   EMPLOYEE
WHERE  ( SELECT COUNT(*) FROM DEPENDENT WHERE Ssn = Essn ) ⩾ 2;
```

→ 부양가족이 2명 이상: Smith(3명), Wong(3명)

> **수업 메모** 05주-1
> COUNT(*)를 안 쓰면 투플 8개가 그대로 나오지만, COUNT(*)를 쓰면 숫자 하나(8)만 나온다. Research는 4. 서로 다른 급여 값은 25,000이 겹쳐서 6. 집계 결과는 값 하나라서 '부양가족 2명 이상'처럼 조건의 일부로 쓸 수 있다: 안쪽 질의가 사원마다 부양가족 수 하나를 돌려주고 그걸 2와 비교.

§ GROUP BY

그룹화: 릴레이션의 투플을 **겹치지 않는 부분집합(그룹)** 으로 나눠서 그룹마다 집계 함수를 적용.

- **그룹화 속성 (grouping attribute)**: 그룹을 나누는 기준. 같은 그룹 = 그룹화 속성 값이 같은 투플들
- **GROUP BY** 절에 지정. 그룹화 속성은 SELECT에도 나와야 결과에서 어느 그룹인지 알 수 있다

질의 24: 부서별 부서 번호, 사원 수, 평균 급여

```
SELECT   Dno, COUNT(*), AVG(Salary)
FROM     EMPLOYEE
GROUP BY Dno;
```

→ (5, 4, 33250), (4, 3, 31000), (1, 1, 55000)

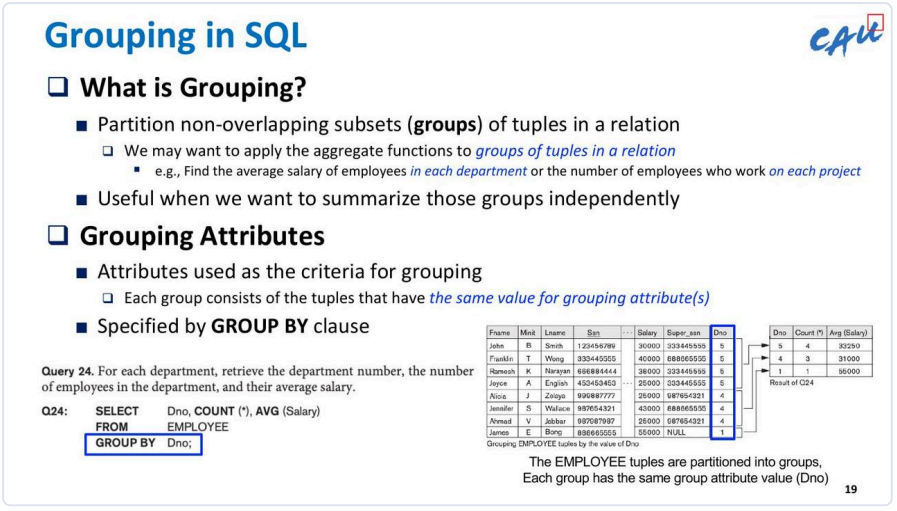
질의 25: 프로젝트별 번호, 이름, 일하는 사원 수

```
SELECT   Pnumber, Pname, COUNT(*)
FROM     PROJECT, WORKS_ON
WHERE    Pnumber = Pno
GROUP BY Pnumber, Pname;
```

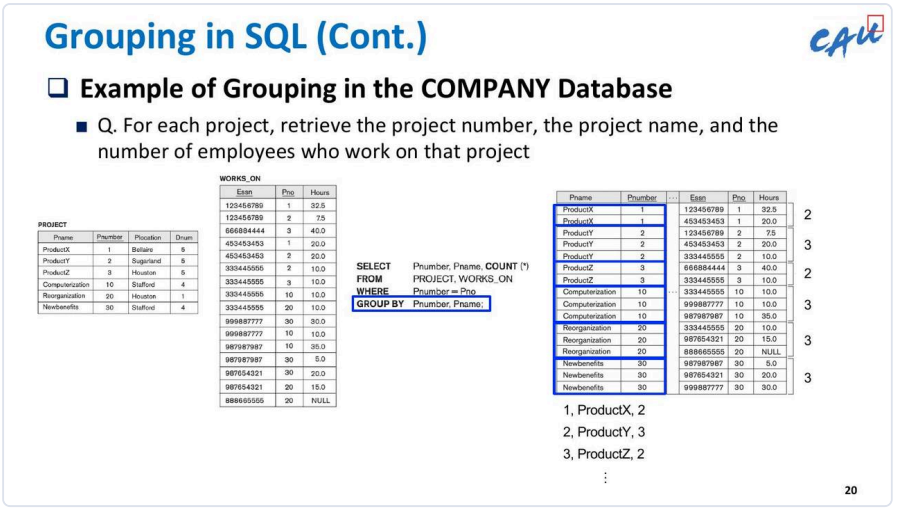
→ ProductX 2, ProductY 3, ProductZ 2, Computerization 3, Reorganization 3, Newbenefits 3

함정: GROUP BY를 쓰면 SELECT에는 **그룹화 속성과 집계 함수만** 올 수 있다. 그룹마다 값이 하나로 정해지지 않는 속성(예: Dno로 묶었는데 Lname)을 SELECT에 쓰면 안 된다.

수업 메모 `05주-1`
그룹화는 그룹마다 따로 요약할 때 쓴다. Dno로 묶으면 5번 부서, 4번 부서, 1번 부서 세 그룹이 생기고 그룹마다 COUNT, AVG를 계산.



05 p.19 · 질의 24: Dno로 그룹화



05 p.20 · 질의 25: 프로젝트별 사원 수

§ HAVING

HAVING: 그룹의 **요약 정보에 대한 조건**. GROUP BY와 함께 씀. 조건을 만족하는 **그룹만** 결과에.

질의 26: 사원이 **두 명 넘게** 일하는 프로젝트마다 번호, 이름, 사원 수

```
SELECT   Pnumber, Pname, COUNT(*)
FROM     PROJECT, WORKS_ON
WHERE    Pnumber = Pno
GROUP BY Pnumber, Pname
HAVING   COUNT(*) > 2;
```

→ ProductY, Computerization, Reorganization, Newbenefits (각 3명)

연습 2: 평균 급여가 $30,000 넘는 부서마다 부서 이름과 사원 수

```
SELECT   D.Dname, COUNT(*)
FROM     EMPLOYEE AS E, DEPARTMENT AS D
WHERE    E.Dno = D.Dnumber
GROUP BY D.Dname
HAVING   AVG(E.Salary) > 30000;
```

→ Research 4, Administration 3, Headquarters 1 (교안 풀이는 GROUP BY E.Dno로 묶음. 표준 SQL에서는 SELECT의 D.Dname도 GROUP BY에 있어야 안전)

> 시험 포인트: **WHERE = 투플 조건 (그룹 만들기 전)**, **HAVING = 그룹 조건 (집계 후)**. "평균이 3만 넘는"처럼 집계에 대한 조건은 WHERE에 못 쓴다.

실행 순서: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

> **수업 메모** `05주-1`
> GROUP BY는 그룹별 요약, HAVING은 조건을 만족하는 그룹만 요약. 연습 2 풀이 때 'EMPLOYEE만으로 안 되고 왜 DEPARTMENT가 필요하냐'고 물음 → EMPLOYEE엔 부서 번호만 있고 부서 '이름'이 없어서 조인해야 한다. 평균 조건(> 30000)은 HAVING에.

Grouping in SQL (Cont.)

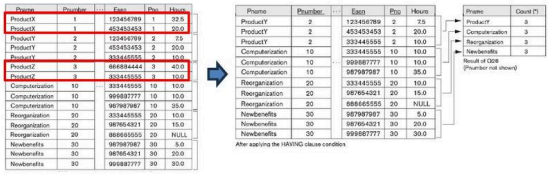
❑ HAVING Clause

- Specify a "***condition***" on *the summary information regarding the group of tuples*
- Can appear in conjunction with a GROUP BY clause

Query 26. For each project *on which more than two employees work*, retrieve the project number, the project name, and the number of employees who work on the project.

```
Q26:  SELECT     Pnumber, Pname, COUNT (*)
      FROM       PROJECT, WORKS_ON
      WHERE      Pnumber = Pno
      GROUP BY   Pnumber, Pname
      HAVING     COUNT (*) > 2;
```

Only groups satisfying this condition are retrieved



05 p.21 · 질의 26: HAVING COUNT(*) > 2

Assertion · Trigger

교안 범위: 5-1 p.24–29

§ Assertion

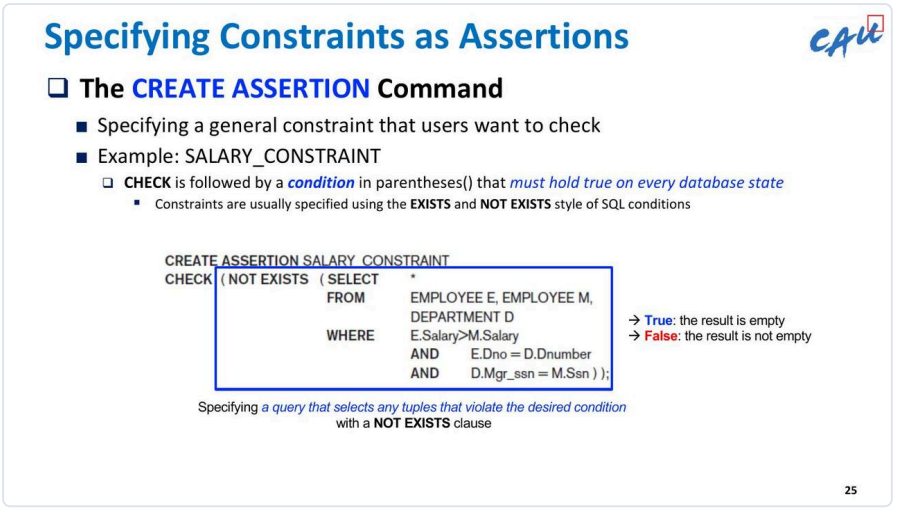
내장 제약(키, 개체 무결성, 참조 무결성)으로 표현할 수 없는 일반 제약을 지정.

- 예) 사원의 급여는 그 사원이 속한 부서 관리자의 급여보다 많으면 안 된다

```
CREATE ASSERTION SALARY_CONSTRAINT
CHECK ( NOT EXISTS ( SELECT *
                     FROM   EMPLOYEE E, EMPLOYEE M, DEPARTMENT D
                     WHERE  E.Salary > M.Salary
                       AND  E.Dno = D.Dnumber
                       AND  D.Mgr_ssn = M.Ssn ) );
```

- **CHECK** 뒤 괄호 안의 조건은 **모든 DB 상태에서 참**이어야 한다
- 관례: 원하는 조건을 **위반하는 투플을 고르는 질의**를 쓰고 **NOT EXISTS**로 감싼다 → 결과가 비면 TRUE(만족), 비지 않으면 FALSE(위반)

> **수업 메모** 05주-1
> CREATE ASSERTION 이름 CHECK (조건). 관례는 '확인하고 싶은 조건을 위반하는 투플을 찾는 질의'를 쓰고 NOT EXISTS로 감싸는 것. 사원 급여가 관리자 급여보다 크면 위반 → 그 질의 결과가 비어 있지 않으면 문제가 있다는 뜻.



05 p.25 · CREATE ASSERTION SALARY_CONSTRAINT (위반 질의를 NOT EXISTS로)

§ Trigger (ECA 규칙)

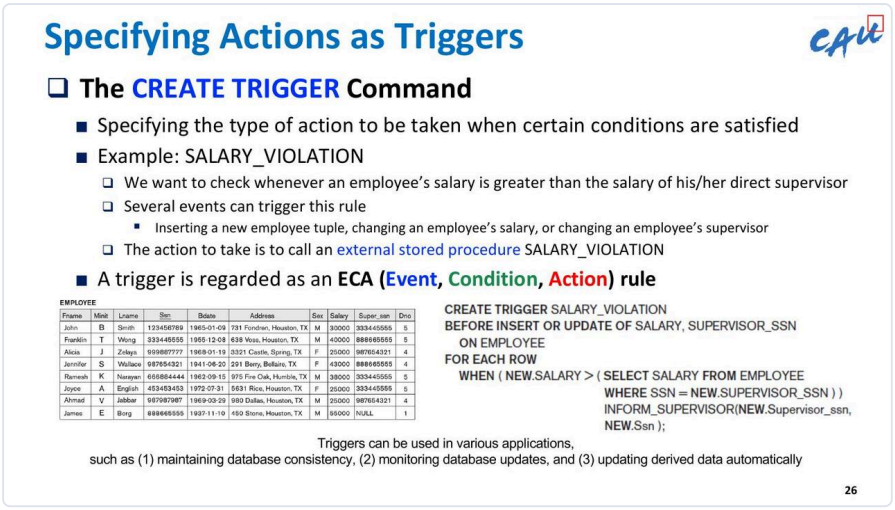
특정 이벤트가 일어나고 조건이 맞을 때 자동으로 실행할 동작을 지정. **ECA (Event, Condition, Action) 규칙**.

```
CREATE TRIGGER SALARY_VIOLATION
BEFORE INSERT OR UPDATE OF SALARY, SUPERVISOR_SSN
    ON EMPLOYEE
FOR EACH ROW
    WHEN ( NEW.SALARY > ( SELECT SALARY FROM EMPLOYEE
                          WHERE SSN = NEW.SUPERVISOR_SSN ) )
        INFORM_SUPERVISOR(NEW.Supervisor_ssn, NEW.Ssn);
```

- **Event**: DB에 명시적으로 적용되는 갱신 연산 (새 사원 삽입, 급여 변경, 상사 변경). **BEFORE / AFTER**로 그 연산 전/후에 실행. 가능한 이벤트를 **모두** 고려해야 함
- **Condition**: 이벤트가 일어난 뒤 동작을 실행할지 결정. **WHEN 절**. 선택 사항. 참일 때만 동작 실행
- **Action**: 자동 실행할 동작 = SQL 문장들, DB 트랜잭션, 또는 외부 프로그램 (예: 외부 저장 프로시저 INFORM_SUPERVISOR)
- 용도: (1) DB 일관성 유지, (2) DB 갱신 모니터링, (3) 유도 데이터 자동 갱신
- 예) 사원의 출장비가 한도를 넘으면 관리자에게 알림

함정: Assertion = **제약** (위반하면 연산을 막음), Trigger = **동작** (조건이 맞으면 무언가를 실행). 교안 예는 둘 다 급여 비교지만 하나는 "막기", 하나는 "알리기".

수업 메모 `05주-1`
트리거 = Event, Condition, Action 세 요소. Event는 조건을 깰 가능성이 있는 갱신 연산(새 사원 삽입, 급여 변경, 상사 변경)이고 BEFORE/AFTER로 시점 지정. Condition은 WHEN 절, 참이 아니면 동작은 건너뜀. Action은 SQL 문장, 트랜잭션, 외부 프로그램 무엇이든 가능 (예: 메신저로 상사에게 알림).



05 p.26 · CREATE TRIGGER SALARY_VIOLATION (ECA)

뷰 (View)

교안 범위: 5-1 p.30–47

§ 뷰의 정의와 쓰임

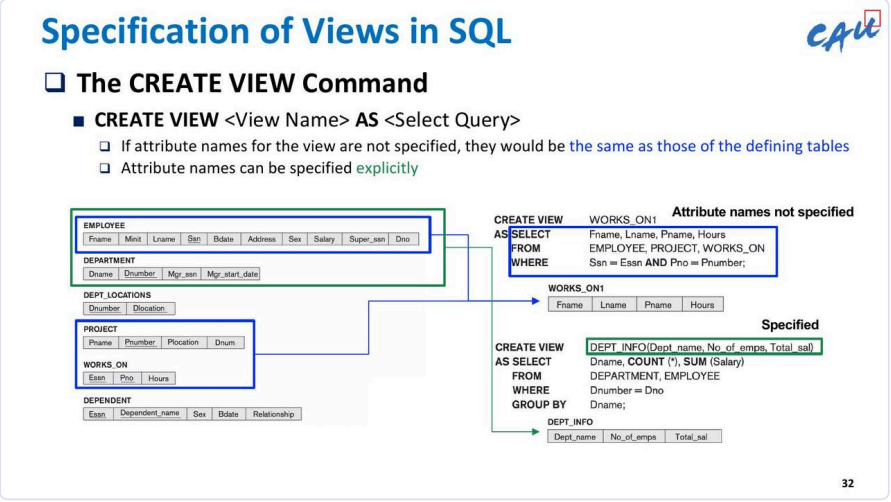
뷰 (view): 기본 테이블(base table)에서 유도된 **가상 테이블**. 물리적으로 존재하지 않을 수도 있다.

- 검색: 제한 없음. 갱신: 보통 제한됨 (사실상 읽기 전용)
- 쓰는 이유: **복잡한 질의를 단순화**. 여러 테이블 조인·집계를 자주 쓰면 뷰로 만들어 두고 **단일 테이블 질의**처럼 쓴다

```
CREATE VIEW WORKS_ON1
AS SELECT Fname, Lname, Pname, Hours
   FROM   EMPLOYEE, PROJECT, WORKS_ON
   WHERE  Ssn = Essn AND Pno = Pnumber;

CREATE VIEW DEPT_INFO (Dept_name, No_of_emps, Total_sal)
AS SELECT   Dname, COUNT(*), SUM(Salary)
   FROM     DEPARTMENT, EMPLOYEE
   WHERE    Dnumber = Dno
   GROUP BY Dname;
```

- 속성 이름을 안 주면 정의 질의의 속성 이름을 그대로 씀 (WORKS_ON1), 명시할 수도 있음 (DEPT_INFO)
- 뷰 질의: `SELECT Fname, Lname FROM WORKS_ON1 WHERE Pname = 'ProductX';` → 매번 세 테이블을 조인할 필요 없음
- **DROP VIEW WORKS_ON1;** → 정의 테이블에는 영향 없음



05 p.32 · CREATE VIEW WORKS_ON1, DEPT_INFO

§ 뷰 구현: 질의 수정 vs 실체화

뷰는 **항상 최신(up-to-date)** 이어야 한다 → 정의 테이블이 바뀌면 DBMS가 뷰에 반영할 책임.

방식	내용	특징
질의 수정 (query modification)	뷰 질의를 **기본 테이블에 대한 질의로 변환**. 뷰를 물리적으로 유지하지 않음	사용자는 편하지만 DB 시스템엔 비효율 (복잡한 뷰를 매번 다시 계산)
뷰 실체화 (view materialization)	뷰가 처음 질의될 때 **물리적으로 뷰 테이블을 만들어** 한동안 유지	이후 뷰 질의가 자주 온다는 가정. **추가 저장 공간**. 정의 테이블이 바뀌면 **점진적 갱신(incremental update)**

실체화된 뷰를 **언제** 갱신하나:

- **즉시 갱신 (immediate)**: 기본 테이블이 바뀌자마자. 항상 최신이지만 불필요하고 비효율적일 때가 있음
- **주기적 갱신 (periodic)**: 주기적으로. 뷰 질의가 최신이 아닌 결과를 받을 수 있음
- **지연 갱신 (lazy)**: 뷰 질의가 필요할 때. 질의가 오기 전까진 최신이 아님

일정 기간 질의되지 않으면 실체화된 뷰를 지우고, 나중에 필요하면 처음부터 다시 계산.

---

§ 뷰 갱신

뷰에 대한 검색은 언제나 가능하지만, INSERT/DELETE/UPDATE는 많은 경우 불가능.

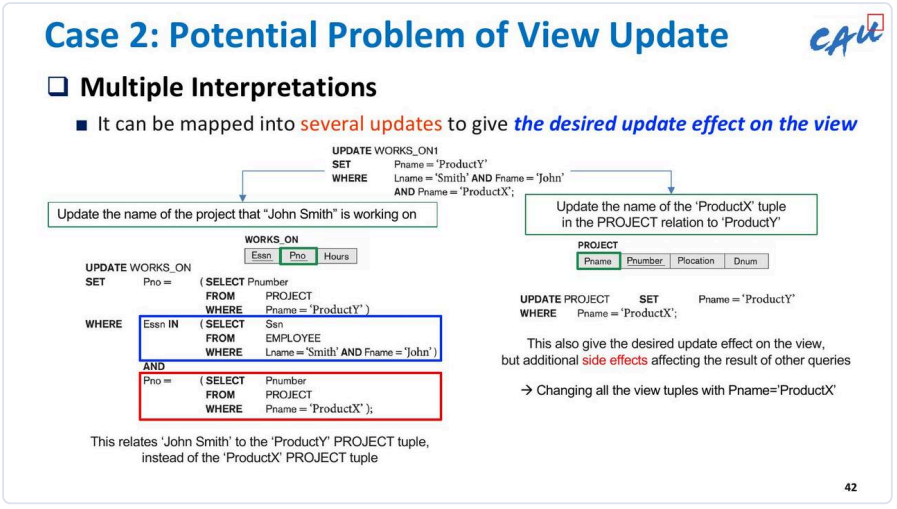
갱신 가능 조건: 뷰 갱신을 이루는 기본 릴레이션 갱신이 딱 하나뿐일 때.

- **단일 테이블 + 집계 없음** → 기본 테이블 갱신 하나로 매핑 → **허용**. 단 뷰 속성에 (1) 기본 릴레이션의 **PK**, (2) DEFAULT가 없는 **NOT NULL 속성 전부**가 들어 있어야 함 (뷰로 투플을 넣을 때 그 값들이 비면 안 되니까)
- **조인 (여러 테이블)** → 여러 방식으로 매핑될 수 있어 **일반적으로 불가**
- **그룹화·집계** → 그 갱신이 **말이 안 됨** → **불가**

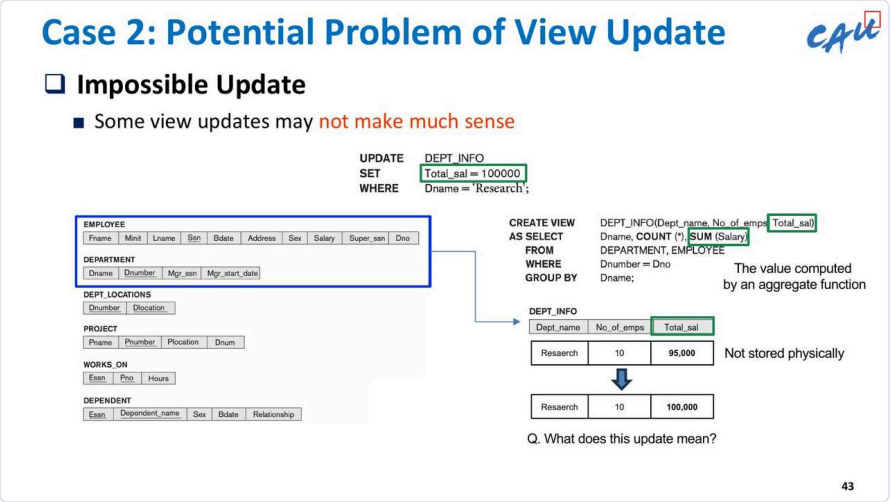
예 1 (여러 해석): `UPDATE WORKS_ON1 SET Pname = 'ProductY' WHERE Lname = 'Smith' AND Fname = 'John' AND Pname = 'ProductX';`

- 해석 a: Smith가 일하는 프로젝트를 바꿈 → WORKS_ON의 Pno를 ProductY 번호로 (의도대로)
- 해석 b: PROJECT의 'ProductX' 이름을 'ProductY'로 → 뷰 결과는 같아 보이지만 **부작용**: ProductX에서 일하던 모든 투플이 바뀜

예 2 (불가능): `UPDATE DEPT_INFO SET Total_sal = 100000 WHERE Dname = 'Research';` → Total_sal은 SUM으로 **계산된 값**, 물리적으로 저장 안 됨 → 의미 없음



05 p.42 · WORKS_ON1 갱신의 두 가지 해석



05 p.43 · DEPT_INFO의 Total_sal 갱신은 의미 없음

§ 뷰를 이용한 권한 부여

뷰로 권한 없는 사용자에게 **특정 속성이나 투플을 숨길** 수 있다.

```
CREATE VIEW DEPT5EMP AS
SELECT * FROM EMPLOYEE WHERE Dno = 5;            -- 투플 제한

CREATE VIEW BASIC_EMP_DATA AS
SELECT Fname, Lname, Address FROM EMPLOYEE;      -- 속성 제한
```

- DEPT5EMP: 5번 부서 관리자는 5번 부서 사원 정보만 보게 함. 사용자에게 EMPLOYEE가 아니라 **뷰에 대한 질의 권한만** 부여
- BASIC_EMP_DATA: 일반 사원은 이름과 주소만

교안 연습 3·4 풀이:

```
CREATE VIEW DEPT_INFO (DeptName, MgrFname, MgrLname, MgrSalary)
AS SELECT D.Dname, E.Fname, E.Lname, E.Salary
   FROM   DEPARTMENT AS D, EMPLOYEE AS E
   WHERE  D.Mgr_ssn = E.Ssn;                      -- 부서와 관리자 정보

CREATE VIEW ProjectInfo (Pname, Dname, NumEmployees, TotalHours)
AS SELECT   P.Pname, D.Dname, COUNT(*), SUM(W.Hours)
   FROM     PROJECT AS P, WORKS_ON AS W, DEPARTMENT AS D
   WHERE    P.Pnumber = W.Pno AND D.Dnumber = P.Dnum
   GROUP BY P.Pnumber, P.Pname, D.Dname;          -- 프로젝트 현황
```

Views as Authorization

❑ Authorization Mechanisms using Views

- Views can be used to hide **certain attributes** or **tuples** from unauthorized users.
- Example 1: DEPT5EMP
  - ❑ The manager of department 5 is only allowed to see information for employees working for department 5
    - Create the view 'DEPT5EMP' and grant the user the privilege to query the view but not EMPLOYEE
- Example 2: BASIC_EMP_DATA
  - ❑ Similarly, a view can restrict a user to only see certain columns
  - ❑ Normal employees can see only the basic data: first name, last name, and address of an employee

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	98765432!	4
Jennifer	S	Wallace	98765432!	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	98765432!	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

```
CREATE VIEW   DEPT5EMP   AS
SELECT        *
FROM          EMPLOYEE
WHERE         Dno = 5;

CREATE VIEW   BASIC_EMP_DATA   AS
SELECT        Fname, Lname, Address
FROM          EMPLOYEE;
```

45

05 p.45 · 권한 부여용 뷰 DEPT5EMP (투플 제한), BASIC_EMP_DATA (속성 제한)

U6 · 6-1_relational-algebra.pdf + db-exercises Week 7

관계 대수 (Relational Algebra)

σ π ρ, 집합 연산, ×, ⋈(세타·동등·자연), ÷, 𝔊(집계), 외부 조인. 질의를 "중간 릴레이션 이름 ← 연산" 순서로 쪼개 쓰는 연습이 핵심. 답안에 기호 대신 SELECT, PROJECT, JOIN 같은 단어를 써도 채점돼.

L6-1 · 관계 대수 (RELATIONAL ALGEBRA)

관계 대수 소개 · 질의 트리

교안 범위: 6-1 p.3–6

§ 관계 대수란

- **관계 대수 (relational algebra)**: "데이터를 **어떻게** 가져오나"에 대한 이론적·수학적 모델 → **절차적 (procedural)**
- **SQL**은 고급 **선언적 (declarative)** 언어 → "**무슨** 데이터가 필요한가"만 말하고 어떻게 가져올지는 신경 안 씀
- 형식적 관계 모델의 **기본 연산 집합**. 집합에 대한 수학 개념에 엄격하게 기반

왜 중요한가:

- 관계 모델 연산(선택, 투영, 이름 바꾸기, 조인, 카티션 곱...)의 **형식적 기초**
- **질의 처리와 최적화의 기반**: 결과를 얻기 위한 연산의 **순서(sequence)** 를 명시

> 시험 포인트: "SQL = what (선언적), 관계 대수 = how (절차적)". 그리고 관계 대수는 **닫힌 체계**: 모든 연산의 결과가 다시 릴레이션.

§ 질의 (실행) 트리

질의 트리 (query tree): DBMS 내부에서 질의를 표현하는 트리 = 관계 대수 식.

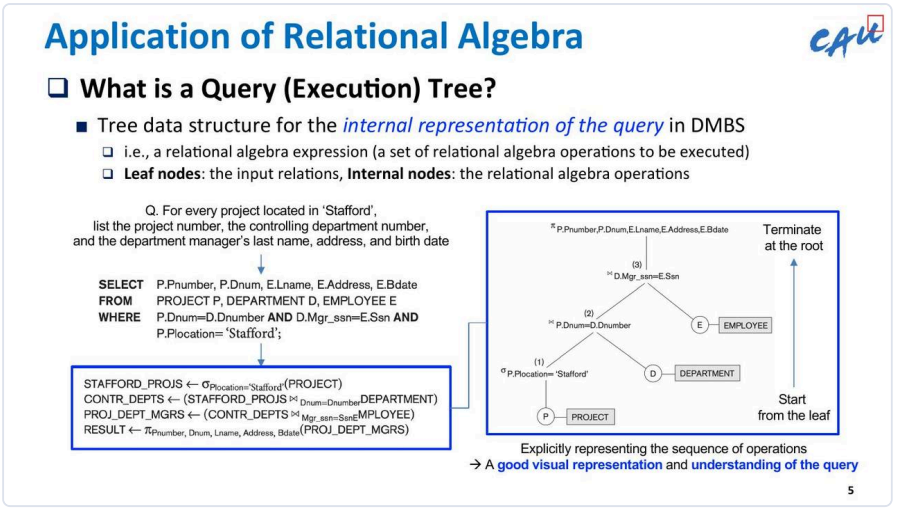
- **잎 노드 = 입력 릴레이션**, **내부 노드 = 관계 대수 연산**
- **잎에서 시작해 루트에서 끝난다** (아래에서 위로 실행)
- 연산 순서를 명시적으로 보여줘서 질의를 이해하기 좋음

교안 예: 'Stafford'에 있는 프로젝트마다 프로젝트 번호, 통제 부서 번호, 부서 관리자의 성·주소·생일

```
SELECT P.Pnumber, P.Dnum, E.Lname, E.Address, E.Bdate
FROM   PROJECT P, DEPARTMENT D, EMPLOYEE E
WHERE  P.Dnum = D.Dnumber AND D.Mgr_ssn = E.Ssn AND P.Plocation = 'Stafford';
```

트리: (1) σ Plocation='Stafford' (P) → (2) ⋈ P.Dnum=D.Dnumber D → (3) ⋈ D.Mgr_ssn=E.Ssn E → 루트 π Pnumber, Dnum, Lname, Address, Bdate

질의 처리: 가능한 실행 전략(질의 트리) 중 적절한 걸 고르는 과정. **SQL 하나에 질의 트리(계획)가 여러 개** 있을 수 있다 → 어느 게 최선인지는 질의 최적화 (13주차).



06 p.5 · 'Stafford' 질의의 질의 트리 (잎 = 릴레이션, 내부 노드 = 연산)

L6-2 · 관계 대수 (RELATIONAL ALGEBRA)

단항 연산: SELECT · PROJECT · RENAME

교안 범위: 6-1 p.8–15

§ SELECT (σ)

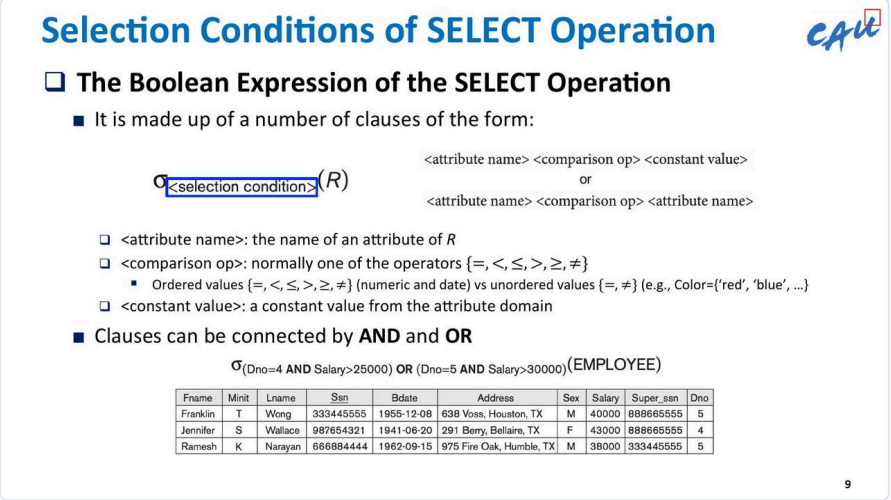
σ<선택 조건>(R): 릴레이션에서 **선택 조건을 만족하는 투플의 부분집합**을 고름 → **수평 필터링**.

- 조건은 투플 **각각에** 개별 적용
- 결과는 R과 **같은 속성** → **결과 차수 = R의 차수**
- 조건 = `<속성> <비교연산> <상수>` 또는 `<속성> <비교연산> <속성>` 절들을 **AND, OR**로 연결
  - 비교 연산 {=, <, ≤, >, ≥, ≠}. 순서 있는 값(숫자, 날짜)은 전부, 순서 없는 값(색깔 등)은 {=, ≠}만
- 예) σ Dno=4 (EMPLOYEE) → Zelaya, Wallace, Jabbar
- 예) σ (Dno=4 AND Salary>25000) OR (Dno=5 AND Salary>30000) (EMPLOYEE) → Wong, Wallace, Narayan

성질

- **교환 법칙**: σ c1(σ c2(R)) = σ c2(σ c1(R))
- 여러 σ는 AND로 **하나로 합칠 수 있음**: σ c1(σ c2(...σ cn(R))) = σ c1 AND c2 AND ... AND cn(R)

함정: 관계 대수의 SELECT(σ)는 SQL의 **SELECT 절이 아니라 WHERE 절**에 대응. `σ Dno=4 AND Salary>25000 (EMPLOYEE)` = `SELECT * FROM EMPLOYEE WHERE Dno=4 AND Salary>25000;`

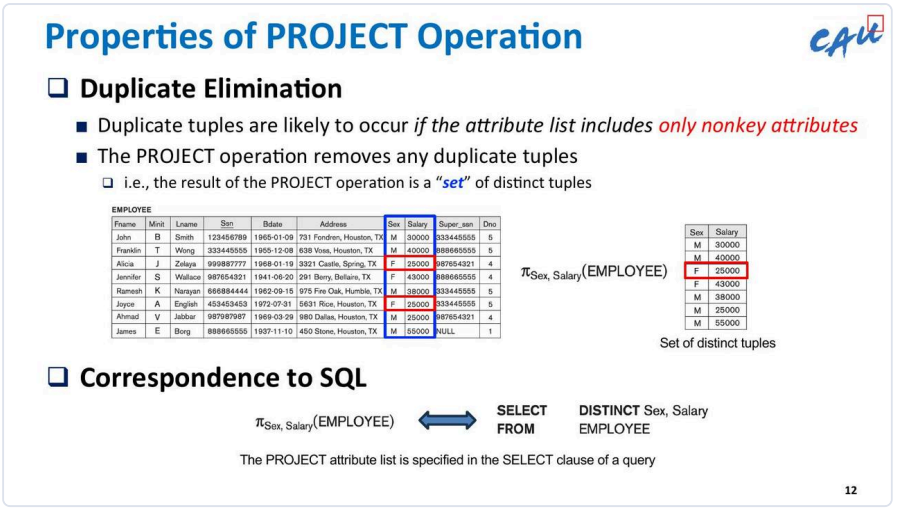


06 p.9 · 선택 조건을 AND/OR로 연결한 σ

§ PROJECT (π)

π<속성 리스트>(R): 원하는 **열만** 남기고 나머지 열은 버림 → **수직 필터링**.

- 결과 차수 = 속성 리스트의 속성 수
- **중복 제거**: 속성 리스트에 키가 없으면(비키 속성만) 중복 투플이 생길 수 있는데, **π는 중복을 제거** → 결과는 서로 다른 투플의 **집합**
  - 예) π Sex, Salary (EMPLOYEE) → 8행이 아니라 **7행** ((F, 25000)이 두 번 → 한 번)
- SQL 대응: π 속성 리스트 = **SELECT 절**. 그런데 SQL은 중복을 안 지우므로 정확한 대응은 `SELECT DISTINCT Sex, Salary FROM EMPLOYEE;`

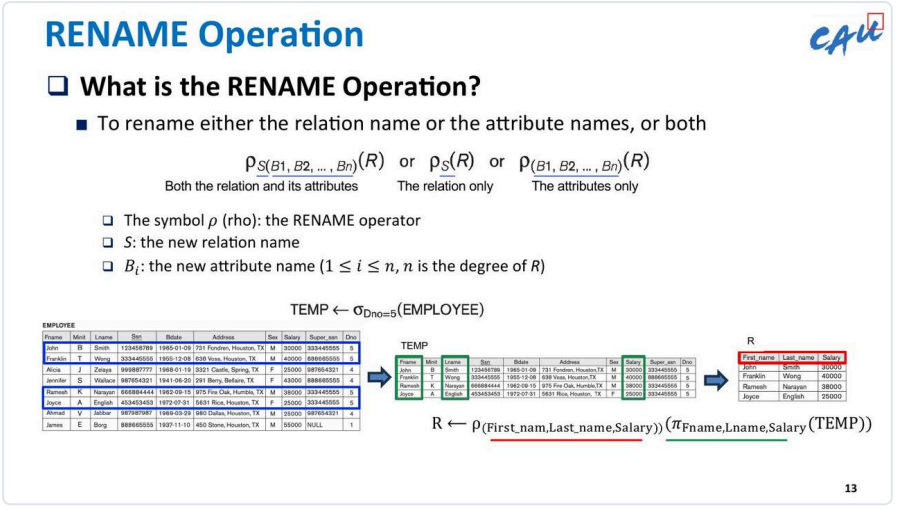


06 p.12 · π Sex, Salary: 중복 제거 → SQL의 SELECT DISTINCT

§ RENAME (ρ)

ρ: 릴레이션 이름이나 속성 이름, 또는 둘 다 바꿈.

- ρ S(B₁, ..., Bₙ)(R): 둘 다 / ρ S(R): 릴레이션 이름만 / ρ (B₁, ..., Bₙ)(R): 속성 이름만
- 대입 형태로도 씀: `R(First_name, Last_name, Salary) ← π Fname, Lname, Salary (TEMP)`
- SQL에서는 **AS (별칭)** 로 이름 바꾸기를 한다



06 p.13 · RENAME ρ의 세 가지 형태

§ 연산의 순서 (중간 릴레이션)

여러 연산을 쓸 때 두 방법:

- **방법 1**: 중간 릴레이션에 이름을 붙여 순서를 명시

```
DEP5_EMPS ← σ Dno=5 (EMPLOYEE)
RESULT    ← π Fname, Lname, Salary (DEP5_EMPS)
```

- **방법 2**: 연산을 **중첩**해서 하나의 식으로

```
π Fname, Lname, Salary ( σ Dno=5 (EMPLOYEE) )
```

→ Smith 30000, Wong 40000, Narayan 38000, English 25000

복잡한 질의는 하나의 식보다 **중간 결과 릴레이션으로 쪼개는 게 보통 더 쉽다**.

집합 연산 · 카티션 곱

교안 범위: 6-1 p.17–22

§ 합 · 교 · 차

합집합 R ∪ S, **교집합 R ∩ S**, **차집합 R − S** (MINUS, EXCEPT). 결과에서 **중복 제거**.

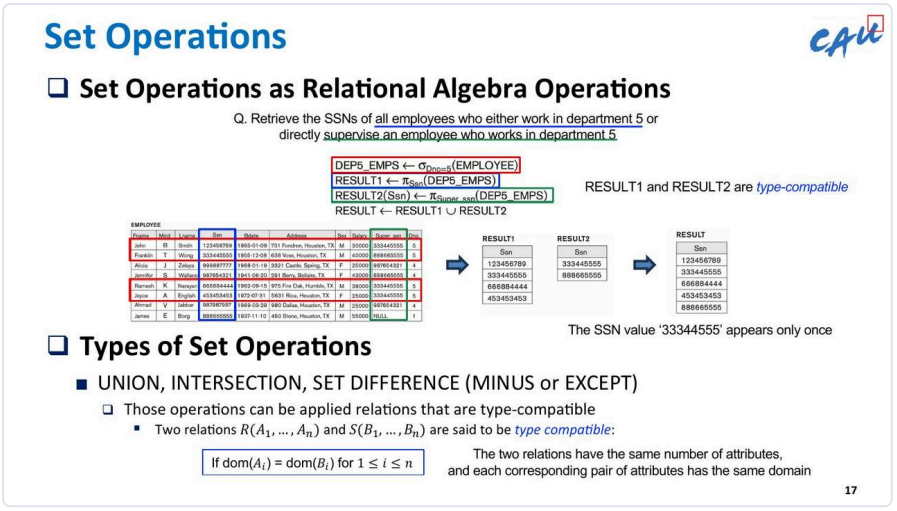
- **타입 호환 (type compatible, union compatible)** 인 릴레이션에만 적용: R(A_1…A_n)과 S(B_1…B_n)의 **속성 수가 같고**, 모든 i에 대해 **dom(A_i) = dom(B_i)**
- 교안 예: 5번 부서에서 일하거나, 5번 부서 사원을 직접 감독하는 사원의 SSN

```
DEP5_EMPS ← σ Dno=5 (EMPLOYEE)
RESULT1   ← π Ssn (DEP5_EMPS)
RESULT2(Ssn) ← π Super_ssn (DEP5_EMPS)
RESULT    ← RESULT1 ∪ RESULT2
```

→ RESULT1 = {123456789, 333445555, 666884444, 453453453}, RESULT2 = {333445555, 888665555}, RESULT = 5개 (333445555는 한 번만)

성질

- ∪와 ∩는 **교환** (R ∪ S = S ∪ R) **결합** (R ∪ (S ∪ T) = (R ∪ S) ∪ T)
- **−는 교환 법칙이 성립하지 않는다**: R − S ≠ S − R (STUDENT − INSTRUCTOR와 INSTRUCTOR − STUDENT는 다름)
- 결과 속성 이름은 관례상 **첫 번째 릴레이션**의 이름을 따름



06 p.17 · RESULT1 ∪ RESULT2 (타입 호환)

§ 카티션 곱 (×)

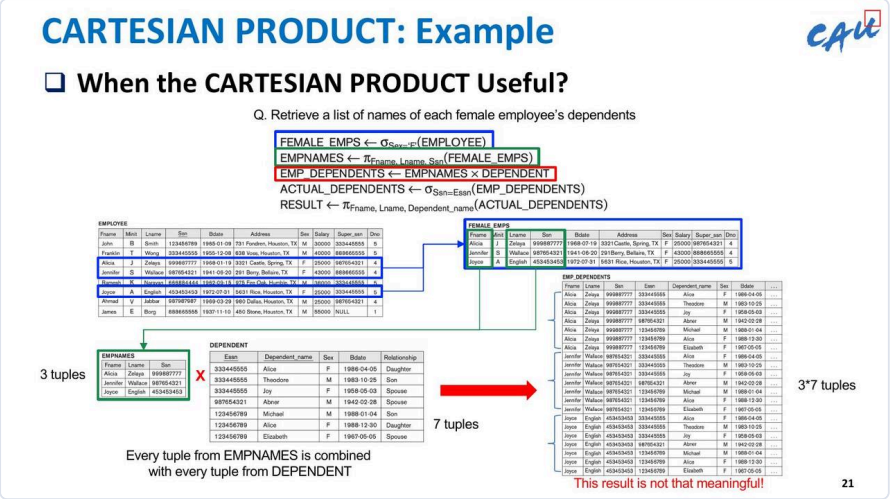
R(A_1…A_n) × S(B_1…B_m) → Q(A_1…A_n, B_1…B_m): 두 릴레이션 투플의 **모든 가능한 조합**을 이어 붙임. CROSS PRODUCT, CROSS JOIN.

- **타입 호환 아니어도** 적용 가능한 이항 집합 연산
- 결과 속성 수 = **n + m**, 투플 수 = **|R| × |S|**
- 그 자체로는 대개 의미 없음 → **뒤에 SELECT가 와서** 관련 있는 투플만 고를 때 유용

교안 예: 여성 사원의 부양가족 이름

```
FEMALE_EMPS       ← σ Sex='F' (EMPLOYEE)
EMPNAMES          ← π Fname, Lname, Ssn (FEMALE_EMPS)
EMP_DEPENDENTS    ← EMPNAMES × DEPENDENT            -- 3 × 7 = 21행 (의미 없음)
ACTUAL_DEPENDENTS ← σ Ssn=Essn (EMP_DEPENDENTS)
RESULT            ← π Fname, Lname, Dependent_name (ACTUAL_DEPENDENTS)
```

→ Jennifer Wallace – Abner 한 행. **× 다음 σ = JOIN** (다음 레슨).



06 p.21 · EMPNAMES × DEPENDENT = 3 × 7 = 21행

JOIN · DIVISION · 완전 집합

교안 범위: 6-1 p.24–34

§ JOIN (⋈)

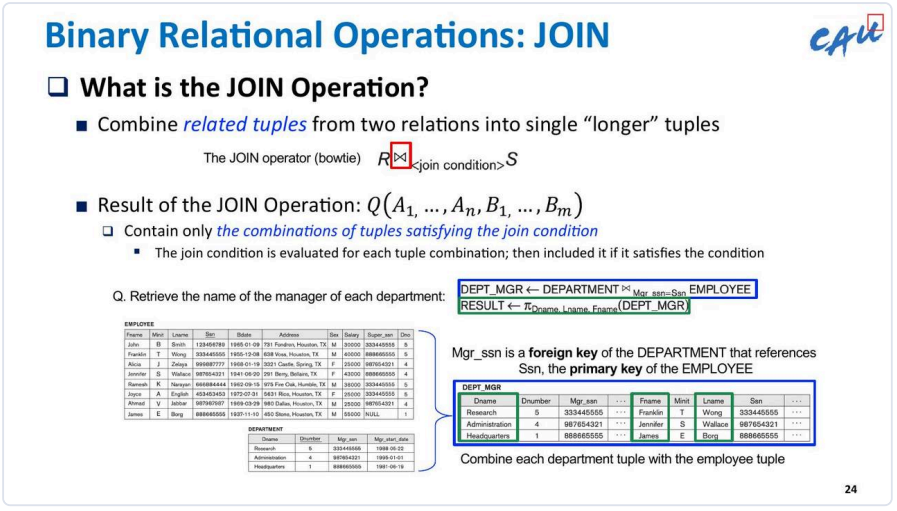
R ⋈<조인 조건> S: 두 릴레이션의 **관련 있는 투플을 하나의 긴 투플로** 결합. 결과 Q(A₁…Aₙ, B₁…Bₘ)에는 **조인 조건을 만족하는 조합만**.

- 예) 각 부서 관리자의 이름

```
DEPT_MGR ← DEPARTMENT ⋈ Mgr_ssn=Ssn EMPLOYEE
RESULT   ← π Dname, Lname, Fname (DEPT_MGR)
```

→ Research–Wong, Administration–Wallace, Headquarters–Borg

- **JOIN = 카티션 곱 + SELECT**: `EMP_DEPENDENTS ← EMPNAMES × DEPENDENT; ACTUAL ← σ Ssn=Essn (EMP_DEPENDENTS)` 두 줄을 `ACTUAL ← EMPNAMES ⋈ Ssn=Essn DEPENDENT` 한 줄로

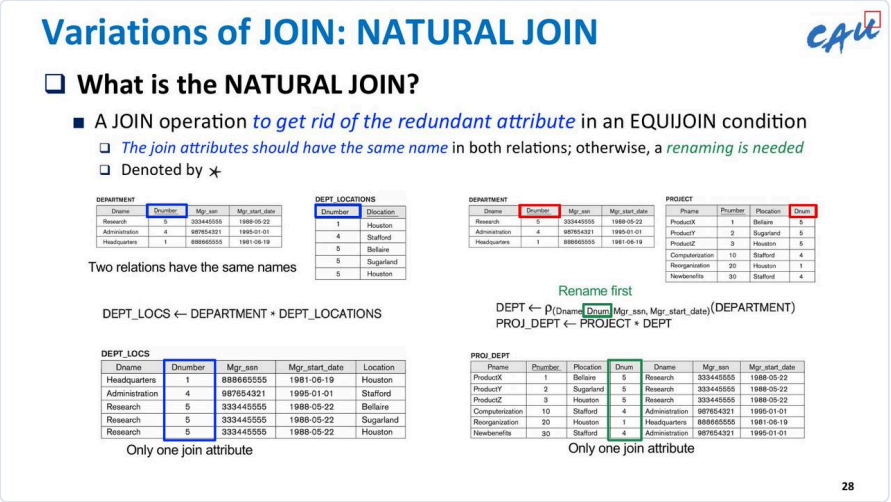


06 p.24 · DEPARTMENT ⋈ Mgr_ssn=Ssn EMPLOYEE

§ 세타 · 동등 · 자연 조인

- **세타 조인 (THETA JOIN)**: 일반 조인 조건. `<조건> AND <조건> AND …`, 각 조건은 **Aᵢ θ Bⱼ** (θ ∈ {=, <, ≤, >, ≥, ≠}). 조건을 모두 만족하는 투플만
- **동등 조인 (EQUIJOIN)**: 비교 연산자가 **= 만** 쓰인 조인. 결과에 값이 똑같은 속성 쌍이 생김 (예: Mgr_ssn과 Ssn) → 하나는 불필요
- **자연 조인 (NATURAL JOIN, * 로 표기)**: 동등 조인에서 **중복 속성 하나를 없앤 것**
  - 조인 속성은 두 릴레이션에서 **이름이 같아야** 함. 다르면 **먼저 이름을 바꿈**
  - `DEPT_LOCS ← DEPARTMENT * DEPT_LOCATIONS` (Dnumber가 같은 이름) → Dnumber 한 번만
  - `DEPT ← ρ (Dname, Dnum, Mgr_ssn, Mgr_start_date) (DEPARTMENT); PROJ_DEPT ← PROJECT * DEPT` (Dnum으로 맞춤)

함정: 자연 조인은 이름이 같은 속성 **전부**를 조인 조건으로 쓴다. 의도치 않게 이름이 같은 속성(예: 둘 다 Name)이 있으면 엉뚱한 조건이 붙는다.



06 p.28 · 자연 조인 (*): 이름이 같은 속성으로, 다르면 먼저 RENAME

§ 조인 결과의 크기

- **0 ≤ |R ⋈ S| ≤ |R| × |S|**
- 최소: 조인 조건을 만족하는 조합이 하나도 없으면 → **빈 릴레이션**
- 최대: 모든 조합이 조건을 만족하거나 조건이 없으면 → **카티션 곱**과 같음
- **조인 선택도 (join selectivity)** = 예상 조인 결과 크기 / (|R| × |S|). 질의 최적화에서 질의 계획의 비용 추정에 씀

§ DIVISION (÷)

"**모든** ~를 하는" 질의를 위한 연산. 기본 연산으로 표현할 수 있지만 따로 둠.

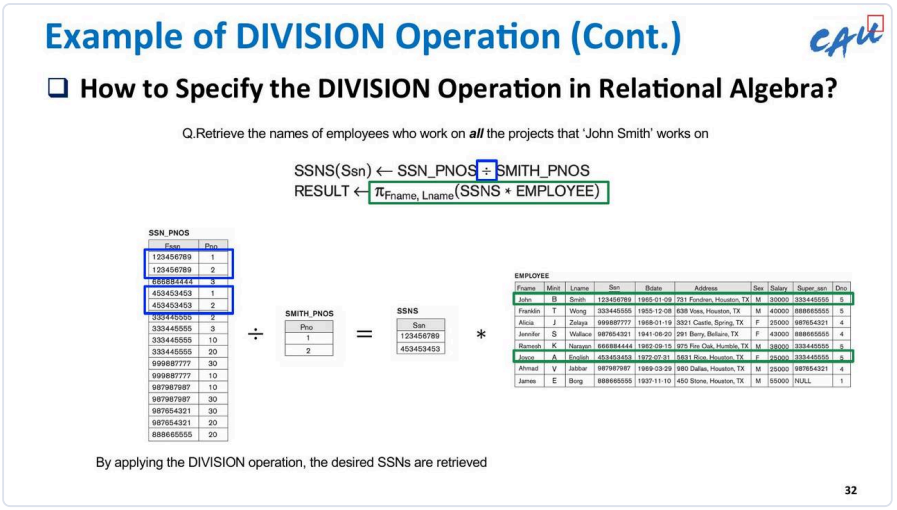
- 형식: **R(Z) ÷ S(X) = T(Y)**, X ⊆ Z (S의 속성은 R 속성의 부분집합), **Y = Z − X**
- T에는 R에서 **S의 모든 값과 짝지어 나타나는** Y 값만 들어감 (분모 S가 분자 R을 제한)
- 예) R(A, B) = {(a1,b1), (a2,b1), (a3,b1), (a4,b1), (a1,b2), (a3,b2), (a2,b3), (a3,b3), (a4,b3), (a1,b4), (a2,b4), (a3,b4)}, S(A) = {a1, a2, a3} → T(B) = {b1, b4}

교안 예: 'John Smith'가 일하는 **모든** 프로젝트에서 일하는 사원의 이름

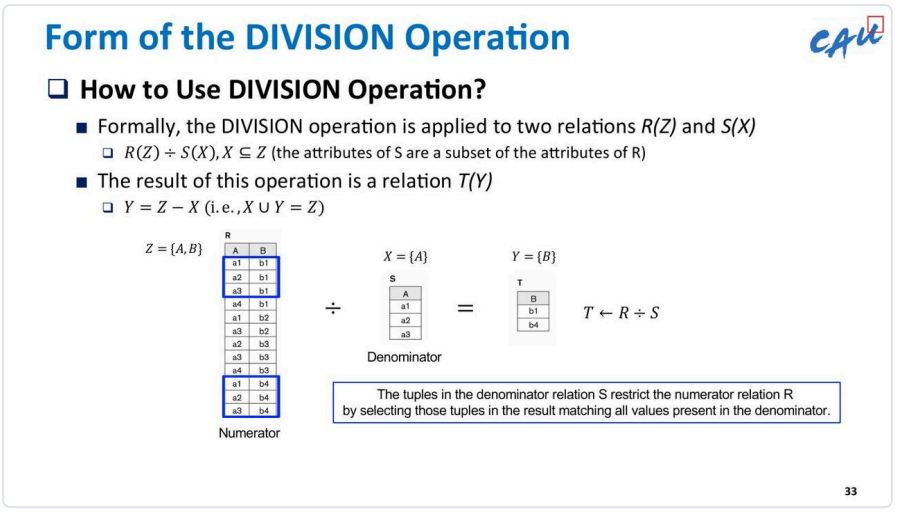
```
SMITH      ← σ Fname='John' AND Lname='Smith' (EMPLOYEE)
SMITH_PNOS ← π Pno (WORKS_ON ⋈ Essn=Ssn SMITH)            -- {1, 2}
SSN_PNOS   ← π Essn, Pno (WORKS_ON)
SSNS(Ssn)  ← SSN_PNOS ÷ SMITH_PNOS                         -- {123456789, 453453453}
RESULT     ← π Fname, Lname (SSNS * EMPLOYEE)
```

→ John Smith, Joyce English

> 시험 포인트: 분자 R의 속성 순서·이름을 맞춰야 한다. SSN_PNOS(Essn, Pno) ÷ SMITH_PNOS(Pno) → 남는 속성 Essn. 결과를 EMPLOYEE와 자연 조인하려고 SSNS(Ssn)으로 이름을 바꿨다.



06 p.32 · SSN_PNOS ÷ SMITH_PNOS



06 p.33 · R(Z) ÷ S(X) = T(Y), Y = Z − X

---

§ 관계 대수 연산의 완전 집합

{σ, π, ∪, ρ, −, ×} 6개가 완전 집합. 다른 모든 (원래의) 관계 대수 연산은 이 6개의 조합으로 표현 가능.

- **교집합**: R ∩ S ≡ (R ∪ S) − ((R − S) ∪ (S − R))
- **조인**: R ⋈<조건> S ≡ σ<조건>(R × S)
- **자연 조인**: 카티션 곱 앞에 RENAME, 뒤에 SELECT와 PROJECT
- 엄밀히는 ∩와 ⋈가 필요 없지만 **자주 쓰여서** 별도 연산으로 둔다

Complete Set of Relational Algebra Operations

❑ A Complete Set of Relational Algebra Operations

- The following six operations are elements of this set

$$\{\sigma, \pi, \cup, \rho, -, \times\}$$

- ❑ Any other original relational algebra operations can be expressed as a sequence of these operations

INTERSECTION

$$R \cap S \equiv (R \cup S) - ((R - S) \cup (S - R))$$

Strictly, **INTERSECTION** and **JOIN** are not required.

However, they are important to include as separate operations since they are very commonly used in DB applications

JOIN

$$R \bowtie_{<\text{condition}>} S \equiv \sigma_{<\text{condition}>}(R \times S)$$

JOIN = **CARTESIAN PRODUCT** followed by **SELECT**

NATURAL JOIN can be specified as a **CARTESIAN PRODUCT**, preceded by **RENAME** and followed by **SELECT** and **PROJECT**

34

06 p.34 · 완전 집합 {σ, π, ∪, ρ, −, ×}

Takeaways: Relational Algebra Operations

	OPERATION	PURPOSE	NOTATION
Unary Relational Operations	SELECT	Selects all tuples that satisfy the selection condition from a relation R.	$\sigma_{<\text{selection condition}>}(R)$
	PROJECT	Produces a new relation with only some of the attributes of R, and removes duplicate tuples.	$\pi_{<\text{attribute list}>}(R)$
Binary Relational Operations	THETA JOIN	Produces all combinations of tuples from R_1 and R_2 that satisfy the join condition.	$R_1 \bowtie_{<\text{join condition}>} R_2$
	EQUIJOIN	Produces all the combinations of tuples from R_1 and R_2 that satisfy a join condition with only equality comparisons.	$R_1 \bowtie_{<\text{join condition}>} R_2$, OR $R_1 \bowtie_{(<\text{join attributes 1}>),(<\text{join attributes 2}>)} R_2$
	NATURAL JOIN	Same as EQUIJOIN except that the join attributes of R_2 are not included in the resulting relation; if the join attributes have the same names, they do not have to be specified at all.	$R_1 *_{<\text{join condition}>} R_2$, OR $R_1 *_{(<\text{join attributes 1}>),(<\text{join attributes 2}>)} R_2$ OR $R_1 * R_2$
Set Operations	UNION	Produces a relation that includes all the tuples in R_1 or R_2 or both R_1 and R_2; R_1 and R_2 must be union compatible.	$R_1 \cup R_2$
	INTERSECTION	Produces a relation that includes all the tuples in both R_1 and R_2; R_1 and R_2 must be union compatible.	$R_1 \cap R_2$
	DIFFERENCE	Produces a relation that includes all the tuples in R_1 that are not in R_2; R_1 and R_2 must be union compatible.	$R_1 - R_2$
	CARTESIAN PRODUCT	Produces a relation that has the attributes of R_1 and R_2 and includes as tuples all possible combinations of tuples from R_1 and R_2.	$R_1 \times R_2$

44

06 p.44 · 관계 대수 연산 요약표

추가 연산: 집계 · 그룹 · 외부 조인

교안 범위: 6-1 p.36–43

§ 집계 함수와 그룹화 (ℱ)

기본 연산만으로는 **집계(SUM, AVERAGE, MAXIMUM, MINIMUM, COUNT)**, **그룹별 집계**, **외부 조인**을 표현 못 함 → 추가 연산으로 **표현력 확장**.

<그룹화 속성> ℱ <함수 리스트> (R) (ℱ = script F)

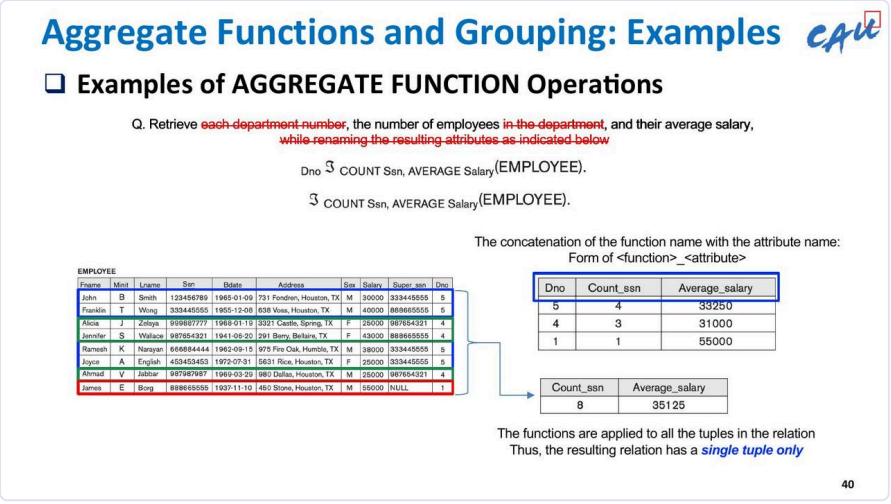
- 그룹화 속성: R의 속성 중 그룹을 나눌 기준 (없어도 됨)
- 함수 리스트: (<함수> <속성>) 쌍들. 함수 = SUM, AVERAGE, MAXIMUM, MINIMUM, COUNT
- **결과**: 그룹화 속성 + 함수 리스트 원소마다 속성 하나

예

```
ρ R(Dno, No_of_employees, Average_sal) ( Dno ℱ COUNT Ssn, AVERAGE Salary (EMPLOYEE) )
```

→ (5, 4, 33250), (4, 3, 31000), (1, 1, 55000)

- **이름을 안 바꾸면**: 속성 이름 = `<함수>_<속성>` → Dno, Count_ssn, Average_salary
- **그룹화 속성이 없으면**: 함수를 **모든 투플에** 적용 → 결과는 **투플 하나** `ℱ COUNT Ssn, AVERAGE Salary (EMPLOYEE)` → (8, 35125)



06 p.40 · 이름 안 바꾼 결과(Count_ssn, Average_salary)와 그룹화 속성 없는 경우

§ 집계의 성질

- 집계 함수를 적용할 때 **중복은 제거하지 않는다** (급여 합·평균을 제대로 구해야 하니까)
- **NULL은 집계에서 고려하지 않는다**
- 관계 대수는 **닫힌 수학 체계**: 집계 결과도 스칼라 숫자가 아니라 **릴레이션** (값 하나, 투플 하나여도)

§ 외부 조인

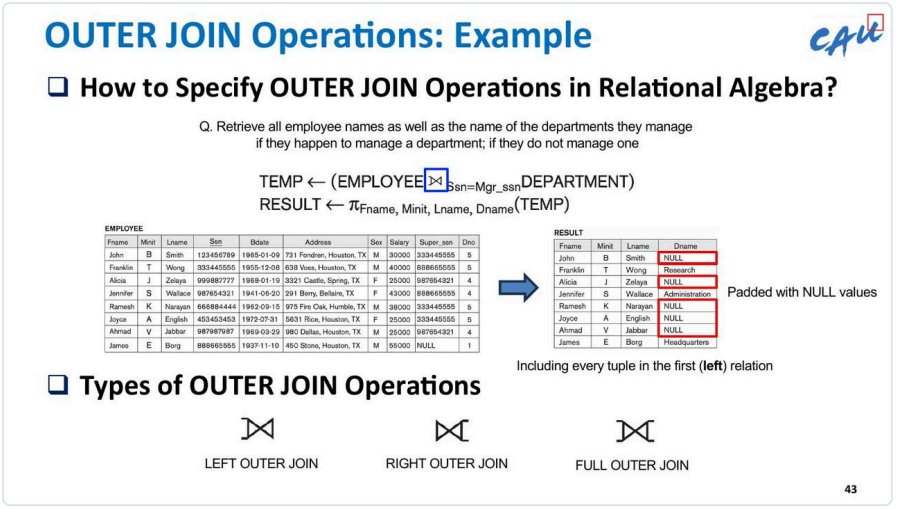
EQUIJOIN·NATURAL JOIN(내부 조인)은 **조건을 만족하는 투플만** 결과에 넣고, 짝이 없는 투플은 버린다. 짝이 없어도 **한쪽(또는 양쪽) 릴레이션의 모든 투플을 남기고 싶을 때** 외부 조인을 쓴다.

- 예) 모든 사원 이름과, 그 사원이 부서를 관리하면 그 부서 이름 (관리 안 하면 NULL)

```
TEMP   ← EMPLOYEE ⋈ Ssn=Mgr_ssn DEPARTMENT
RESULT ← π Fname, Minit, Lname, Dname (TEMP)
```

→ 8행. Wong–Research, Wallace–Administration, Borg–Headquarters, 나머지 5명은 Dname = NULL

- 종류: **LEFT OUTER JOIN** ⟕ (왼쪽 모든 투플), **RIGHT OUTER JOIN** ⟖ (오른쪽 모든 투플), **FULL OUTER JOIN** ⟗ (양쪽 모두). 짝 없는 쪽 속성은 **NULL로 채움**



06 p.43 · EMPLOYEE ⋈ Ssn=Mgr_ssn DEPARTMENT

관계 대수 작성 연습

교안 범위: db-exercises Week 7 (p.55–59)

§ 연습 1–2: 조인 경로

연습 1: 'Research' 부서에서 일하는 모든 사원의 이름과 주소

```
RESEARCH_DEPT ← σ Dname='Research' (DEPARTMENT)
RESEARCH_EMPS ← RESEARCH_DEPT ⋈ Dnumber=Dno EMPLOYEE
RESULT        ← π Fname, Lname, Address (RESEARCH_EMPS)
```

연습 2: 'Stafford'에 있는 프로젝트마다 프로젝트 번호, 통제 부서 번호, 관리자의 성·주소·생일

```
STAFFORD_PROJS ← σ Plocation='Stafford' (PROJECT)
CONTR_DEPTS    ← STAFFORD_PROJS ⋈ Dnum=Dnumber DEPARTMENT
PROJ_DEPT_MGRS ← CONTR_DEPTS ⋈ Mgr_ssn=Ssn EMPLOYEE
RESULT         ← π Pnumber, Dnum, Lname, Address, Bdate (PROJ_DEPT_MGRS)
```

→ (10, 4, Wallace, 291 Berry, Bellaire, TX, 1941-06-20), (30, 4, Wallace, ...)

요령: 조건이 있는 릴레이션에서 **σ를 먼저** → FK–PK를 따라 ⋈ → 마지막에 π.

§ 연습 3: 합집합

성이 'Smith'인 사원이 일꾼으로든, 프로젝트를 통제하는 부서의 관리자로든 관련된 프로젝트 번호

```
SMITHS(Essn)        ← π Ssn (σ Lname='Smith' (EMPLOYEE))
SMITH_WORKER_PROJS  ← π Pno (WORKS_ON * SMITHS)
MGRS                ← π Lname, Dnumber (EMPLOYEE ⋈ Ssn=Mgr_ssn DEPARTMENT)
SMITH_MANAGED_DEPTS(Dnum) ← π Dnumber (σ Lname='Smith' (MGRS))
SMITH_MGR_PROJS(Pno)      ← π Pnumber (SMITH_MANAGED_DEPTS * PROJECT)
RESULT              ← SMITH_WORKER_PROJS ∪ SMITH_MGR_PROJS
```

- ∪를 하려고 두 결과의 속성 이름을 **Pno로 맞춤** (타입 호환)
- SMITHS(Essn)으로 이름을 바꿔서 WORKS_ON과 **자연 조인**이 Essn으로 걸리게 함

§ 연습 4–5: 차집합과 교집합

연습 4: 부양가족이 **없는** 사원의 이름

```
ALL_EMPS             ← π Ssn (EMPLOYEE)
EMPS_WITH_DEPS(Ssn)  ← π Essn (DEPENDENT)
EMPS_WITHOUT_DEPS    ← ALL_EMPS - EMPS_WITH_DEPS
RESULT               ← π Lname, Fname (EMPS_WITHOUT_DEPS * EMPLOYEE)
```

연습 5: 부양가족이 **하나 이상 있는 관리자**의 이름

```
MGRS(Ssn)            ← π Mgr_ssn (DEPARTMENT)
EMPS_WITH_DEPS(Ssn)  ← π Essn (DEPENDENT)
MGRS_WITH_DEPS       ← MGRS ∩ EMPS_WITH_DEPS
RESULT               ← π Lname, Fname (MGRS_WITH_DEPS * EMPLOYEE)
```

→ Wong, Wallace

시험 포인트: "~가 없는" = **차집합 (−)**, "A이면서 B" = **교집합 (∩)**, "A이거나 B" = **합집합 (∪)**, "모든 ~" = **나눗셈 (÷)**. SQL의 NOT EXISTS / EXISTS AND EXISTS / UNION / NOT EXISTS(EXCEPT)와 1:1로 대응시켜 외워.